Grupo 6

Compañia Logistica

5°3° Computación

Turno Mañana

Integrantes: Benjamin Elias Saragusti,Santiago Goldfarb,

Santiago Pedraza, Xiomara Ureña, Nicolas Rosato

INICIO DE LA CARPETA

24/06/2026

Este día entre todos propusimos preguntas para usarlas de forma de relevamiento, siendo asignados por los profesores las necesidades del proyecto, el tema a elegir, que terminamos eligiendo el proyecto de logistica, cada uno fue buscando entre sus personas cercanas para ver si alguien conseguía fuente de relevamiento. Esto se extendió hasta el 26, la parte de agregar preguntas para conseguir una fuente variable del relevamiento, en vez de preguntas cortas.

26/06/2026

Goldfarb intentó encontrar una entrevista con mercado libre, por medio de un familiar aunque esta nunca pudo llegar a hacerse por motivos no especificados.

01/07/2026

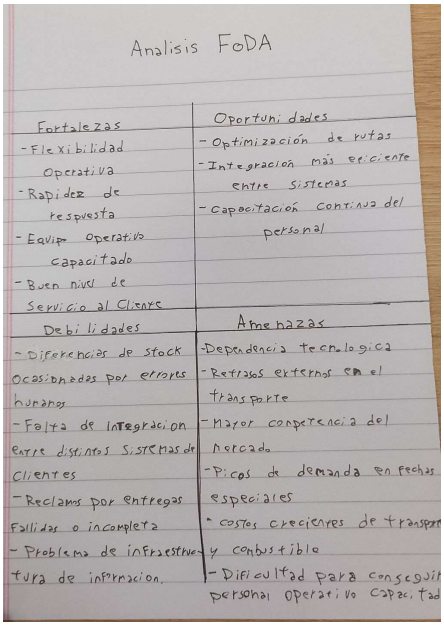
Benjamin empezó a mandar mails a distintas compañías, siendo 6 empresas de logística, enviando todas las preguntas recogidas por todos los integrantes del equipo, siendo que a los emails solo 1 de las compañías le contestó, solicitando información personal para poder corroborar el hecho de que fuera estudiante y que necesitaba esta informacion para un proyecto escolar.

03/07/2026

Se consiguió el certificado de alumno regular, pudiendo mandarlo a las autoridades correspondientes para recibir la información, el mismo dia recibimos un mail con toda la información que les habíamos consultado, que entre otros estaban los datos de las personas involucradas en el transporte, como funcionaba el sistema, problemáticas y cosas a mejorar, y las cosas buenas del sistema para poder hacer los análisis previos y tener una base para el futuro codificamiento del programa.

06/07/2026

Los integrantes Goldfarb y Saragusti realizaron a la mañana el análisis foda, separando la información útil de la no en el documento previamente recibido de la compañía de logística, dejando los datos planteados en el cuadro, en una hoja, con su respectiva organización.


(Fotografia de como quedo el Análisis foda, realizado en base a los datos obtenidos, discriminando la mayoria de informacion que iria a uso en próximas secciones).

Ademas este dia a la tarde se empezó con la determinación de las clases para JAVA UML, siendo estas: Persona, cliente, proveedor, transporte analista, seguimiento, producto, tecnología, decoración, indumentaria, accesorios, envios, vehículo avión , barco tren, camión, empresa y empresa adherida.

Estas fueron propuestas entre todos, en base al relevamiento previamente hecho, debido que en este aparecen las personas que trabajan en la empresa, por lo que nos inspiramos en este para ponerlas.

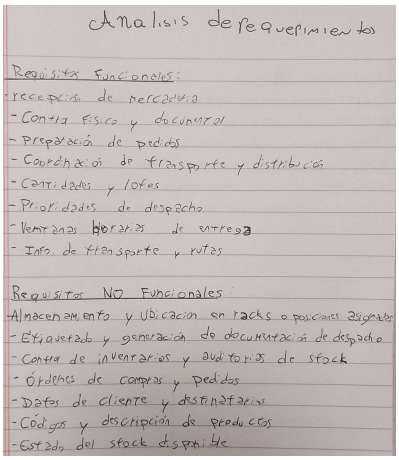
07/07/2026

Se dividieron las clases, en este caso quienes tenían que agregar campos a cada una de las clases, quedando distribuidas de la siguiente forma:

Goldfarb persona e hijas, Benjamin Empresa, Xiomara Envios,Rosato Empresas adheridas, Pedraza productos e hijas, Zownir vehículos e hijas y Goldfarb/Benjamin Paquetes

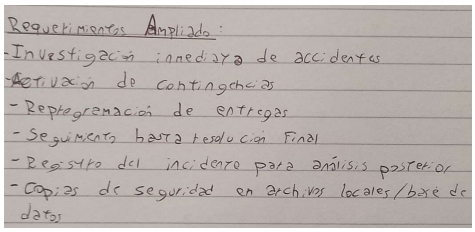
También se tenían que pactar sus métodos.

En la hora de base de datos se realizó el análisis de requerimientos, quedando los funcionales, no funcionales y los extras que venían a agregar funcionalidades que no eran indispensables.


Analisis de requerimientos

Requisitos Funcionales:
- Recepción de mercadería
- Control físico y documental
- Preparación de pedidos
- Coordinación de transporte y distribución
- Cantidades y lotes
- Prioridades de despacho
- Ventanas horarias de entrega
- Info. de transporte y rutas

Requisitos NO Funcionales:
- Almacenamiento y Ubicación en racks o posiciones asignadas
- Etiquetado y generación de documentación de despacho
- Control de inventarios y auditorías de stock
- Órdenes de compras y pedidos
- Datos de cliente y destinatario
- Códigos y descripción de productos
- Estado del stock disponible


Requerimientos Ampliado:
- Investigación inmediata de accidentes
- Activación de contingencias
- Reprogramación de entregas
- Seguimiento hasta resolución final
- Registro del incidente para análisis posterior
- Copias de seguridad en archivos locales/base de datos

Benjamin en esa misma hora hizo su clase, empresa, en una hoja, con sus componentes y sus métodos, después goldfarb hizo sus clases siendo personas e hijas. Después nos juntamos para hacer la clase paquetes, con sus atributos y los métodos.

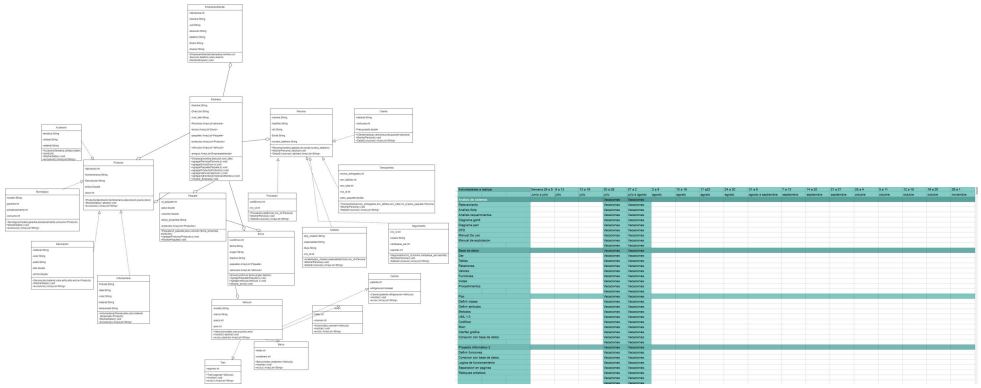
11/07/2026 al 12/07/2026

Los integrantes restantes en este periodo de tiempo fueron desglosando cada uno de sus clases, colocando sus respectivos métodos necesarios y atributos que las definan para que sean de utilidad para el programa, siendo que la fecha máxima pactada fue el domingo 12, teniendo para esa fecha todas las hojas con todos los atributos y métodos predispuestos.

13/07/2026

Este dia Benjamin se encargó de hacer el diagrama gant, la primera version que despues se fue organizando y actualizando con toda la información de los plazos de los tiempos que es lo que se ve a todo lo demás, teniendo poca información de todo lo que se hacia por materia porque en ese momento todavía no lo habían explicado todo. Además ese día fue haciendo el

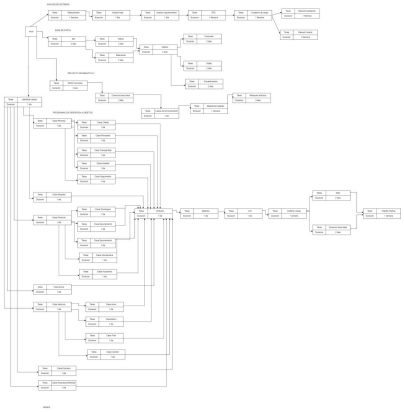
UML basándose de todas las hojas hechas por los integrantes, normalizando el diseño para tenerlo todo en un mismo lugar en vez de cada parte dividida por hojas.



(El uml a la izquierda, la primera versión sola obvio, y despues el gantt de ese entonces)

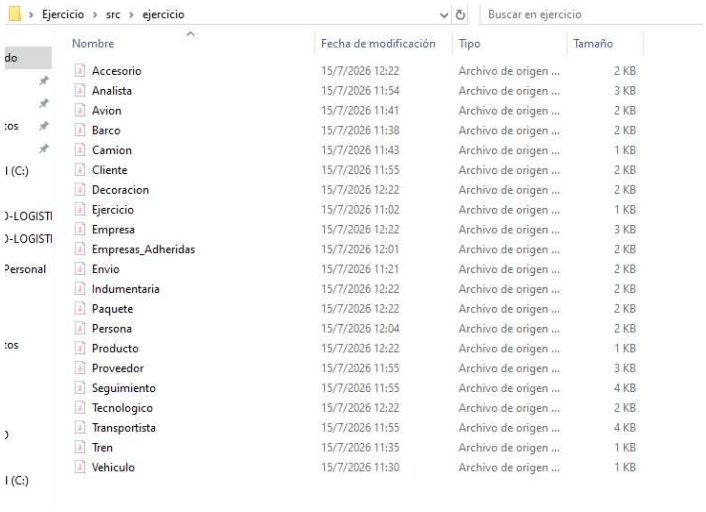
14/07/2026

Benjamin realizó el PERT basándose en cómo se veía el gantt, que fue más difícil de adaptar con el tiempo debido a su tipo de archivo, además este diagrama se fijaba más en la cantidad y las dependencias de los trabajos, siendo diagramado con burbujas.



15/07/2026

Este dia fue un dia organizativo, en el cual cada uno realizó la codificación de sus clases correspondientes, pasando lo puesto en palabras a código de abstracción o herencia según el caso de java, creando la primera carpeta con todas las clases perdidas en ese momento, que en su futuro algunas fueron descartadas por ser reiterativas.



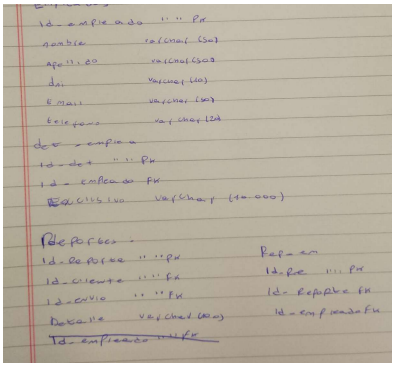
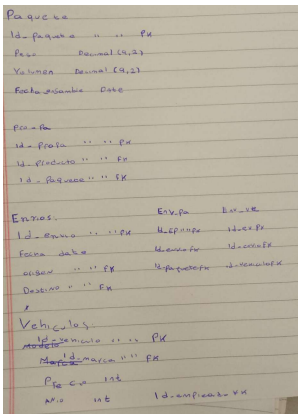
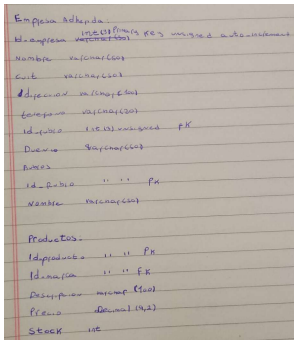
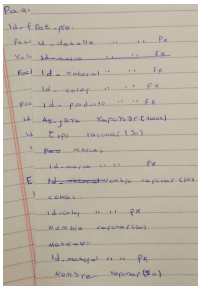
(Todas las clases, sin contar el main que estaba vacío, pero en si estaban todos los archivos con sus respectivos contenidos).

02/08/2026

Se realizó la investigación por parte de Nicolas Rosato sobre la competencia, generando el análisis de estado del arte, en el que nos comparamos con otras compañías de logística como andreani y correo argentino para poder ver nuestras ventajas y desventajas además de entender sus falencias para utilizarlas como un pie de lo que tiene que tener la nuestra.

03/08/2026

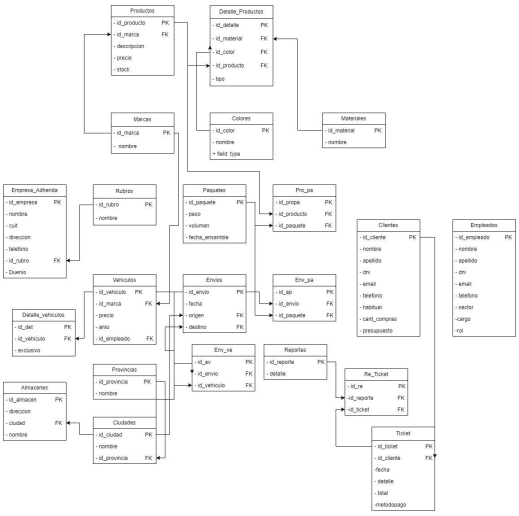
Benjamin realizó el diccionario de datos, para tener una fuente de cuales iban a ser las tablas, sus atributos campos, relaciones entre otros, para tener una base de cómo iba a después quedar el futuro DER, este contaba con todas las clases hasta el momento, dándole tablas auxiliares a algunas para integrar mas datos específicos de los usadores de las herramientas.




 (Todas las tablas del primer diccionario de datos, completas, basadas en el uml realizado previamente como inspiración).

05/08/2026

Nos hicieron la devolución de las cosas que veníamos teniendo realizadas hasta el momento, teniendo que eliminar tablas del uml y del diccionario debido a que eran redundantes y se podían reemplazar con atributos de las tablas que compensan la explicación de los tipos dentro de la base de datos y del uml. Por lo que Rosato y Zowmir se pusieron a realizar el UML actualizado, recibiendo luego unos pequeños retoque por Saragusti debido a que algunos campos estaban con otros tipos de variable pero bien. Luego Pedraza realizó el DER, siendo también corregido por Goldfarb en pequeños aspectos que fue comentando el profesor en ese momento.

Además se actualizó el diagrama Gant por parte de Xiomara y Benjamin, poniendolo al dia con las tareas hechas hasta el momento.



06/08/2026

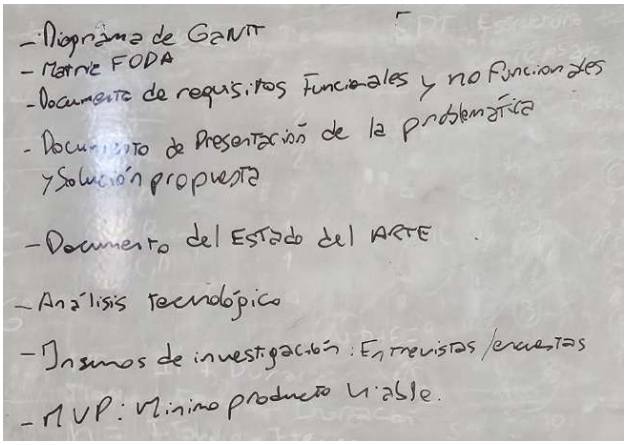
Este dia hicimos entre todos el presupuesto que formarán parte de la infraestructura de la empresa, incluyendo PCs, periféricos, routers, switches, cables, etc., para posteriormente calcular el presupuesto final. Intentamos emplear la menor cantidad de dinero posible. Pedraza busco y proporciono los links de todos los componentes y elementos (la mayoría de ellos, sacados de “Mercado Libre”). Los componentes que encontró y se terminaron implementando al listado final son las computadoras con todos sus componentes, el switch gigabit de 24 puertos, router multi wan gigabit 4 wan gigabit wan 5 puertos 1 lan, cable de red vention 2 metros 40 gbps, cable coaxil fino, 5 metros rg6. Total de todo:13121266

1	Presupuesto Compañia de logistica				
2	Computadoras :	Unidad	Cantidad	Precio total	Total de todo
3	Procesador:Intel core I5 4Gen				13121266
4	Ram: 16gb DDR3				
5	GPU:Intel graphics				
6	Disco duro: SSD 256gb				
7	Windows 10				
8	Puerto Ethernet				
9	Modelo HP PRO DESK				
10	Placa: No dice				
11	Monitor 19 pulgadas				
12	Mouse y teclado				
13	Precio	450000	26	11700000	
14	Switch				
15	Modelo Switch gigabit 24 puertos	169491	5	847455	
16	Router				
17	modelo multi wan gigabit 4 wan 5 puertos 1 lan	79999	3	239997	
18	Cables :				
19	Cable ethernet:				
20	Cable de red vention 2 metros 40gbps	11589	26	301314	
21	Cable coaxil fino				
22	5 metros rg6	6500	5	32500	

980 filas no modificadas

07/08/2026

Este dia nos encargó Gabriel las tareas que necesitábamos para sistemas, siendo las que en su momento antes de las vacaciones nos habían dado la mitad porque no sabia que íbamos a requerir mas cosas, para ese momento ya teníamos el gant, el foda, los requisitos, aunque los tuvimos que expandir a mas requisitos y lo demás lo fuimos haciendo entre todo



08/08/2026 - 09/08/2026

La distribución fue de:

Pedraza: Realizó el documento de presentación de problemática y la solución propuesta, usando de base los análisis vistos, por lo cual usó el foda, los requerimientos y el mvp para poderlo llevar a cabo

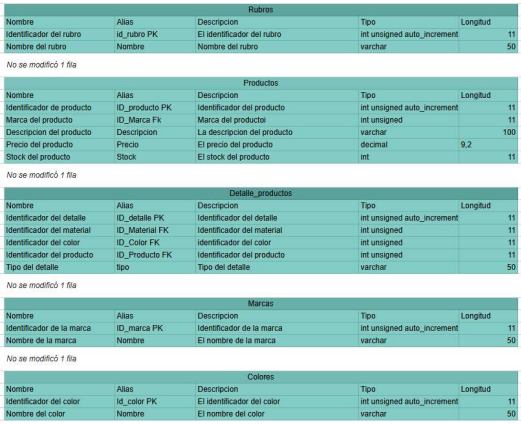
Benjamin: Realizó el diagrama edt con lo visto en clase, además de hacer una red mínima con los componentes provistos por el profesor en ese momento, con verificación de pings para comprobar su funcionalidad.

Xiomara y Nicolas:Investigaron la competencia para encontrar fortalezas y todas las cosas requeridas para evaluar qué es lo que íbamos a requerir para nuestro programa, debido a que nuestra opción tiene que ser mejor para que los clientes opten por usar nuestra pagina en vez de usar otra.

Zownir: Análisis tecnológico de los programas que usamos

12/08/2026

Saragusti: digitalice/actualice el diccionario de datos a una versión de hoja de cálculo con ya todas las cosas que necesitaba el ser actual, También estoy colaborando con la investigación de los aspectos que desconozco del proyecto informático y de sistemas ya que hay cosas que hasta el momento no vimos y que tenemos que investigar



Rubros

Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del rubro	id_rubro PK	El identificador del rubro	int unsigned auto_increment	11
Nombre del rubro	Nombre	Nombre del rubro	varchar	50

No se modificó 1 fila

Productos

Nombre	Alias	Descripcion	Tipo	Longitud
Identificador de producto	ID_producto PK	Identificador del producto	int unsigned auto_increment	11
Marca del producto	ID_Marca Fk	Marca del productoi	int unsigned	11
Descripcion del producto	Descripcion	La descripcion del producto	varchar	100
Precio del producto	Precio	El precio del producto	decimal	9,2
Stock del producto	Stock	El stock del producto	int	11

No se modificó 1 fila

Detalle_productos

Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del detalle	ID_detalle PK	Identificador del detalle	int unsigned auto_increment	11
Identificador del material	ID_Material FK	Identificador del material	int unsigned	11
Identificador del color	ID_Color FK	identificador del color	int unsigned	11
Identificador del producto	ID_Producto FK	Identificador del producto	int unsigned	11
Tipo del detalle	tipo	Tipo del detalle	varchar	50

No se modificó 1 fila

Marcas

Nombre	Alias	Descripcion	Tipo	Longitud
Identificador de la marca	ID_marca PK	Identificador de la marca	int unsigned auto_increment	11
Nombre de la marca	Nombre	El nombre de la marca	varchar	50

No se modificó 1 fila

Colores

Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del color	Id_color PK	El identificador del color	int unsigned auto_increment	11
Nombre del color	Nombre	El nombre del color	varchar	50

(Fragmento del diccionario de datos digitalizado la cual no constituye a la versión final, el resto que se actualizo se verá cuando se vuelva a mencionar el diccionario de datos)

13/08

Ureña: Este día con mi compañero y líder Golfard comenzamos con el Mvp, el boceto fue realizado en la página figma. Él creó el proyecto y me lo compartió por mail para que lo podamos editar al mismo tiempo. Al ser una página nueva que nunca usamos se nos complicó llegar a entender cómo funcionaba y poder terminar con el boceto. Lo único que pudimos hacer fueron estas dos páginas que voy a dejar a continuación. También pensamos como queremos que quede la página final pero el motivo que dije anteriormente no lo pudimos llevar a cabo.

Landing page

Nombre del sitio

Ver envíos Armar pedido Contacto Iniciar Sesión

Proyecto Logistica.name

Tu compañía de logística de confianza! Arma tus pedidos!

Armar pedido

Pon tu codigo de pedido aquí!

Ej: hola_pedido

Como armar mi pedido?

Crea el envío

Toca el botón de "Crear envío". Después, completa los datos necesarios.

Ticket

Al crear el envío, se te otorgará un ticket en la sección "Ver mis pedidos" que tendrás que imprimir y pegar sobre el paquete.

Entregar a sucursal

Llevá el paquete con el ticket pegado y listo!

Armar pedido Ver pedidos

Dos tipos de entrega

Sucursal

Se entregara el paquete en una sucursal y el cliente tendra q pasar a buscarlo. Menor precio de envio.

Domicilio

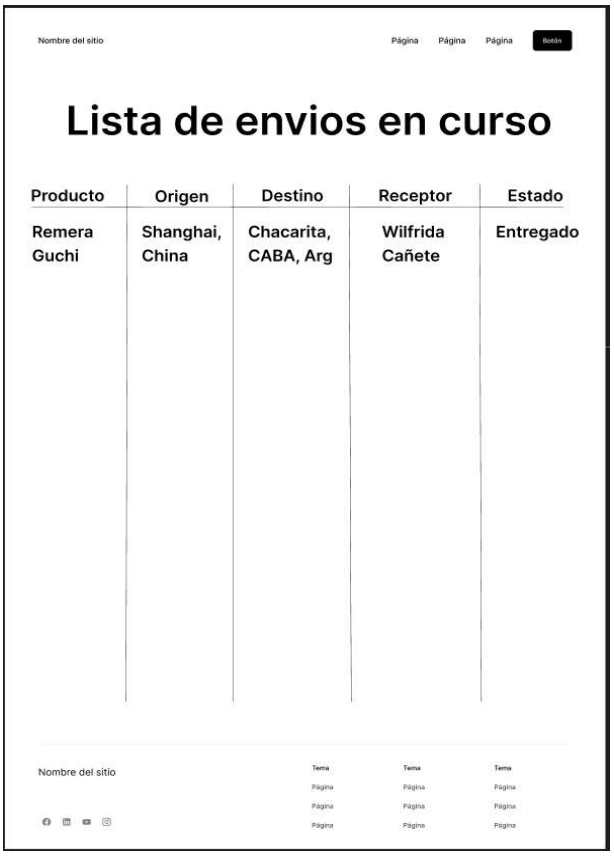
Directo a la casa del cliente. Mayor precio de envio.

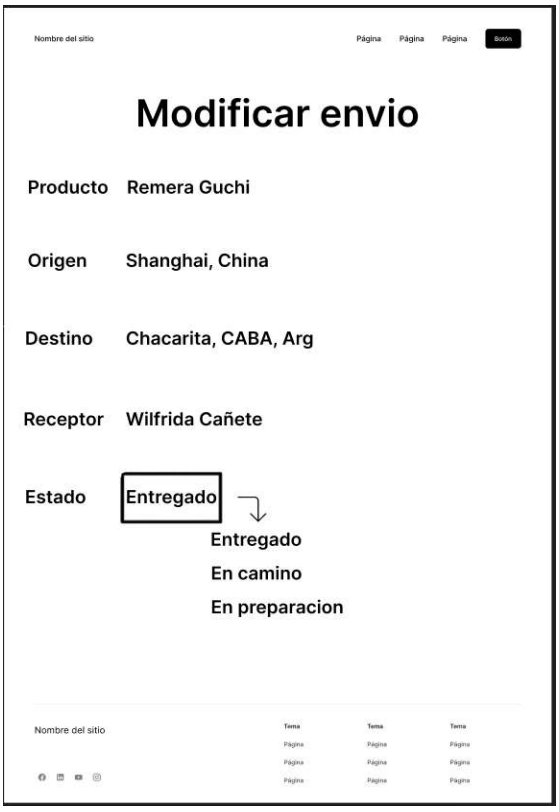
Al día siguiente, tras evaluar contratiempos se procedió a una restauración de la planificación de tareas, por lo cual la responsabilidad de continuar el desarrollo del MVP fue estratégicamente reasignada para mis compañeros Golfard y Pedraza.

17/08

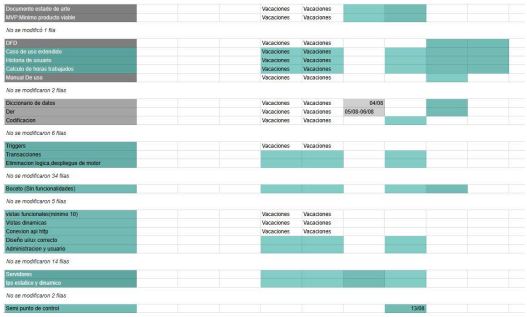
Goldfarb y Pedraza: bocetos de la pagina en figma.

ADMINISTRADOR -->
Nombre del sitio
Página
Página
Página
Botón
Seguimiento del producto
Ubicacion en tiempo real
Tiempo aproximado de llegada: 00:02:45:12
Vehiculo: Motocicleta
Destino: (direccion registrada)
Callao-Mtro. Alfredo Bravo
Uruguay
Lavalle
Av. Corrientes
Sarmiento
Catedral Metropolitana de Buenos Aires
Plaza de Mayo
Av. de Mayo
Palacio Barolo
Lima
Almacén Congreso
CiberWall Depósito
Moreno
MONSERRAT
Av. Belgrano
La Trastienda
Luna Park
Nombre del sitio
Tema
Tema
Tema
Página
Página
Página
Página
Página
Página
Página
Página
Página


Nombre del sitio
Página
Página
Página
Botón
Lista de envios en curso
Producto
Origen
Destino
Receptor
Estado
Remera Guchi
Shanghai, China
Chacarita, CABA, Arg
Wilfrida Cañete
Entregado
Nombre del sitio
Tema
Página
Página
Página
Tema
Página
Página
Página
Tema
Página
Página
Página


Nombre del sitio
Página
Página
Página
Botón
Modificar envio
Producto
Remera Guchi
Origen
Shanghai, China
Destino
Chacarita, CABA, Arg
Receptor
Wilfrida Cañete
Estado
Entregado
Entregado
En camino
En preparacion
Nombre del sitio
Tema
Página
Página
Página
Tema
Página
Página
Página
Tema
Página
Página
Página

Saragusti: modificaciones al código de java, para crear el borrado y buscado universal, además de que ajuste el uml para que quedara exactamente igual al código de las clases ya hechas, tambien actualize el gant, añadiendo las últimas tareas restantes además de actualizar el pert para que quedara igual que el gant.



(Modificaciones al gant hechas al dia del 17 de agosto)

(El borrado universal y el buscador funcionan a base de devolver un índice y no un objeto, el índice luego se aplica para obtener el resultado)

```
public int buscador_universal(int id,int opcion){
    int contador=0;
   switch(opcion){
        case 1:
            for(Cliente c:clientes){
                if(c.getIdCliente()==id){
                    return contador;
                }
                contador++;
            }
            break;
        case 2:
            for(Empleado e:empleados){
                if(e.getIdempleado()==id){
                    return contador;
                }
                contador++;
            }
            break;
        case 3:
            for (Reporte r:reportes){
                if(r.getIdReporte()==id){
                    return contador;
                }
                contador++;
            }
            break;
        case 4:
            for(Envio e :envios){
                if(e.getNumEnvio()==id){
                    return contador;
                }
                contador++;
            }
        case 5:
```

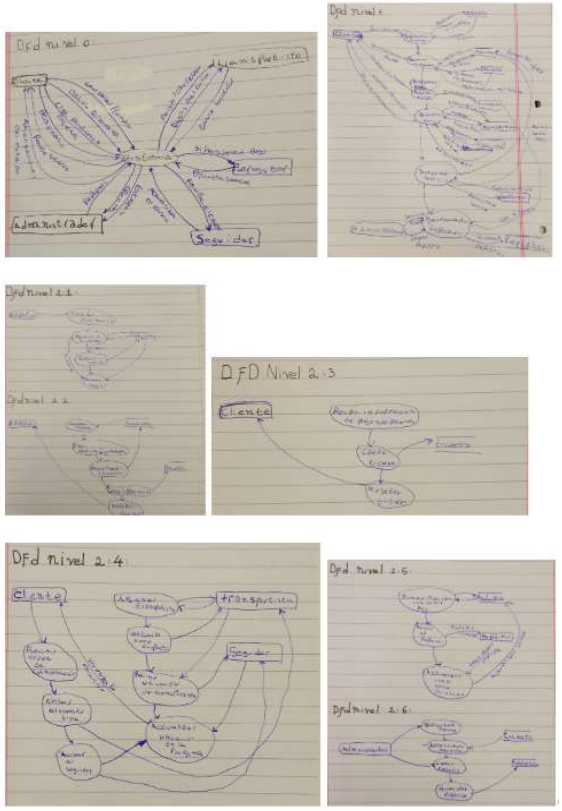
Ureña: Dados los requerimientos mi compañero Benjamin se encargó de establecer la tarea de cada uno de mis compañeros. En mi caso me tocó hacer el caso de uso extendido y el conteo de horas.

18/08

Rosato: Se dio inicio formal a la planificación temporal del proyecto mediante la construcción del Diagrama de Gantt. El propósito de este artefacto es establecer la previsión cronológica, la secuenciación de las tareas y la asignación de recursos para el desarrollo del software logístico. Para su elaboración, se adoptó un enfoque metodológico híbrido utilizando la herramienta Microsoft Excel para la matriz de tiempos y la plataforma Draw.io para la representación gráfica del mapa de dependencias.

19/08

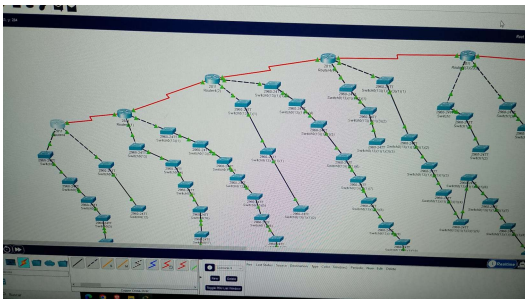
Saragusti: El miércoles 19 de agosto hice el dfd acompañado de Goldfarb y Rosato, estableciendo las clases y quedando en 6 procesos en el dfd nivel 1 luego explotadas esas burbujas para cada uno de los de nivel 2 siendo estas : Gestionar clientes, Gestionar Paquetes, Gestionar Tickets, Gestionar Envio, Gestionar Reposición y gestionar Reportes.



20/8 y 21/8

Saragusti:

El jueves 20 de agosto realize la red, solo con los switches y routers correspondientes para ya tenerla pre configurada, contando con 10 routers, 20 redes, y aproximadamente 40 o más switches.

 (En esta imagen se ve una parte de todo lo que había puesto, estando configurado parcialmente y sin los dispositivos finales)

El 21 de agosto corregi las redes, desde el router 5 hasta el 10 porque me había confundido en como estableci las redes porque había repetido en el router 5 192.168.8.1 cuando deberia de haberlo reemplazado por 9, pero al terminarlo ya lo deje corregido, además actualice el gant marcando los periodos de actividades como completadas.

22/8

Rosato: Se llevó a cabo una sesión de control y seguimiento sobre el avance del Diagrama de Gantt en colaboración con el compañero Benjamin Saragusti. Durante esta jornada, se analizaron las dependencias lógicas entre las tareas de desarrollo y se ajustaron los tiempos estimados de los módulos de software. Esta revisión intermedia permitió actualizar la matriz de tiempos en la hoja de cálculo de Microsoft Excel, asegurando que el cronograma refleje de manera fiel el progreso del equipo antes de proceder con el modelado gráfico final.

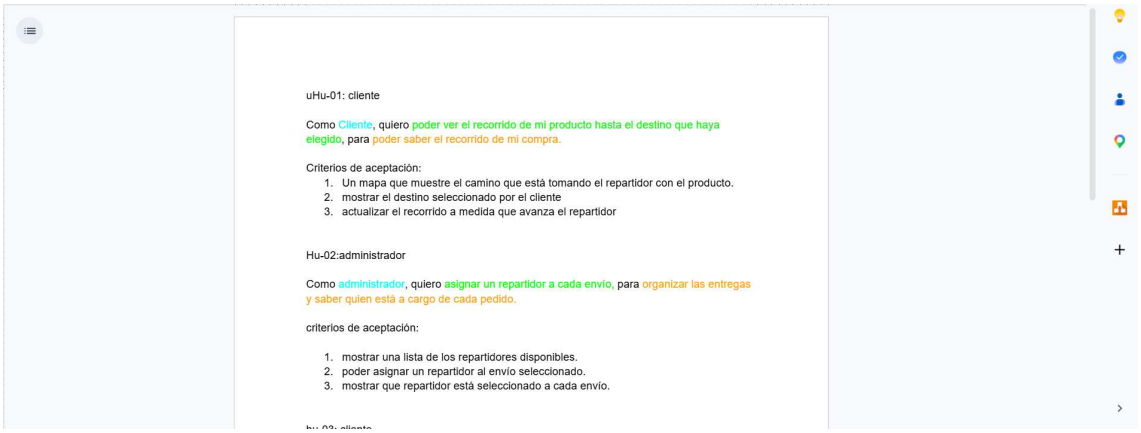
23/8

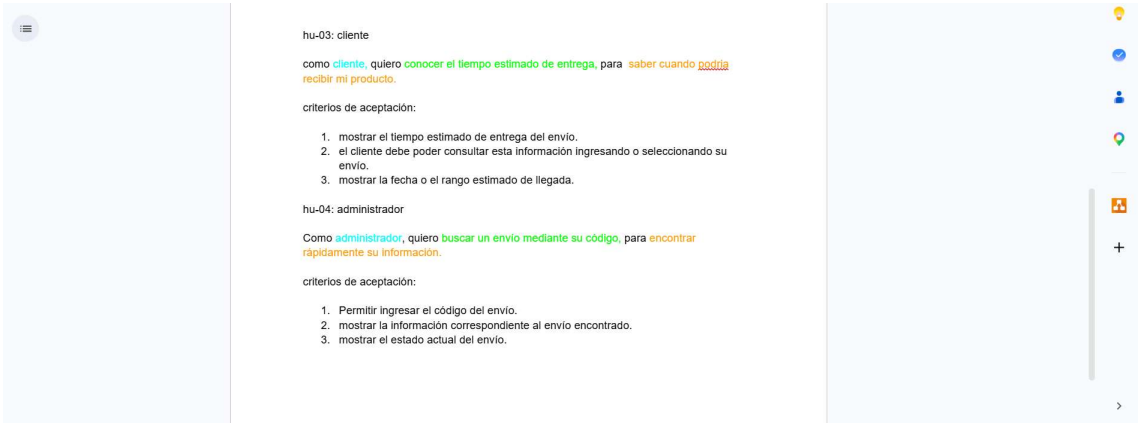
Zowmir: me dispuse a hacer el trabajo que se me fue asignado el dia viernes 21/08 que era

hacer:

1) historias de usuario
2) Actualización del DER.

para hacer las historias del usuario pense tanto como las acciones que debería y podría hacer un administrador y un cliente que está interesado en nuestros servicios, como por ejemplo que pueda ver el seguimiento del repartidor con su pedido. y del lado del administrador que pueda asignarle a cada envío que se cree un repartidor.





y bueno para actualizar el DER utilice el diccionario de datos que ya estaba actualizado, y nomas me fijé que cosas había que agregar o quitar.

24/8

Saragusti: El 24 de agosto proporcione una parte de código para poder leer el buscador universal, ya que este devuelve un índice y no un objeto, por lo que el código de ejemplo fue:

int indice=buscador universal(identificador,opción);

Cliente c=clientes.get(indice);

Además de eso realice los servidores dhcp y dns para la primera red con los 200 dispositivos aproximadamente, lo que permite la conectividad entre las computadoras y las impresoras, ya que todavía no nos enseñaron a configurar los teléfonos ip, y solo obtienen la ip gracias al dhcp que se las da automáticamente pero nada más.

26/8

Zowmir: El gran líder Goldfarb me dijo que haga el crud de nuestro proyecto. Con mi compañero Santiago Pedraza. Básicamente tenemos que hacer los métodos de las clases. Hasta hoy logramos hacer solo 2. El agregar un cliente Y el agregar de empleado.

Rosato: Se concluyó de forma exitosa el desarrollo del Diagrama de Gantt y la determinación analítica del Camino Crítico (CPM) del proyecto. A través de la herramienta Draw.io, se modeló visualmente la red de actividades indispensables y sus respectivas dependencias temporales. Este análisis permitió identificar con absoluta precisión la secuencia de tareas críticas sin margen de holgura, las cuales dictaminan la duración total del proyecto de software y garantizan la entrega a término de los entregables del sistema.

I	Simbolo	Dependiente de:		DIAS:
Relevamiento	A1	-		3
Transcribir relevamiento	A2		A1	2
Analisis foda	A3		A1	1
Analisis requerimientos	A4		A1	1
Diagrama gantt	A5	-		-
Diagrama pert	A6	-		-
Diagrama CPM Texto	A7		A5	1
Diagrama CPM Dibujito	A8		A5	2
Diagrama EDT	A8	-		1
Documentacion presentacion problematica y solucion	A9		A1	2
Documento estado de arte	A10	-		2
MVP:Minimo producto viable	A11		A1	2
Analisis tecnologico	A12		A4	1
DFD	A13		A3	2
Caso de uso extendido	A14		A4	3
Historia de usuario	A15		A4	3
Calculo de horas trabajados	A16		A5	3
Manual De uso	A17	-		7
Manual de explotacion	A18	-		7
Base de datos				
Diccionario de datos	B1	-		2
Der	B2		B1	3
Codificacion	B3		B4	7
Tablas	B4		B2	1
Relaciones	B5		B2	1
Valores	B6		B3	2
Funciones	B7		B3	7
Vistas	B8		B7	7

Vistas	B8		B7	7
Procedimientos	B9		B8	7
Triggers	B10		B8	7
Transacciones	B11		B9	7
Eliminacion logica,despliegue de motor	B12	-		7
Poo				
Identificar clases	C1	-		1
Padre:Persona	C2		C1	1
Hijas:Cliente,Proveedor,Transportista,Analista,Seguimiento	C3		C2	1
Paquete	C4		C1	1
Padre:Producto	C5		C1	1
Hijas:Tecnologico,Decoramiento,Indumentaria,Accesorios	C6		C5	1
Envio	C7		C1	1
Padre Vehiculo	C8		C1	1
Hijas:Avion,Barco,Tren,Camion	C9		C8	1
Empresa	C10		C1	1
EmpresasAdheridas	C11		C1	1
Definir atributos	C12	-		1
Metodos	C13	-		1
UML 1.0	C14		C1	2
Codificar	C15		C14	5
Padre:Persona	C16	-		1
Hijas:Cliente,Proveedor,Transportista,Analista,Seguimiento	C17	-		1
Paquete	C18	-		1
Padre:Producto	C19	-		1
Hijas:Tecnologico,Decoramiento,Indumentaria,Accesorios	C20	-		1
Envio	C21	-		1

Padre Vehiculo	C22	-		1
Hijas:Avion,Barco,Tren,Camion	C23	-		1
Empresa	C24	-		1
EmpresasAdheridas	C25	-		1
Sacar clases hijas persona, vehiculo, producto	C26	-		2
Agregar clases administrador, ticket y reporte	C27	-		1
Modificar codigo java	C28		C1	5
Main	C29		C1	7
Interfaz grafica	C30		C1	7
Conexion con base de datos	C31	-		7
	C32			
Proyecto informatico 2				
Boceto (Sin funcionalidades)	D1	-		3
Definir funciones	D2	-		7
Logica de funcionamiento	D3	-		7
Separacion en paginas	D4		D3	7
Retoques artisticos	D5	-		7
vistas funcionales(minimo 10)	D6		D4	7
Vistas dinamicas	D7		D6	7
Conexion api http	D8	-		7
Diseño ui/ux correcto	D9		D5	7
Administracion y usuario	D10	-		7
Redes				
Identificar equipos	E1	-		1
Identificar cableado	E2	-		1
identificar Switches	E3	-		1
Identificar Routers	E4	-		1
Buscar precios	E5		E1-E4	1
Calcular presupuestos	E6		E5	1
Encontrar correlaciones en Cisko	E7	.-		1
Armar red en cisko (Profundizacion abajo)	E8	-		7
Colocar Dispositivos	E9	-		2
Conectarlos adecuadamente	E10	-		1
Configurar routers	E11	-		2
Comprobar conectividad con pings	E12	-		1
Guardar resultados	E13	-		1
Servidores	E14		E11	2
Ips estatico y dinamico	E15		E14	2

Tambien realize el camino critico mediante draw.io

27/8

Saragusti: El 27 de agosto con el comentario del profesor, rehice la distribución de componentes en la red ya que se cambió el máximo a 300, por lo que adapte los componentes de la lista a capacidades que llegasen a cumplir con el máximo sin superarlo.

Zowmir: Con mi compañero pedraza nos dispusimos a seguir y terminar con el crud que se nos fué asignado a hacer. Terminando los demás métodos. los cuales son:

3) Agregar reporte.

Agregar envío.

Agregar paquete.

Agregar producto.

Agregar vehículo.

Agregar adheridos. (empresas adheridas)

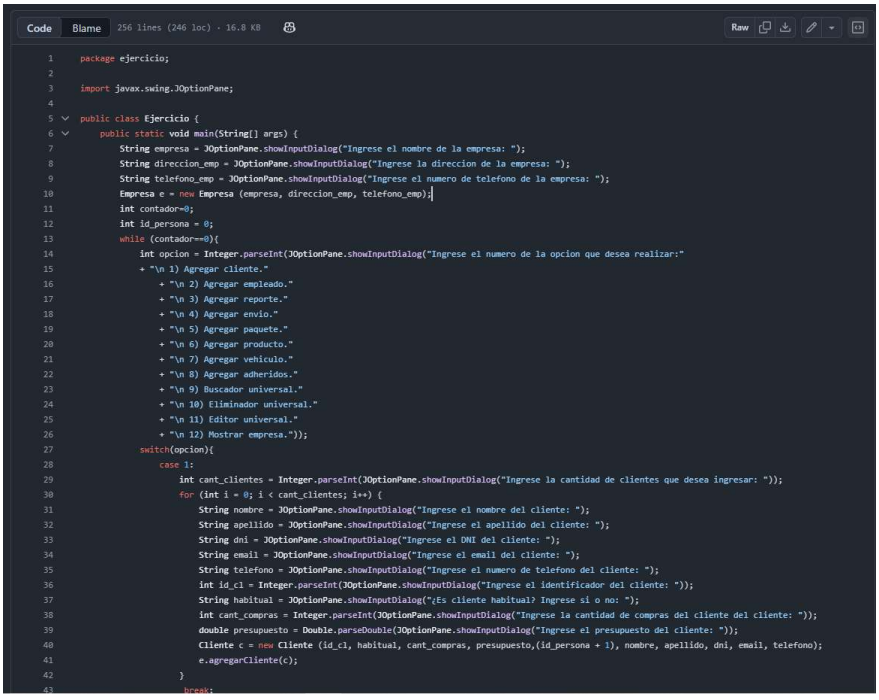
Buscador universal. (buscar cualquier elemento)

Eliminador universal. (eliminar cualquier elemento)

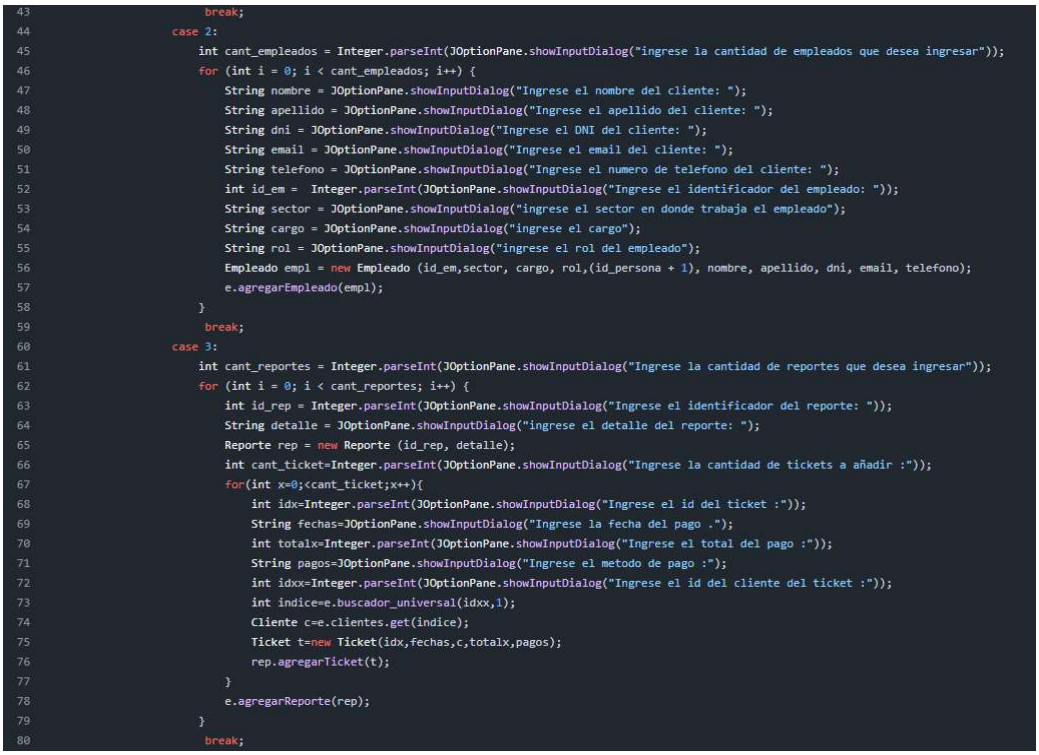
Editor universal. (editar cualquier elemento)

Mostrar empresa. (mostrar todos los datos y elementos de la empresa).

usamos como ayuda los Trabajos Prácticos que tenia mi compañero hechos en la materia POO, quedando tal que así:



```java
package ejercicio;

import javax.swing.JOptionPane;

public class Ejercicio {
    public static void main(String[] args) {
        String empresa = JOptionPane.showInputDialog("Ingrese el nombre de la empresa: ");
        String direccion_emp = JOptionPane.showInputDialog("Ingrese la direccion de la empresa: ");
        String telefono_emp = JOptionPane.showInputDialog("Ingrese el numero de telefono de la empresa: ");
        Empresa e = new Empresa (empresa, direccion_emp, telefono_emp);
        int contador=0;
        int id_persona = 0;
        while (contador==0){
            int opcion = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el numero de la opcion que desea realizar:"
            + "\n 1) Agregar cliente."
                + "\n 2) Agregar empleado."
                + "\n 3) Agregar reporte."
                + "\n 4) Agregar envio."
                + "\n 5) Agregar paquete."
                + "\n 6) Agregar producto."
                + "\n 7) Agregar vehiculo."
                + "\n 8) Agregar adheridos."
                + "\n 9) Buscador universal."
                + "\n 10) Eliminador universal."
                + "\n 11) Editor universal."
                + "\n 12) Mostrar empresa."));
            switch(opcion){
                case 1:
                    int cant_clientes = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de clientes que desea ingresar: "));
                    for (int i = 0; i < cant_clientes; i++) {
                        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente: ");
                        String apellido = JOptionPane.showInputDialog("Ingrese el apellido del cliente: ");
                        String dni = JOptionPane.showInputDialog("Ingrese el DNI del cliente: ");
                        String email = JOptionPane.showInputDialog("Ingrese el email del cliente: ");
                        String telefono = JOptionPane.showInputDialog("Ingrese el numero de telefono del cliente: ");
                        int id_cl = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del cliente: "));
                        String habitual = JOptionPane.showInputDialog("¿Es cliente habitual? Ingrese si o no: ");
                        int cant_compras = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de compras del cliente del cliente: "));
                        double presupuesto = Double.parseDouble(JOptionPane.showInputDialog("Ingrese el presupuesto del cliente: "));
                        Cliente c = new Cliente (id_cl, habitual, cant_compras, presupuesto,(id_persona + 1), nombre, apellido, dni, email, telefono);
                        e.agregarCliente(c);
                    }
                     break;
```



```
                    break;
                case 2:
                    int cant_empleados = Integer.parseInt(JOptionPane.showInputDialog("ingrese la cantidad de empleados que desea ingresar"));
                    for (int i = 0; i < cant_empleados; i++) {
                        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente: ");
                        String apellido = JOptionPane.showInputDialog("Ingrese el apellido del cliente: ");
                        String dni = JOptionPane.showInputDialog("Ingrese el DNI del cliente: ");
                        String email = JOptionPane.showInputDialog("Ingrese el email del cliente: ");
                        String telefono = JOptionPane.showInputDialog("Ingrese el numero de telefono del cliente: ");
                        int id_em =  Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del empleado: "));
                        String sector = JOptionPane.showInputDialog("ingrese el sector en donde trabaja el empleado");
                        String cargo = JOptionPane.showInputDialog("ingrese el cargo");
                        String rol = JOptionPane.showInputDialog("ingrese el rol del empleado");
                        Empleado empl = new Empleado (id_em,sector, cargo, rol,(id_persona + 1), nombre, apellido, dni, email, telefono);
                        e.agregarEmpleado(empl);
                    }
                    break;
                case 3:
                    int cant_reportes = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de reportes que desea ingresar"));
                    for (int i = 0; i < cant_reportes; i++) {
                        int id_rep = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del reporte: "));
                        String detalle = JOptionPane.showInputDialog("ingrese el detalle del reporte: ");
                        Reporte rep = new Reporte (id_rep, detalle);
                        int cant_ticket=Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de tickets a añadir :"));
                        for(int x=0;<cant_ticket;x++){
                            int idx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el id del ticket :"));
                            String fechas=JOptionPane.showInputDialog("Ingrese la fecha del pago .");
                            int totalx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el total del pago :"));
                            String pagos=JOptionPane.showInputDialog("Ingrese el metodo de pago :");
                            int idxx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el id del cliente del ticket :"));
                            int indice=e.buscador_universal(idxx,1);
                            Cliente c=e.clientes.get(indice);
                            Ticket t=new Ticket(idx,fechas,c,totalx,pagos);
                            rep.agregarTicket(t);
                        }
                        e.agregarReporte(rep);
                    }
                    break;
```

Imágenes de Pedraza.

28/8

Rosato: Se realizó la etapa de diseño teórico y el planteo estructurado de los componentes avanzados para la base de datos relacional. Se definieron las especificaciones iniciales para la creación de Procedimientos Almacenados (Stored Procedures), Vistas (Views), control de Transacciones (Transactions) y Disparadores (Triggers), orientados a garantizar la integridad y eficiencia de los datos.

Saragusti: arme la red hasta la 10, siendo que todos los componentes, o por lo menos las compus, pudieran comunicarse con el servidor dhcp, dns y con el http que no modifique el código pero se puede observar la pestaña provista por el. Siendo que quedaron 6 servidores en la red 192.168.2.0 siendo:

“.2 Para el Dhcp, “3 Para el DNS, y “.4 para el HTTP.

Además estuve guiando a los chicos en qué manera realizar el menú y que además les explique como funciona el buscador universal que había realizado, debido a que este no se parecía en casi nada al buscador presentado por el profesor en la clase.
No solo esto sino que también les comente que estructura y que numero le tenía que corresponder cada opcion, ademas de que orden corresponde a cada función pero no programe nada

29/8

Ureña:

Justo un día antes de tener que hacer la pre entrega, y encima el mismo día en que vencía el plazo límite, me puse a redactar el documento del caso de uso extendido. La verdad es que se me juntó todo a último momento y fue un margen de tiempo super ajustado, a mi me hubiera encantado terminar muchísimo antes para trabajar con más tranquilidad, pero se me hizo imposible porque en esa mismas fechas tenía otros exámenes que también exigen bastante tiempo.

Casos de uso extendido - Sistema de Logística

CU-01: Crear y Armar Paquete

- **Nombre:** Crear y armar un paquete
- **Actor principal:** Empleado
- **Actores secundarios:** Sistema de base de datos
- **Precondición:** El empleado debe estar autenticado. Los productos deben estar registrados y contar con stock disponible.
- **Postcondición:** El paquete queda registrado y asociado a los productos seleccionados.

Flujo principal

1) El empleado selecciona la opción "Crear nuevo paquete".
2) El sistema solicita los datos del paquete: peso, volumen y fecha de ensamble.
3) El empleado ingresa los datos solicitados.
4) El sistema muestra los productos disponibles registrados.
5) El empleado selecciona uno o varios productos para incluir en el paquete.
6) El sistema verifica que los productos seleccionados tengan stock disponible.
7) El sistema registra el paquete en la tabla **Paquetes**.
8) El sistema registra la relación entre el paquete y los productos en **Pro_pa**.
9) El sistema muestra la confirmación del paquete creado.

Flujo alternativo

5.a. Producto sin stock:
El sistema informa que el producto seleccionado no tiene stock disponible y solicita seleccionar otro producto.

3.a. Datos inválidos:
Si el peso o volumen ingresados son inválidos, el sistema solicita corregir los datos.

Flujo de excepción

7.a. Error al registrar el paquete:
Si ocurre un error al guardar la información, el sistema informa que no fue posible crear el paquete y permite reintentar la operación.

CU-02: Registrar Envío

- **Nombre:** Registrar envío
- **Actor principal:** Empleado
- **Actores secundarios:** Sistema de base de datos
- **Precondición:** Deben existir registros de clientes, paquetes y ciudades. El empleado debe estar autenticado.

Por otro lado, la idea original era que yo también me encargara de armar el conteo de horas del equipo. Sin embargo, por el mismo motivo y otras cosas imprevistas no pude cumplir con esta parte de la tarea. Para no retrasar al grupo mi compañero Benjamín reasigno la responsabilidad a mi compañero Agustin Zowmir, quien se hizo cargo de registrar y pasar en limpio el conteo de horas.

31/8

Rosato: Se llevó a cabo el planteo lógico y diseño estructural de los componentes esenciales para la gestión operativa de la base de datos, dividiendo los requerimientos en procedimientos almacenados para la manipulación de datos y vistas para la extracción automatizada de información:

- **Modelado de Procedimientos Almacenados (*Stored Procedures*):**
    1. **registrar_nuevo_pedido:** Centraliza la recepción de solicitudes de los clientes, automatizando la apertura del flujo logístico.

2. **asignar_pedido_transporte:** Resuelve la asignación física de la carga vinculando un pedido específico con su chofer y vehículo correspondiente.
3. **actualizar_estado_entrega:** Permite la actualización paramétrica de los estados del paquete para asegurar la trazabilidad en tiempo real.

- **Modelado de Vistas Estructuradas (*Views*):**
  1. **vista_pedidos_pendientes:** Consolida los registros de órdenes activas que requieren preparación física o despacho inmediato.
  2. **vista_entregas_por_chofer:** Agrupa el histórico y rendimiento de despachos completados por operario para auditoría de productividad.
  3. **vista_vehiculos_disponibles:** Filtra en tiempo real las unidades de la flota que se encuentran libres de rutas asignadas y operativas.

1/9

Rosato: Se dio continuidad y cierre técnico a las tareas críticas de automatización e integridad referencial encomendadas por el líder del grupo Goldfard. Se establecieron las especificaciones para los bloques transaccionales y los disparadores reactivos que protegerán la consistencia lógica del inventario:

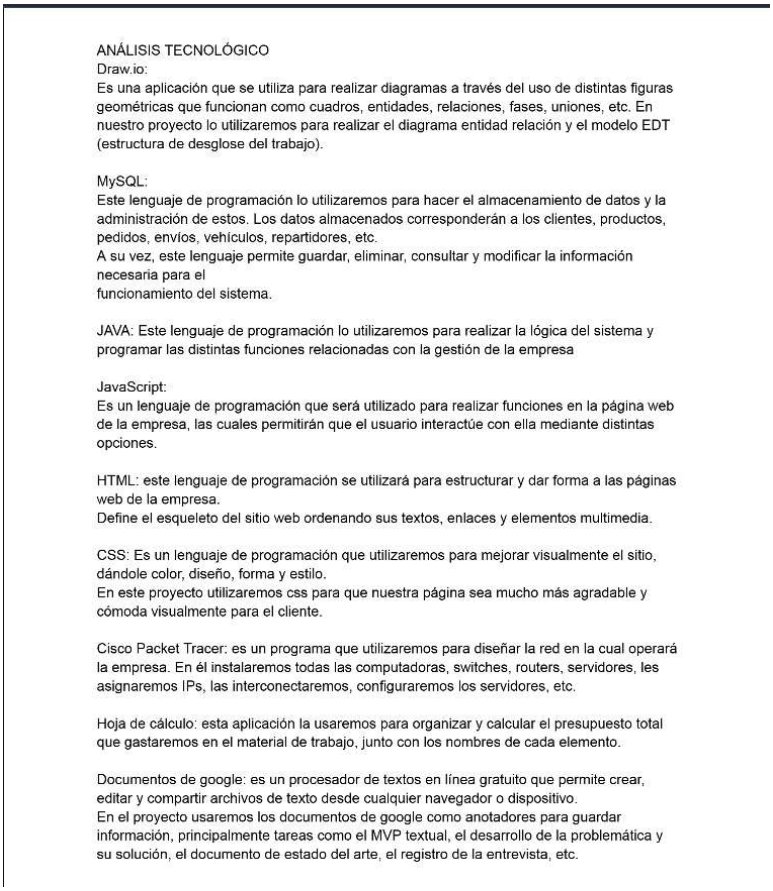
- **Especificación de Bloques Transaccionales (Propiedades ACID):**
  1. **Transacción de Venta e Inventario:** Encapsula de forma atómica el registro del pedido y la disminución física de las unidades del stock; ante cualquier inconsistencia de saldo, ejecuta un ROLLBACK.
  2. **Transacción de Reversa por Cancelación:** Restablece las condiciones iniciales ante la anulación de una orden, reincorporando la mercadería al depósito de origen.
  3. **Transacción de Movimiento Inter-Depósito:** Coordina de manera simultánea el egreso de stock en un nodo logístico emisor y el ingreso en el nodo receptor, evitando pérdidas de mercadería en tránsito.
- **Especificación de Disparadores (*Triggers*):**
  1. **tg_descontar_stock_pedido:** Actúa de forma automática (AFTER INSERT) para deprimir las existencias físicas del producto al momento de anexarlo a una orden logística.
  2. **tg_reponer_stock_devolucion:** Automatiza el incremento del stock disponible en el instante en que se procesa el ingreso formal de un rechazo o devolución.
  3. **tg_actualizar_estado_pedido:** Escucha las modificaciones en el módulo de entregas y, al confirmarse la recepción conforme del cliente, conmuta el estado global de la orden a "Finalizado"

2/9

Pedraza: El día **2/9** tuvimos otro “checkpoint” de lo realizado hasta el momento.

Aproximadamente

Una hora antes de que tuviéramos que presentar nuestros avances, se me ordenó corregir el Análisis Tecnológico, correspondiente al análisis de sistemas. El mismo había sido realizado previamente por mi compañero Agustin Zowmir, pero tenía errores de redacción, faltantes de información e información errónea. Por lo tanto, cumpli con mi deber a tiempo y quedando así la versión final del análisis tecnológico.



ANÁLISIS TECNOLÓGICO

Draw.io:
Es una aplicación que se utiliza para realizar diagramas a través del uso de distintas figuras geométricas que funcionan como cuadros, entidades, relaciones, fases, uniones, etc. En nuestro proyecto lo utilizaremos para realizar el diagrama entidad relación y el modelo EDT (estructura de desglose del trabajo).

MySQL:
Este lenguaje de programación lo utilizaremos para hacer el almacenamiento de datos y la administración de estos. Los datos almacenados corresponderán a los clientes, productos, pedidos, envíos, vehículos, repartidores, etc.
A su vez, este lenguaje permite guardar, eliminar, consultar y modificar la información necesaria para el
funcionamiento del sistema.

JAVA: Este lenguaje de programación lo utilizaremos para realizar la lógica del sistema y programar las distintas funciones relacionadas con la gestión de la empresa

JavaScript:
Es un lenguaje de programación que será utilizado para realizar funciones en la página web de la empresa, las cuales permitirán que el usuario interactúe con ella mediante distintas opciones.

HTML: este lenguaje de programación se utilizará para estructurar y dar forma a las páginas web de la empresa.
Define el esqueleto del sitio web ordenando sus textos, enlaces y elementos multimedia.

CSS: Es un lenguaje de programación que utilizaremos para mejorar visualmente el sitio, dándole color, diseño, forma y estilo.
En este proyecto utilizaremos css para que nuestra página sea mucho más agradable y cómoda visualmente para el cliente.

Cisco Packet Tracer: es un programa que utilizaremos para diseñar la red en la cual operará la empresa. En él instalaremos todas las computadoras, switches, routers, servidores, les asignaremos IPs, las interconectaremos, configuraremos los servidores, etc.

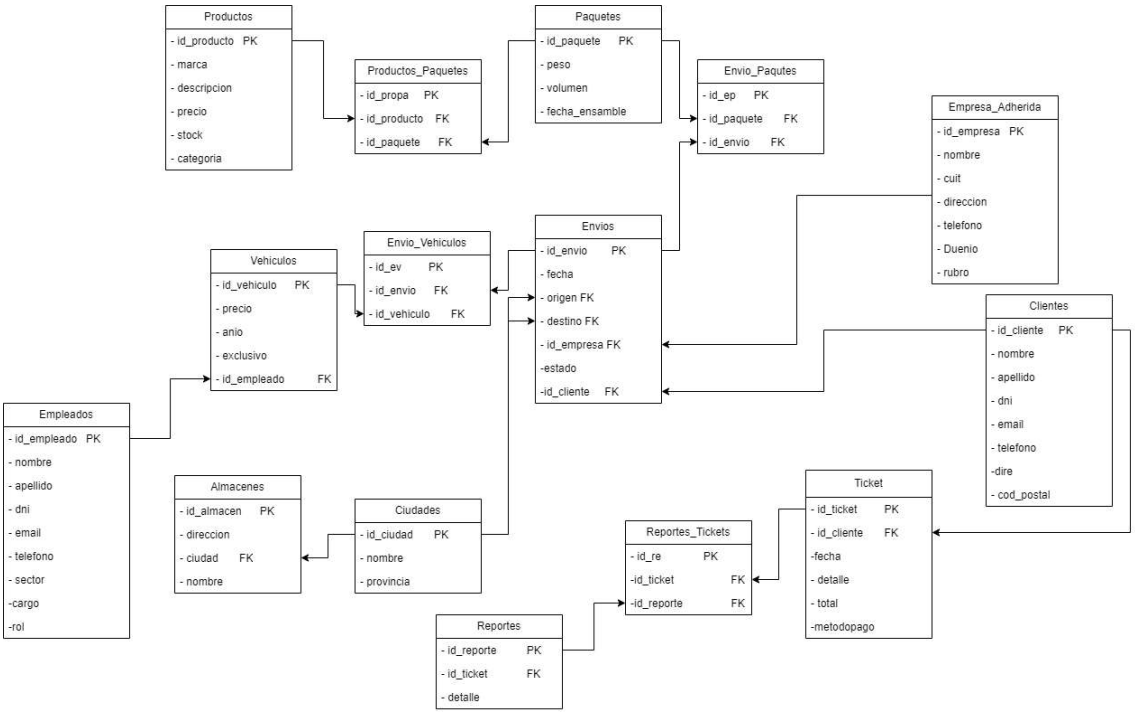
Hoja de cálculo: esta aplicación la usaremos para organizar y calcular el presupuesto total que gastaremos en el material de trabajo, junto con los nombres de cada elemento.

Documentos de google: es un procesador de textos en línea gratuito que permite crear, editar y compartir archivos de texto desde cualquier navegador o dispositivo.
En el proyecto usaremos los documentos de google como anotadores para guardar información, principalmente tareas como el MVP textual, el desarrollo de la problemática y su solución, el documento de estado del arte, el registro de la entrevista, etc.

Posterior a la devolución del checkpoint, en el cual uno de los principales detalles que nos mencionaron fue la ilegibilidad del Diagrama Entidad Relación (DER), lo primero que realicé fue, justamente, una actualización del mismo. El líder Santiago Goldfarb señaló que debíamos quitar algunas tablas que, en lugar de normalizar, organizar mejor el diagrama y la

futura base de datos, hacían más confuso su entendimiento. Por lo tanto, yo me encargue de

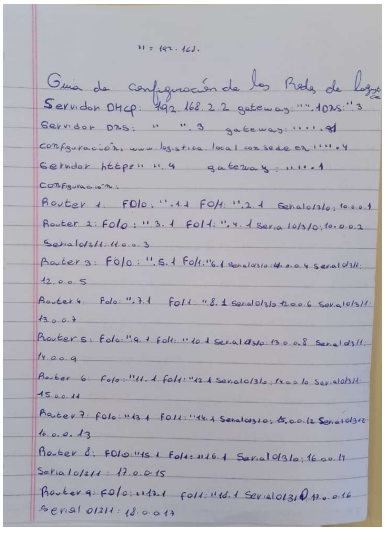
eso.

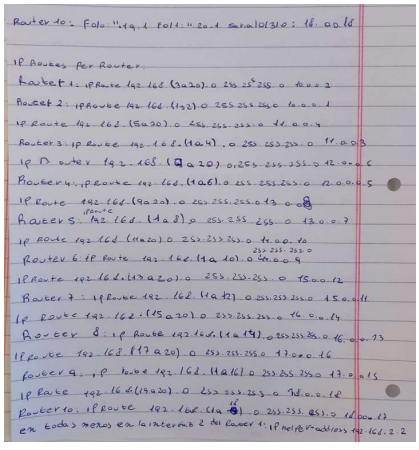


Rosato: Una vez terminada la pre entrega, nos reunimos virtualmente para ver cómo dividimos el trabajo. Nuestro líder de equipo, Golfard, analizó la situación y nos asigno a Xiomara y a mí la tarea de encargarnos de todas las correcciones correspondientes al código de Java, que son justamente las que se pueden ver en la foto que adjunto acá abajo. Cabe aclarar un detalle muy importante, nos pusimos de acuerdo en que solamente íbamos a resolver los primeros 9 puntos de la lista, ya que las consignas restantes exigen usar herramientas y conceptos de programación que todavía no llegamos a ver en la materia de POO, por lo que era imposible avanzar con eso por el momento.

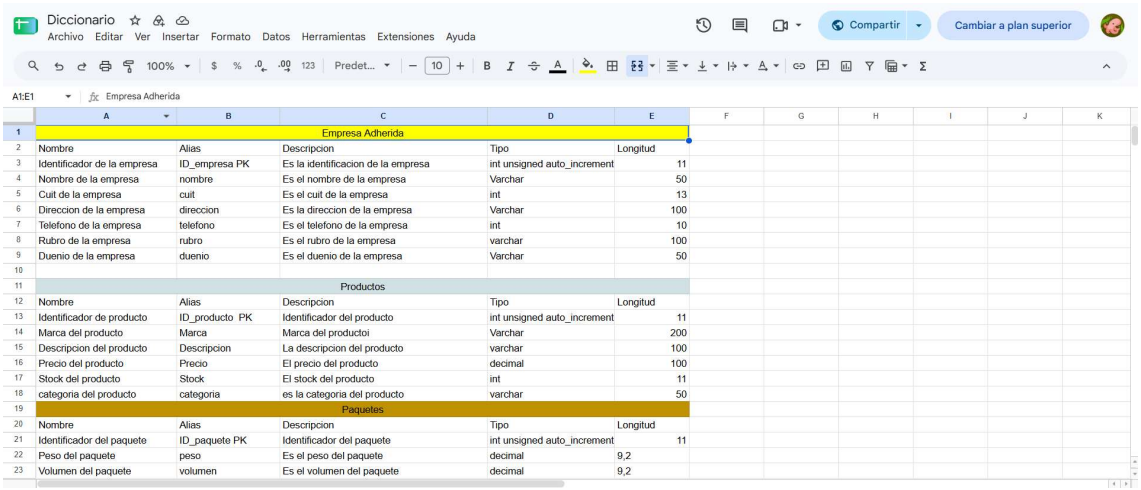
```
        }
        if(dni.length()!=8){
            JOptionPane.showMessageDialog(null, "ERROR - El DNI debe tener solo 8 digitos");
            contador++;
            return;
        }


        // condicionales telefono
        if(!telefono.matches("[0-9]+") && !telefono.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El telefono solo puede contener numeros.");
            contador++;
            return;
        }
        if(telefono.length()!=11 && telefono.length()!=10){
            JOptionPane.showMessageDialog(null, "ERROR - El telefono debe contar con 10 u 11 digitos");
            contador++;
            return;
        }
```

Saragusti: además de la revisión yo termine de configurar y conectar las últimas 10 redes que faltaban con dhcp,dns y servidor http funcional, que igual no tuve que modificar los datos de los routers ya que previamente los había dejado preparados para la extensión de la red. También hice una hoja escrita en la que deje la estructura plasmada de la configuración de los equipos para que mis compañeros pudieran entender la estructura y como estaba armada la red, ya que una explicación oral no servirá tanto como el código plasmado en texto de una manera sencilla para fácil deducción de los ideales.





Zowmir: Entonces nuestro líder mandó a hacer el DER a Pedraza y a mí me tocó hacer el diccionario de datos. Tuve que esperar a que mi compañero terminara su parte para poder hacer el diccionario. Donde quité las clases que estaban de más y de paso corregí las faltas ortográficas que había.



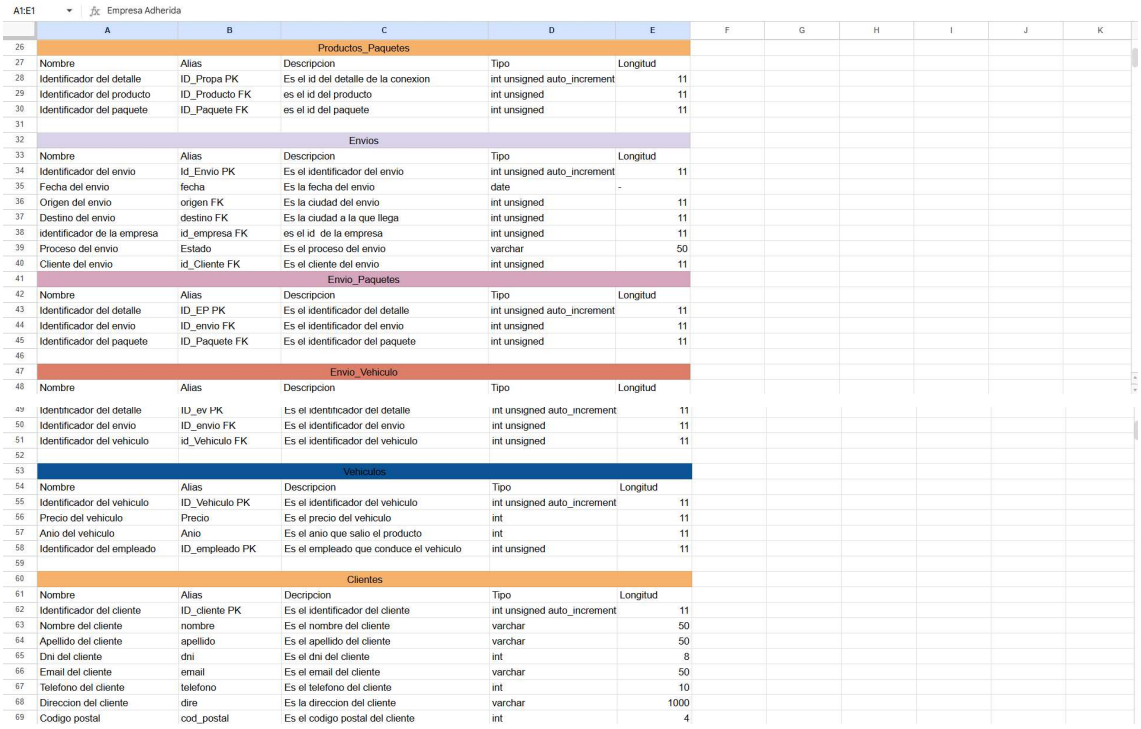
Diccionario

Archivo Editar Ver Insertar Formato Datos Herramientas Extensiones Ayuda

Compartir | Cambiar a plan superior

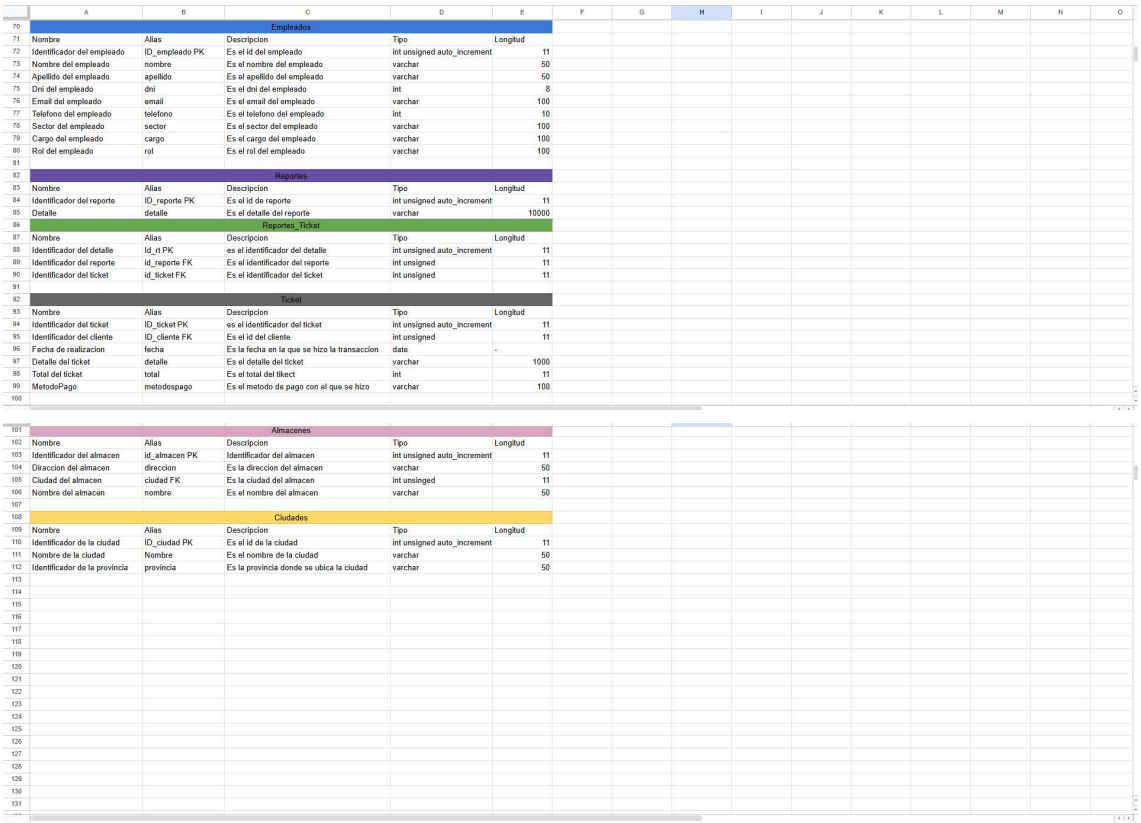
A1:E1 | Empresa Adherida

	A	B	C	D	E
1	Empresa Adherida				
2	Nombre	Alias	Descripcion	Tipo	Longitud
3	Identificador de la empresa	ID_empresa PK	Es la identificacion de la empresa	int unsigned auto_increment	11
4	Nombre de la empresa	nombre	Es el nombre de la empresa	Varchar	50
5	Cuit de la empresa	cuit	Es el cuit de la empresa	int	13
6	Direccion de la empresa	direccion	Es la direccion de la empresa	Varchar	100
7	Telefono de la empresa	telefono	Es el telefono de la empresa	int	10
8	Rubro de la empresa	rubro	Es el rubro de la empresa	varchar	100
9	Duenio de la empresa	duenio	Es el duenio de la empresa	Varchar	50
10					
11	Productos				
12	Nombre	Alias	Descripcion	Tipo	Longitud
13	Identificador de producto	ID_producto PK	Identificador del producto	int unsigned auto_increment	11
14	Marca del producto	Marca	Marca del productoi	Varchar	200
15	Descripcion del producto	Descripcion	La descripcion del producto	varchar	100
16	Precio del producto	Precio	El precio del producto	decimal	100
17	Stock del producto	Stock	El stock del producto	int	11
18	categoria del producto	categoria	es la categoria del producto	varchar	50
19	Paquetes				
20	Nombre	Alias	Descripcion	Tipo	Longitud
21	Identificador del paquete	ID_paquete PK	Identificador del paquete	int unsigned auto_increment	11
22	Peso del paquete	peso	Es el peso del paquete	decimal	9,2
23	Volumen del paquete	volumen	Es el volumen del paquete	decimal	9,2



A1:E1 | Empresa Adherida

	A	B	C	D	E
26	Productos_Paquetes				
27	Nombre	Alias	Descripcion	Tipo	Longitud
28	Identificador del detalle	ID_Propa PK	Es el id del detalle de la conexion	int unsigned auto_increment	11
29	Identificador del producto	ID_Producto FK	es el id del producto	int unsigned	11
30	Identificador del paquete	ID_Paquete FK	es el id del paquete	int unsigned	11
31					
32	Envios				
33	Nombre	Alias	Descripcion	Tipo	Longitud
34	Identificador del envio	Id_Envio PK	Es el identificador del envio	int unsigned auto_increment	11
35	Fecha del envio	fecha	Es la fecha del envio	date	-
36	Origen del envio	origen FK	Es la ciudad del envio	int unsigned	11
37	Destino del envio	destino FK	Es la ciudad a la que llega	int unsigned	11
38	identificador de la empresa	id_empresa FK	es el id de la empresa	int unsigned	11
39	Proceso del envio	Estado	Es el proceso del envio	varchar	50
40	Cliente del envio	id_Cliente FK	Es el cliente del envio	int unsigned	11
41	Envio_Paquetes				
42	Nombre	Alias	Descripcion	Tipo	Longitud
43	Identificador del detalle	ID_EP PK	Es el identificador del detalle	int unsigned auto_increment	11
44	Identificador del envio	ID_envio FK	Es el identificador del envio	int unsigned	11
45	Identificador del paquete	ID_Paquete FK	Es el identificador del paquete	int unsigned	11
46					
47	Envio_Vehiculo				
48	Nombre	Alias	Descripcion	Tipo	Longitud
49	Identificador del detalle	ID_ev PK	Es el identificador del detalle	int unsigned auto_increment	11
50	Identificador del envio	ID_envio FK	Es el identificador del envio	int unsigned	11
51	Identificador del vehiculo	id_Vehiculo FK	Es el identificador del vehiculo	int unsigned	11
52					
53	Vehiculos				
54	Nombre	Alias	Descripcion	Tipo	Longitud
55	Identificador del vehiculo	ID_Vehiculo PK	Es el identificador del vehiculo	int unsigned auto_increment	11
56	Precio del vehiculo	Precio	Es el precio del vehiculo	int	11
57	Anio del vehiculo	Anio	Es el anio que salio el producto	int	11
58	Identificador del empleado	ID_empleado PK	Es el empleado que conduce el vehiculo	int unsigned	11
59					
60	Clientes				
61	Nombre	Alias	Decripcion	Tipo	Longitud
62	Identificador del cliente	ID_cliente PK	Es el identificador del cliente	int unsigned auto_increment	11
63	Nombre del cliente	nombre	Es el nombre del cliente	varchar	50
64	Apellido del cliente	apellido	Es el apellido del cliente	varchar	50
65	Dni del cliente	dni	Es el dni del cliente	int	8
66	Email del cliente	email	Es el email del cliente	varchar	50
67	Telefono del cliente	telefono	Es el telefono del cliente	int	10
68	Direccion del cliente	dire	Es la direccion del cliente	varchar	1000
69	Codigo postal	cod_postal	Es el codigo postal del cliente	int	4



Empleados				
Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del empleado	ID_empleado PK	Es el id del empleado	int unsigned auto_increment	11
Nombre del empleado	nombre	Es el nombre del empleado	varchar	50
Apellido del empleado	apellido	Es el apellido del empleado	varchar	50
Dni del empleado	dni	Es el dni del empleado	int	8
Email del empleado	email	Es el email del empleado	varchar	100
Telefono del empleado	telefono	Es el telefono del empleado	int	10
Sector del empleado	sector	Es el sector del empleado	varchar	100
Cargo del empleado	cargo	Es el cargo del empleado	varchar	100
Rol del empleado	rol	Es el rol del empleado	varchar	100
Reportes				
Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del reporte	ID_reporte PK	Es el id de reporte	int unsigned auto_increment	11
Detalle	detalle	Es el detalle del reporte	varchar	10000
Reportes_Ticket				
Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del detalle	Id_rt PK	es el identificador del detalle	int unsigned auto_increment	11
Identificador del reporte	id_reporte FK	Es el identificador del reporte	int unsigned	11
Identificador del ticket	id_ticket FK	Es el identificador del ticket	int unsigned	11
Ticket				
Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del ticket	ID_ticket PK	es el identificador del ticket	int unsigned auto_increment	11
Identificador del cliente	ID_cliente FK	Es el id del cliente	int unsigned	11
Fecha de realizacion	fecha	Es la fecha en la que se hizo la transaccion	date	-
Detalle del ticket	detalle	Es el detalle del ticket	varchar	1000
Total del ticket	total	Es el total del tikect	int	11
MetodoPago	metodospago	Es el metodo de pago con el que se hizo	varchar	100

Almacenes				
Nombre	Alias	Descripcion	Tipo	Longitud
Identificador del almacen	id_almacen PK	Identificador del almacen	int unsigned auto_increment	11
Diraccion del almacen	direccion	Es la direccion del almacen	varchar	50
Ciudad del almacen	ciudad FK	Es la ciudad del almacen	int unsinged	11
Nombre del almacen	nombre	Es el nombre del almacen	varchar	50
Ciudades				
Nombre	Alias	Descripcion	Tipo	Longitud
Identificador de la ciudad	ID_ciudad PK	Es el id de la ciudad	int unsigned auto_increment	11
Nombre de la ciudad	Nombre	Es el nombre de la ciudad	varchar	50
Identificador de la provincia	provincia	Es la provincia donde se ubica la ciudad	varchar	50

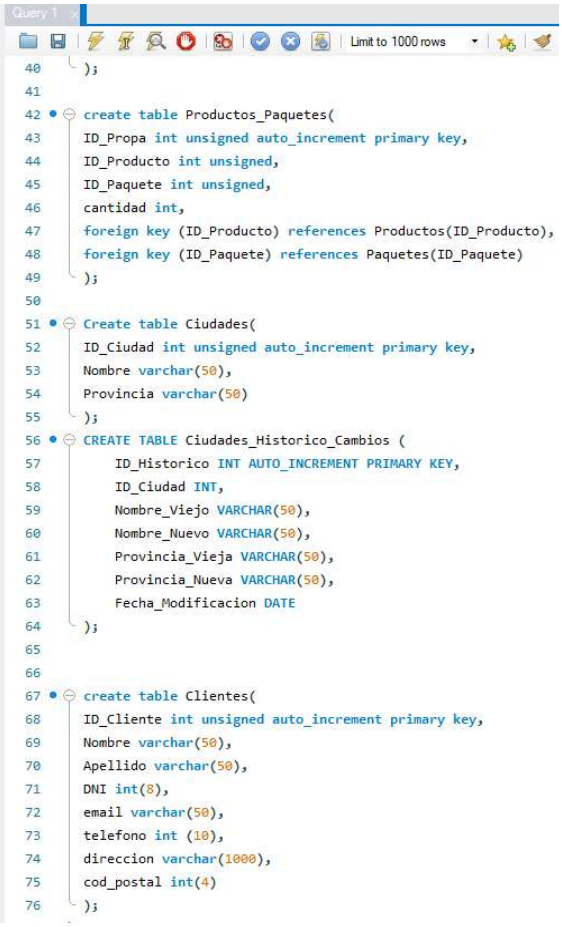
Pedraza: realicé las tablas completas de la Base de Datos, en MySQL Workbench 8.0, a partir de la versión final del Diagrama Entidad Relación. Durante la realización de las tablas, mi compañero Benjamin Saragusti me iba señalando si me equivocaba en algo.

Si bien yo tengo muy claro el cómo crear las tablas de una base de datos, lo que ocurrió seguido fue que, al basarme en el diccionario de datos para la realización de la misma, encontrábamos errores en este, los cuales debía corregir yo. Como estoy redactando este texto días después del día mencionado, no recuerdo con certeza qué partes del diccionario de datos tuve que modificar.

Lo que sí recuerdo es que eran errores no muy graves, por ejemplo que un atributo que debía ser de tipo “int” figurase como “decimal”, o que figure “varchar(25)” cuando lo ideal era “varchar(50)”, y más errores de ese estilo.

La base de datos quedo asi:

```sql
drop database if exists Logistica32;
create database Logistica32;
use Logistica32;

create table Empresa_Adherida(
    ID_empresa int unsigned auto_increment primary key,
    nombre varchar(50),
    cuit varchar(13),
    direccion varchar(100),
    telefono int (10),
    rubro varchar(100),
    duenio varchar(50)
);

create table Empresa_Adherida_eliminada(
    ID_empresa_eliminada int unsigned auto_increment primary key,
    nombre_eliminado varchar(50),
    cuit_eliminado varchar(13),
    direccion_eliminada varchar(100),
    telefono_eliminado int (10),
    rubro_eliminado varchar(100),
    duenio_eliminado varchar(50)
);


create table Productos(
ID_Producto int unsigned auto_increment primary key,
marca varchar(200),
descripcion varchar(100),
precio decimal(10,2),
stock int,
categoria varchar(50)
);

create table Paquetes(
ID_Paquete int unsigned auto_increment primary key,
peso decimal(9,2),
volumen decimal(9,2),
fecha_ensamble date
);
```



```sql
);

create table Productos_Paquetes(
ID_Propa int unsigned auto_increment primary key,
ID_Producto int unsigned,
ID_Paquete int unsigned,
cantidad int,
foreign key (ID_Producto) references Productos(ID_Producto),
foreign key (ID_Paquete) references Paquetes(ID_Paquete)
);

Create table Ciudades(
ID_Ciudad int unsigned auto_increment primary key,
Nombre varchar(50),
Provincia varchar(50)
);
CREATE TABLE Ciudades_Historico_Cambios (
    ID_Historico INT AUTO_INCREMENT PRIMARY KEY,
    ID_Ciudad INT,
    Nombre_Viejo VARCHAR(50),
    Nombre_Nuevo VARCHAR(50),
    Provincia_Vieja VARCHAR(50),
    Provincia_Nueva VARCHAR(50),
    Fecha_Modificacion DATE
);


create table Clientes(
ID_Cliente int unsigned auto_increment primary key,
Nombre varchar(50),
Apellido varchar(50),
DNI int(8),
email varchar(50),
telefono int (10),
direccion varchar(1000),
cod_postal int(4)
);
```

```
);
CREATE TABLE Clientes_Historico_Borrados (
    ID_Historico INT AUTO_INCREMENT PRIMARY KEY,
    ID_Cliente INT,
    Nombre VARCHAR(50),
    Apellido VARCHAR(50),
    DNI INT,
    Fecha_Baja date
);



create table Envios(
ID_Envio int unsigned auto_increment primary key,
Fecha date,
Origen int unsigned,
Destino int unsigned,
ID_Empresa int unsigned,
estado varchar(50),
ID_Cliente int unsigned,
foreign key (Origen) references Ciudades(ID_Ciudad),
foreign key (Destino) references Ciudades(ID_Ciudad),
foreign key (ID_Empresa) references Empresa_Adherida(ID_Empresa),
foreign key (ID_Cliente) references Clientes(ID_Cliente)
);

create table Envios_eliminados(
ID_Envio_eliminado int unsigned auto_increment primary key,
Fecha_eliminado date,
Origen_eliminado int unsigned,
Destino_eliminado int unsigned,
ID_Empresa_eliminado int unsigned,
estado_eliminado varchar(50),
ID_Cliente_eliminado int unsigned,
foreign key (Origen_eliminado) references Ciudades(ID_Ciudad),
foreign key (Destino_eliminado) references Ciudades(ID_Ciudad),
foreign key (ID_Empresa_eliminado) references Empresa_Adherida(ID_Empresa),
foreign key (ID_Cliente_eliminado) references Clientes(ID_Cliente)
);

```

```

create table Empleados(
ID_Empleado int unsigned auto_increment primary key,
Nombre varchar(50),
Apellido varchar(50),
Dni int(8),
email varchar(100),
telefono int (10),
sector varchar(100),
cargo varchar(100),
rol varchar(100)
);

create table Vehiculos (
ID_Vehiculo int unsigned auto_increment primary key,
Precio int (11),
anio int (11),
ID_Empleado int unsigned,
foreign key (ID_Empleado) references Empleados (ID_Empleado)
);

Create table Envio_Vehiculos(
ID_EV int unsigned auto_increment primary key,
ID_Envio int unsigned,
ID_Vehiculo int unsigned,
foreign key (ID_Envio) references Envios(ID_Envio),
foreign key (ID_Vehiculo) references Vehiculos(ID_Vehiculo)
);

create table Envio_Paquetes (
ID_ep int unsigned auto_increment primary key,
ID_Envio int unsigned,
ID_Paquete int unsigned,
foreign key (ID_Envio) references Envios(ID_Envio),
foreign key (ID_Paquete) references Paquetes(ID_Paquete)
);

```

```
create table Reportes(
ID_Reporte int unsigned auto_increment primary key,
detalle varchar(1000)
);

create table Tickets(
ID_Ticket int unsigned auto_increment primary key,
ID_Cliente int unsigned,
fecha date,
detalle varchar(1000),
total int(11),
metodospago varchar(100),
foreign key (ID_Cliente) references Clientes (ID_Cliente)
);

create table Reportes_Ticket(
ID_rt int unsigned auto_increment primary key,
ID_Reporte int unsigned,
ID_Ticket int unsigned,
foreign key (ID_Reporte) references Reportes (ID_Reporte),
foreign key (ID_Ticket) references Tickets (ID_Ticket)
);

create table Almacenes (
ID_Almacen int unsigned auto_increment primary key,
direccion varchar(50),
ciudad int unsigned,
nombre varchar(50),
foreign key (ciudad) references Ciudades (ID_Ciudad)
);
```

Goldfarb: Dijimos que procedimientos de qué tabla va a hacer cada uno. A mi me salieron las siguientes tablas:

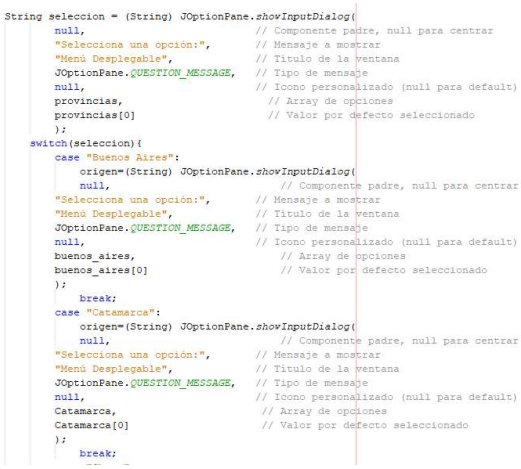
-Envíos

-Envios_paquetes

-Empresas_adheridas

Saragusti: estuve colaborando en la creación de la base de datos, realizada por Pedraza. Y también brinda ideas para vistas, procedimientos y uno de los triggers que vamos a usar.
Los procedimientos que hice fueron: Mostrar productos por paquete, tickets por reporte, vehículos por envío y paquetes por envío. Básicamente están hechos con el expreso objetivo de ver más sencillamente todos los datos sin tener que estar revisando solo números de tickets y reportes o usando selects.

```
}
String [] provincias ={"Buenos Aires","Catamarca","Chaco","Chubut","Cordoba","Corrientes",
        "Entre Rios","Formosa","Jujuy","La Pampa","La Rioja","Mendoza","Misiones","Neuquen",
        "Rio Negro","Salta","San Juan","San Luis","Santa Cruz","Santa Fe",
        "Santiago del Estero","Tierra del Fuego","Tucuman"};
String[]buenos_aires={"Ciudad Autonoma","La plata","Mar del plata","Bahia Blanca"};
String[]Catamarca={"San Fernando del Valle de Catamarca","Andalgala"};
String[]Chaco={"Resistencia","Charata"};
String[]Chubut={"Comodoro Rivadevia","Puerto Madryn"};
String[]Cordoba={"Cordoba","Villa Maria"};
String[]Corrientes={"Corrientes","Goya"};
String[]Entre_rios={"Parana","Concordia"};
String[]Formosa={"Formosa","Clorinda"};
String[]Jujuy={"San Salvador de Jujuy","Libertador General San Martin"};
String[]la_pampa={"Santa Rosa","General Pico"};
String[]la_rioja={"La Rioja","Chilecito"};
String[]Mendoza={"Mendoza","Godoy Cruz"};
String[]Misiones={"Posadas","Obera"};
String[]Neuquen={"Neuquen","Cutral Co"};
String[]Rio_negro={"Viedma","San Carlos de Bariloche"};
String[]Salta={"Salta","Cafayate"};
String[]San_juan={"San Juan","Rawson"};
String[]San_luis={"San Luis","Villa Mercedes"};
String[]Santa_cruz={"Rio Gallegos","El Calafate"};
String[]Santa_fe={"Santa Fe de la Vera Cruz","Rosario"};
String[]Santiago_del_estero={"Santiago Del Estero","La Banda"};
String[]Tierra_del_fuego={"Ushuaia","Rio Grande"};
String[]Tucuman={"San Miguel de Tucuman","Tafi Viejo"};
String origenes =JOptionPane.showInputDialog("Ingrese el origen del envio"+"\n - Nacional"+"\n - Internacional");
```



```
String seleccion = (String) JOptionPane.showInputDialog(
        null,                           // Componente padre, null para centrar
        "Selecciona una opción:",       // Mensaje a mostrar
        "Menú Desplegable",             // Titulo de la ventana
        JOptionPane.QUESTION_MESSAGE,   // Tipo de mensaje
        null,                           // Icono personalizado (null para default)
        provincias,                       // Array de opciones
        provincias[0]                     // Valor por defecto seleccionado
        );
    switch(seleccion){
        case "Buenos Aires":
            origen=(String) JOptionPane.showInputDialog(
            null,                           // Componente padre, null para centrar
        "Selecciona una opción:",       // Mensaje a mostrar
        "Menú Desplegable",             // Titulo de la ventana
        JOptionPane.QUESTION_MESSAGE,   // Tipo de mensaje
        null,                           // Icono personalizado (null para default)
        buenos_aires,                       // Array de opciones
        buenos_aires[0]                     // Valor por defecto seleccionado
        );
            break;
        case "Catamarca":
            origen=(String) JOptionPane.showInputDialog(
            null,                           // Componente padre, null para centrar
        "Selecciona una opción:",       // Mensaje a mostrar
        "Menú Desplegable",             // Titulo de la ventana
        JOptionPane.QUESTION_MESSAGE,   // Tipo de mensaje
        null,                           // Icono personalizado (null para default)
        Catamarca,                       // Array de opciones
        Catamarca[0]                     // Valor por defecto seleccionado
        );
            break;
```

Zowmir: para el dia de hoy me dediqué a hacer las vistas que se me fueron asignadas, las cuales son:

-vehículos con empleados
- envíos no entregados
-envíos cancelados

-empleados con rol administrador
-productos con stock bajo
-paquetes pesados

```
create view vehiculo_empleado as select vehiculos.ID_Vehiculo, vehiculos.Marca, vehiculos.Modelo, empleados.Nombre,empleados.Apellido,empleados.Dni from vehiculos left join empleados on empleados.ID_Empleado = vehiculos.ID_Empleado;

create view envios_noentregados as select * from envios
where estado = "NO ENTREGADO";

create view envios_cancelados as select * from envios
where estado = "CANCELADO";

create view administradores as select * from empleados
where empleados.rol = "ADMINISTRADOR";

create view Productos_critico as select * from productos
where productos.stock <10;

create view Paquetes_pesados as select * from paquetes
where paquetes.peso <100;
```

foto sacada del github, estas vistas las hice con la ayuda de benjamín saragusti

6/9

Rosato: El viernes 6 de septiembre nos juntamos con xiomara para terminar de realizar lo pedido, nos quedaban pendientes 7 puntos los cuales eran:

4) Indicar un ejemplo de identificador al usuario.

5) Validar solo tipos de datos.

6) Indicar un mensaje de error en caso de que el usuario ingrese una opción que no exista en el menú.

7) Agregar la opción de salir.

8) En la sección de “Origen de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

9) En la sección de “Destino de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

Cuando llegamos al punto 8 nos dimos cuenta que lo de menú desplegables no lo sabíamos porque todavía no lo habíamos visto en clase. Nuestro compañero Benjamin por su parte investigó como armar los menús desplegables en Java, logró entender la lógica del código, se encargó de programarlo él mismo.

ureña: El viernes 6 de septiembre nos juntamos con Nicolas para terminar de realizar lo pedido, nos quedaban pendientes 7 puntos los cuales eran:

4) Indicar un ejemplo de identificador al usuario.

5) Validar solo tipos de datos.

6) Indicar un mensaje de error en caso de que el usuario ingrese una opción que no exista en el menú.

7) Agregar la opción de salir.

8) En la sección de “Origen de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

9) En la sección de “Destino de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

Cuando llegamos al punto 8 nos dimos cuenta que lo de menú desplegables no lo sabíamos porque todavía no lo habíamos visto en clase. Nuestro compañero Benjamin por su parte investigó como armar los menús desplegables en Java, logró entender la lógica del código, se encargó de programarlo él mismo.

09/09/2026

rosato: Este dia tuvimos una charla con nuestros profesores en donde nos dieron los requisitos para la pre entrega de septiembre los cuales son:

- POO(Programación Orientada a Objetos): Todas las ventanas que vayamos a usar con CRUD funcional y contador de errores.

- Redes: Servidor FTP y Correo.
- Proyecto Informático: Pantallas hasta donde lleguemos
- Base de datos: Procedimientos.

Una vez finalizada la charla nuestro compañero Benjamin se encargó de asignarnos nuestras respectivas tareas, se nos asigno realizar tres procedures básicos (insertar, editar y eliminar) para las distintas tablas, siendo un total de 9 procedures..

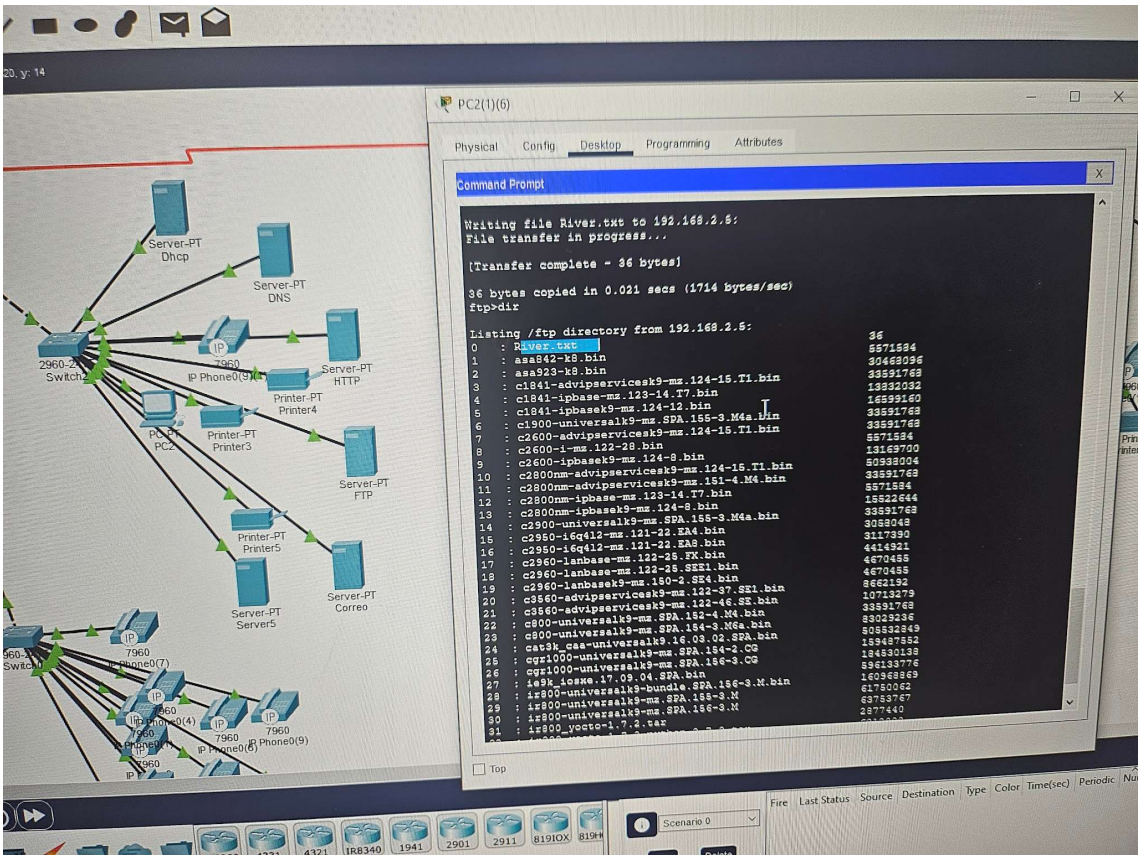
La división fue dos tablas normales y una tabla intermedia por integrante. A mi se me asignaron las tablas “productos”, “productos_paquetes” y “paquetes”.

- producto: Al crear un producto se debe verificar que el cliente exista y no se pueden borrar tickets que están en la tabla intermedia, reportes_tickets.
- Reportes: Al crear un reporte se debe verificar que el cliente exista y no se pueden borrar reportes que están en la tabla intermedia, reportes_tickets.
- Reportes_tickets: Se debe verificar que exista ese reporte y ese ticket.

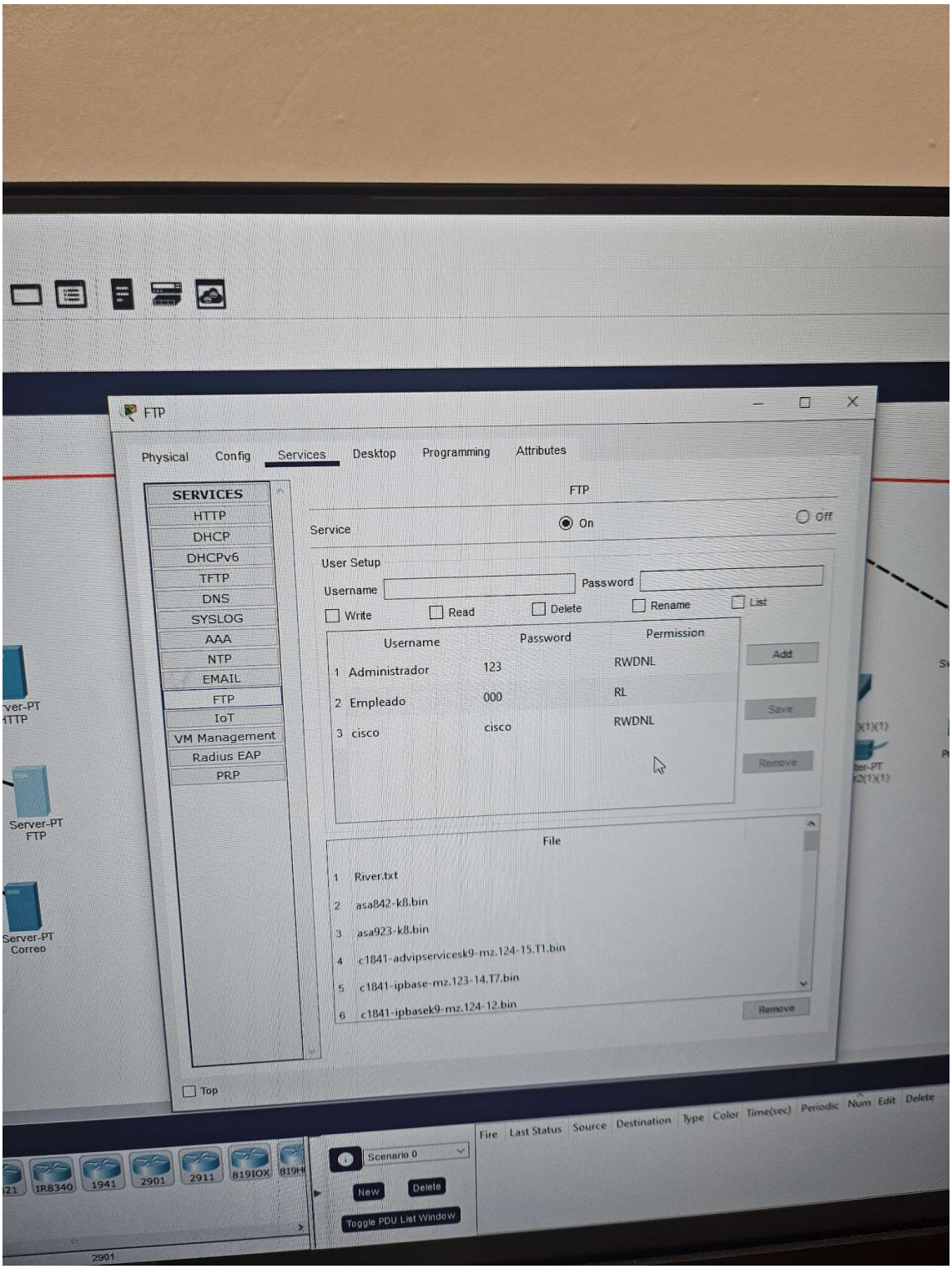
zowmir:En esta fecha, luego de que el profesor nos explicara la teoría sobre los servidores FTP, me encargué, junto con mi compañero Saragusti, de realizar las pruebas de transferencia de archivos entre las distintas redes de nuestra empresa.

Como primer paso, configuramos dos tipos de usuarios. Por un lado, creamos un usuario administrador, al que le otorgamos todos los permisos disponibles. Por otro lado, creamos un usuario empleado, que solamente tenía permitido leer y listar los archivos. Para crear ambos usuarios y sus respectivas contraseñas, recurrimos a la IA para que nos proporcionara algunas opciones.

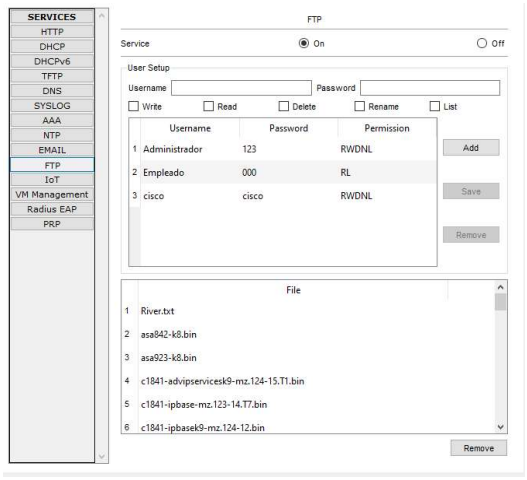
Una vez configurados los usuarios, pasamos a realizar la prueba de transferencia. Para esto, creamos un archivo de texto (.txt) llamado River en una de las máquinas. Luego, utilizamos el servidor FTP para enviarlo hacia un servidor ubicado en otra red, con el objetivo de comprobar que la transferencia de archivos funcionara correctamente.



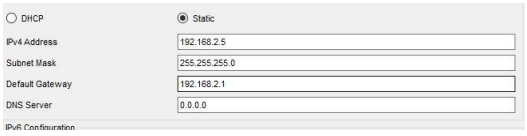
Como se puede observar en la siguiente imagen, el archivo fue transferido correctamente al servidor FTP. Allí se puede ver que el archivo de texto llamado River se encuentra dentro del servidor, lo que nos permitió comprobar que el envío entre las distintas redes se había realizado correctamente.


FTP
Physical
Config
Services
Desktop
Programming
Attributes
SERVICES
HTTP
DHCP
DHCPv6
TFTP
DNS
SYSLOG
AAA
NTP
EMAIL
FTP
IoT
VM Management
Radius EAP
PRP
FTP
Service
On
Off
User Setup
Username
Password
Write
Read
Delete
Rename
List
Username
Password
Permission
1 Administrador 123 RWDNL
2 Empleado 000 RL
3 cisco cisco RWDNL
Add
Save
Remove
File
1 River.txt
2 asa842-k8.bin
3 asa923-k8.bin
4 c1841-advipservicesk9-mz.124-15.T1.bin
5 c1841-ipbase-mz.123-14.T7.bin
6 c1841-ipbasek9-mz.124-12.bin
Remove
Top
Server-PT FTP
Server-PT Correo
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Scenario 0
New
Delete
Toggle PDU List Window
IR8340
1941
2901
2911
819IOX
2901

saragusti:El 10 de septiembre colaboré con zownir para realizar la función de servidor ftp, para poder enviar datos o leerlos de manera online y dinámica con un uso de dos perfiles. Uno para empleados que sirve para leer y ver los archivos y otro de administrador que tiene todos los privilegios para poder editar a su gusto los archivos subidos. Se hace de esta manera por si acaso porque puede que algunos no tengan un rango alto para poder modificar todo. El último servidor que queda sin configurar es el servidor de mails que va a explicar este miércoles el profe.



(Visualización de como quedo el servidor ftp, realizado con agustín zownir)



(Vista de como quedo la configuración de ip y generales del server)

GOLDFARB:

Fecha: 10/9/26

Ubicación: Casa

Hora: 10:10

Actividades Realizadas:

Terminé todos mis procedures.

Procedimientos Goldfarb:

CREAR ENVÍO

```
DELIMITER //
CREATE PROCEDURE verificar_envio(IN p_fecha date, in p_origen int, in p_destino int,in
p_id_empresa int,in p_estado varchar(50) , in p_cliente int , out mensaje varchar(100))
BEGIN
        declare ciudad_origen varchar(100);
   declare ciudad_destino varchar(100);
   declare cliente_envio varchar(100);
   declare adherida_envio varchar(100);

        Start TRANSACTION;
                select ciudades.Nombre into ciudad_origen from ciudades where
ciudades.ID_Ciudad=p_origen;
      select ciudades.Nombre into ciudad_destino from ciudades where
ciudades.ID_Ciudad=p_destino;
      select clientes.Nombre into cliente_envio from clientes where
clientes.ID_Cliente=p_cliente;
      select empresa_adherida.nombre into adherida_envio from empresa_adherida where
empresa_adherida.ID_empresa=p_id_empresa;
      if ciudad_origen is null THEN
         set mensaje="Origen inexistente";
         ROLLBACK;
         ELSEIF ciudad_destino is null THEN
         set mensaje="Destino inexistente";
         ROLLBACK;
         ELSEIF cliente_envio is null THEN
         set mensaje="CLiente inexistente";
         ROLLBACK;
         ELSEIF adherida_envio is null THEN
         set mensaje="Empresa adherida inexistente";
         ROLLBACK;
      ELSE
                        insert into envios (fecha, Origen, Destino, id_empresa, estado,
id_cliente)
        values (p_fecha, p_origen, p_destino, p_id_empresa, p_estado, p_cliente);
                        set mensaje="Registros ingresados";
                        COMMIT;
```

```
                end if;
end//
DELIMITER ;
```

EDITAR ENVÍO

```
DELIMITER //

CREATE PROCEDURE editar_envio(
   IN id_envio INT,
   IN fecha DATE,
   IN origen INT,
   IN destino INT,
   IN estado VARCHAR(50),
   OUT mensaje VARCHAR(200)
)
BEGIN
   DECLARE auxiliar INT;

   START TRANSACTION;

   SELECT envios.ID_Envio INTO auxiliar FROM envios WHERE envios.ID_Envio =
id_envio LIMIT 1;

   IF auxiliar IS NULL THEN
      SET mensaje = 'No existe el envio';
      ROLLBACK;
   ELSE
      IF fecha IS NOT NULL THEN
         UPDATE envios SET Fecha = fecha WHERE ID_Envio = id_envio;
      END IF;

      IF origen IS NOT NULL THEN
         UPDATE envios SET Origen = origen WHERE ID_Envio = id_envio;
      END IF;

      IF destino IS NOT NULL THEN
         UPDATE envios SET Destino = destino WHERE ID_Envio = id_envio;
      END IF;

      IF estado IS NOT NULL THEN
         UPDATE envios SET Estado = estado WHERE ID_Envio = id_envio;
      END IF;
```

```
        SET mensaje = 'Se han editado los campos proporcionados';
        COMMIT;
    END IF;
END //

DELIMITER ;
```

EDITAR ENVÍO

```
delimiter //
 create procedure editar_envio(
 in p_id_envio int unsigned,
 in p_fecha date,
 in p_origen int unsigned,
 in p_destino int unsigned,
 in p_id_empresa int unsigned,
 in p_estado varchar(50),
 in p_cliente int unsigned,
 out mensaje varchar(200)
 )
 begin
 declare auxiliar int unsigned default null;
 start transaction;
 select ID_Envio into auxiliar from Envios where ID_Envio = p_id_envio;

if auxiliar is null then
    set mensaje = 'No existe el envio';
    rollback;
else
    if p_fecha is not null then
        update Envios set Fecha = p_fecha where ID_Envio = p_id_envio;
    end if;
    if p_origen is not null then
        update Envios set Origen = p_origen where ID_Envio = p_id_envio;
    end if;
    if p_destino is not null then
        update Envios set Destino = p_destino where ID_Envio = p_id_envio;
    end if;
    if p_id_empresa is not null then
        update Envios set ID_Empresa = p_id_empresa where ID_Envio = p_id_envio;
    end if;
    if p_estado is not null then
        update Envios set Estado = p_estado where ID_Envio = p_id_envio;
    end if;
```

```
    if p_cliente is not null then
        update Envios set ID_Cliente = p_cliente where ID_Envio = p_id_envio;
    end if;
    set mensaje = 'Se han editado los campos proporcionados';
    commit;
end if;

end //
 delimiter ;
```

ELIMINAR ENVÍO

```
delimiter //
 create procedure eliminar_envio(in p_id_envio int unsigned, out mensaje varchar(150))
 begin
 declare v_envio int unsigned default null;
 declare v_relaciones int default 0;
 start transaction;
 select ID_Envio into v_envio from Envios where ID_Envio = p_id_envio limit 1;

if v_envio is null then
    set mensaje = 'No existe el envio';
    rollback;
else
    select count(*) into v_relaciones from Envio_Paquetes where ID_Envio = p_id_envio;
    if v_relaciones > 0 then
        set mensaje = 'El envio tiene paquetes asociados';
        rollback;
    else
        select count(*) into v_relaciones from Envio_Vehiculos where ID_Envio = p_id_envio;
        if v_relaciones > 0 then
            set mensaje = 'El envio tiene vehiculos asociados';
            rollback;
        else
            delete from Envios where ID_Envio = p_id_envio;
            set mensaje = 'Envio eliminado';
            commit;
        end if;
    end if;
end if;

end //
 delimiter ;
```

MOSTRAR ENVIO

```
DELIMITER //
create procedure mostrar_envios()
begin
   select *
   from envios;
   end//
DELIMITER ;
```

BUSCAR ENVIO:

```
DELIMITER //
create procedure buscar_envio(IN id int)
begin
   select *
   from envios
   where id = envios.ID_Envio;
   end//
DELIMITER ;
```

ENVÍO FECHA RECIENTE:

```
delimiter //
create PROCEDURE mostrar_fecha_reciente()
BEGIN
        select *
   from envios
   order by Fecha asc;
end //
DELIMITER ;
```

ENVÍO FECHA ANTIGUA:

```
delimiter //
create PROCEDURE mostrar_fecha_reciente()
BEGIN
        select *
   from envios
   order by Fecha desc;
end //
DELIMITER ;
```

ENVIO ESTADO “ENTREGADO”:

```
delimiter //
create PROCEDURE mostrar_estado_entregado(iN estado_actual varchar(100))
BEGIN
        select *
   from envios
   where estado_actual = "Entregado"
        end//
DELIMITER ;
```

ENVIO ESTADO “PENDIENTE”:

```
delimiter //
create PROCEDURE mostrar_estado_pendiente(iN estado_actual varchar(100))
BEGIN
        select *
   from envios
   where estado_actual = "Pendiente"
        end//
DELIMITER ;
```

ENVIO ESTADO “CANCELADO”

```
delimiter //
create PROCEDURE mostrar_estado_cancelado(iN estado_actual varchar(100))
BEGIN
        select *
   from envios
   where estado_actual = "Cancelado"
        end//
DELIMITER ;
```

MOSTRAR ENVIOS_PAQUETES

```
DELIMITER //
create procedure mostrar_envios_paquetes()
begin
   select *
   from envio_paquetes;
   end//
DELIMITER ;
```

BUSCAR ENVIOS_PAQUETES

```
DELIMITER //
create procedure buscar_envio_paquetes(IN id int)
begin
   select *
   from envio_paquete
   where id = envios_paquete.ID_EP;
   end//
DELIMITER ;
```

MOSTRAR PAQUETES

```
DELIMITER //
create procedure mostrar_paquetes()
begin
   select *
   from paquetes;
   end//
DELIMITER ;
```

BUSCAR PAQUETES

```
DELIMITER //
create procedure buscar_paquete(IN id int)
begin
   select *
   from paquetes
   where id = paquetes.ID_Paquete;
   end//
DELIMITER ;
```

AGREGAR ENVÍO_PAQUETE

```
DELIMITER //

CREATE PROCEDURE agregar_envio_paquete(
   IN p_id_envio INT,
   IN p_id_paquete INT,
   OUT mensaje VARCHAR(100)
```

```
)
BEGIN
   DECLARE v_estado_envio VARCHAR(100);
   DECLARE v_id_paquete INT;

   START TRANSACTION;

   SELECT envios.estado INTO v_estado_envio FROM envios WHERE envios.ID_Envio =
p_id_envio;

   SELECT paquetes.ID_Paquete INTO v_id_paquete FROM paquetes WHERE
paquetes.ID_Paquete = p_id_paquete;

   IF v_estado_envio IS NULL THEN
      SET mensaje = "Envio inexistente";
      ROLLBACK;
   ELSEIF v_id_paquete IS NULL THEN
      SET mensaje = "Paquete inexistente";
      ROLLBACK;
   ELSE
      INSERT INTO envio_paquetes VALUES (p_id_envio, p_id_paquete);

      SET mensaje = "Registros ingresados con éxito";
      COMMIT;
   END IF;
END //

DELIMITER ;
```

procedimientos para Envio_Paquetes

```
delimiter //
 create procedure agregar_envio_paquete(
 in p_id_envio int unsigned,
 in p_id_paquete int unsigned,
 out mensaje varchar(100)
 )
 begin
 declare v_envio int unsigned default null;
 declare v_paquete int unsigned default null;
 start transaction;
 select ID_Envio into v_envio from Envios where ID_Envio = p_id_envio;
 select ID_Paquete into v_paquete from Paquetes where ID_Paquete = p_id_paquete;
```

```
if v_envio is null then
    set mensaje = 'Envio inexistente';
    rollback;
elseif v_paquete is null then
    set mensaje = 'Paquete inexistente';
    rollback;
else
    insert into Envio_Paquetes (ID_Envio, ID_Paquete) values (p_id_envio, p_id_paquete);
    set mensaje = 'Registro ingresado con exito';
    commit;
end if;

end //
 delimiter ;

delimiter //
 create procedure eliminar_envio_paquete(
 in p_id_envio int unsigned,
 in p_id_paquete int unsigned,
 out mensaje varchar(100)
 )
 begin
 declare v_relacion int default 0;
 start transaction;
 select count(*) into v_relacion from Envio_Paquetes
 where ID_Envio = p_id_envio and ID_Paquete = p_id_paquete;

if v_relacion = 0 then
    set mensaje = 'No existe esa relacion envio-paquete';
    rollback;
else
    delete from Envio_Paquetes where ID_Envio = p_id_envio and ID_Paquete =
p_id_paquete;
    set mensaje = 'Relacion eliminada';
    commit;
end if;

end //
 delimiter ;

delimiter //

create PROCEDURE eliminar_paquetes2(in id_env int)

        BEGIN
```

```
        delete from envio_paquetes where envio_paquetes.ID_Envio=id_env;
    end//
    delimiter;
```

```
-- procedimientos para Empresa_Adherida

delimiter //
 create procedure agregar_empresa_adherida(
 in p_nombre varchar(50),
 in p_cuit varchar(13),
 in p_direccion varchar(100),
 in p_telefono int,
 in p_rubro varchar(100),
 in p_duenio varchar(50),
 out mensaje varchar(100)
 )
 begin
 start transaction;
 insert into Empresa_Adherida (nombre, cuit, direccion, telefono, rubro, duenio)
 values (p_nombre, p_cuit, p_direccion, p_telefono, p_rubro, p_duenio);
 set mensaje = 'Registro ingresado';
 commit;
 end //
 delimiter ;

*DELIMITER //


CREATE PROCEDURE editar_empresa_adherida(

    IN p_id_empresa INT UNSIGNED,

    IN p_nombre VARCHAR(50),

    IN p_cuit VARCHAR(13),

    IN p_direccion VARCHAR(100),

    IN p_telefono INT,
```

```
    IN p_rubro VARCHAR(100),
    IN p_duenio VARCHAR(50),
    OUT mensaje VARCHAR(150)
)
BEGIN
    DECLARE auxiliar INT UNSIGNED DEFAULT NULL;
    START TRANSACTION;

    SELECT ID_empresa INTO auxiliar
    FROM Empresa_Adherida
    WHERE ID_empresa = p_id_empresa;

    IF auxiliar IS NULL THEN
        SET mensaje = 'No existe la empresa adherida';
        ROLLBACK;
    ELSE
        UPDATE Empresa_Adherida
        SET
            nombre = IFNULL(p_nombre, nombre),
            cuit = IFNULL(p_cuit, cuit),
            direccion = IFNULL(p_direccion, direccion),
            telefono = IFNULL(p_telefono, telefono),
            rubro = IFNULL(p_rubro, rubro),
            duenio = IFNULL(p_duenio, duenio)
        WHERE ID_empresa = p_id_empresa;
```

```
        SET mensaje = 'Se han editado los campos proporcionados';

        COMMIT;

    END IF;

END //


DELIMITER ;




delimiter //
 create procedure eliminar_empresa_adherida(in p_id_empresa int unsigned, out mensaje
varchar(150))
 begin
 declare v_empresa int unsigned default null;
 declare v_envios int default 0;
 start transaction;
 select ID_empresa into v_empresa from Empresa_Adherida where ID_empresa =
p_id_empresa limit 1;

if v_empresa is null then
    set mensaje = 'No existe la empresa adherida';
    rollback;
else
    select count(*) into v_envios from Envios where ID_Empresa = p_id_empresa;
    if v_envios > 0 then
        set mensaje = 'La empresa tiene envios asociados';
        rollback;
    else
        delete from Empresa_Adherida where ID_empresa = p_id_empresa;
        set mensaje = 'Empresa adherida eliminada';
        commit;
    end if;
end if;

end //
 delimiter ;

delimiter //
```

```
	create procedure mostrar_adheridos()
		begin
			select * from empresa_adherida;
		end//
	delimiter;
delimiter //
create procedure buscar_Adheridos(in p_id_empresa int)
	begin
		select * from empresa_adherida where ID_empresa=p_id_empresa;
	end//
delimiter;
```

FILTRAR POR NOMBRE (A-Z):

```
delimiter //
create procedure empresa_nombre_az()
begin
	select *
   from empresa_adherida
```

```
   order by nombre asc;
   end //
delimiter ;
```

FILTRAR POR NOMBRE (Z-A):

```
delimiter //
create procedure empresa_nombre_za()
begin
        select *
   from empresa_adherida
   order by nombre desc;
   end //
delimiter ;
```

FILTRAR POR RUBRO

```
delimiter //
create procedure empresa_rubro()
begin
        select *
   from empresa_adherida
   order by rubro asc;
   end //
delimiter ;
```

FILTRAR POR DUEÑO(A-Z):

```
delimiter //
create procedure empresa_duenio()
begin
        select *
   from empresa_adherida
   order by duenio asc;
   end //
delimiter ;
```

FILTRAR POR DUEÑO(Z-A):

```
delimiter //
create procedure empresa_duenio()
begin
        select *
   from empresa_adherida
   order by duenio desc;
   end //
delimiter ;
```

UREÑA:

Procedimientos:

Agregar Tickets:

```
DELIMITER //

CREATE PROCEDURE agregarTickett(
   IN idCliente INT,
   IN fechaN DATE,
   IN detalleN VARCHAR(100),
   IN totalN INT,
   IN metodospagoN VARCHAR(25),
   OUT mensaje VARCHAR(200)
)
BEGIN
   DECLARE v_existe INT DEFAULT 0;

   START TRANSACTION;
      SELECT COUNT(*) INTO v_existe
      FROM Clientes
      WHERE ID_Cliente = idCliente;

      IF v_existe = 0 THEN
         SET mensaje = 'Error: El cliente no existe. Transacción cancelada.';
         ROLLBACK;
      ELSE
         INSERT INTO Tickets (ID_Cliente, fecha, detalle, total, metodospago)
         VALUES (idCliente, fechaN, detalleN, totalN, metodospagoN);

         COMMIT;
         SET mensaje = 'Ticket creado con éxito';
```

```
      END IF;
END //

DELIMITER ;
```

Eliminar Tickets:

```
DELIMITER //
CREATE PROCEDURE eliminarTicket (in p_idTicket int, out mensaje varchar(100))
BEGIN
        DECLARE contador_reportesTickets int;
      START TRANSACTION;
        SELECT COUNT(reportes_ticket.ID_rt) INTO contador_reportesTickets
FROM reportes_ticket WHERE reportes_ticket.ID_Ticket=p_idTicket ;
         IF contador_reportesTickets>0 THEN
                SET mensaje='No se puede eliminar un ticket que este en la tabla
reportes tickets';
            ROLLBACK;
           ELSE
             DELETE FROM tickets
             WHERE tickets.ID_Ticket =p_idTicket ;
             COMMIT;
             SET mensaje='Se ha realizado con exito';
             END IF;
END //
DELIMITER ;
 DELIMITER //
```

Editar Ticket:

```
create procedure editar_ticket(in id_ticket int,
   in id_cliente int,
   IN fechas date,
   in detalles varchar(1000),
   in ttotal int,
   in metodoPago varchar(100),
   OUT mensaje VARCHAR(200)
)
BEGIN
        declare auxiliar varchar(100);
      start TRANSACTION;
      select tickets.detalle into auxiliar from tickets where tickets.ID_Ticket =id_ticket ;
         if auxiliar is null THEN
                set mensaje='No existe el ticket ';
```

```
            rollback;
        else
              if id_cliente  is not null THEN
              update tickets
            set ID_Cliente = id_cliente
            where ID_Ticket=id_ticket;
            end if;
          if fechas  is not null THEN
              update tickets
            set fecha=fechas
            where ID_Ticket=id_ticket;
            end if;
          if detalles is not null THEN
              update tickets
            set detalle =detalles
            where  ID_Ticket=id_ticket;
            end if;
           if ttotal is not null THEN
              update tickets
            set total=ttotal
            where ID_Ticket=id_ticket;
            end if;
           if metodoPago is not null THEN
              update tickets
            set metodosPago=metodoPago
            where ID_Ticket=id_ticket;
            end if;
            set mensaje='Se ha encontrado el ticket, se editaron los campos
proporcionados correctamente';
            commit;
            end if;
end //
delimiter ;
```

Agregar Reportes:

```
DELIMITER //
create procedure agregar_reportes(in detallee varchar(1000), out mensaje
varchar(100))
begin
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   begin
      SET mensaje = "Error. Operacion cancelada";
      rollback;
```

```
    end;
    start transaction;
    insert into Reportes( detalle)
    values (detallee);
    commit;
    set mensaje = "Reporte registrado correctamente.";
end //
DELIMITER ;
```

Eliminar Reportes:

```
DELIMITER //

CREATE PROCEDURE eliminarReporte(
    IN p_idReports INT,
    OUT mensaje VARCHAR(100)
)
BEGIN
    DECLARE contador_reportesTickets INT DEFAULT 0;

    START TRANSACTION;
        SELECT COUNT(*)
        INTO contador_reportesTickets
        FROM reportes_ticket
        WHERE reportes_ticket.ID_Reporte = p_idReports;

        IF contador_reportesTickets > 0 THEN
            SET mensaje = 'No se puede eliminar un reporte que este en la tabla reportes
tickets';
            ROLLBACK;
        ELSE
            DELETE FROM reportes
            WHERE reportes.ID_Reporte = p_idReports;

            SET mensaje = 'Se ha realizado con exito';
            COMMIT;
        END IF;
END //

DELIMITER ;
```

Editar Reportes:

```
 DELIMITER //

CREATE PROCEDURE editar_reporte(
    IN id_Reporte INT,
```

```
    IN detallee VARCHAR(1000),
    OUT mensaje VARCHAR(200)
)
BEGIN
    DECLARE existe_reporte INT DEFAULT 0;

    START TRANSACTION;
        SELECT COUNT(*) INTO existe_reporte
        FROM reportes
        WHERE reportes.ID_Reporte = id_Reporte;

        IF existe_reporte = 0 THEN
            SET mensaje = 'No existe el reporte';
            ROLLBACK;
        ELSE
            IF detallee IS NOT NULL THEN
                UPDATE reportes
                SET detalle = detallee
                WHERE reportes.ID_Reporte = id_Reporte;
            END IF;

            SET mensaje = 'Se ha encontrado el reporte, se editaron los campos
proporcionados correctamente';
            COMMIT;
        END IF;
END //

DELIMITER ;
```

Agregar Reportes_ticket:

```
DELIMITER //
create procedure agregarReportes_Ticket(in idReporte int, in idTicket int, out mensaje
varchar(75))
begin
declare existe_Reporte int default null;
declare existe_Ticket int default null;

select ID_Reporte into existe_Reporte from reportes
where ID_Reporte = idReporte;
select ID_Ticket into existe_Ticket from tickets
where ID_Ticket = idTicket;

if existe_Reporte is null or existe_Ticket is null then
set mensaje = "Error. El ticket o reporte no existe.";
else
insert into reportes_ticket(ID_Reporte, ID_Ticket)
values(idReporte, idTicket);
```

```
set mensaje = "Union Reportes-Ticket registrada correctamente.";
end if;
end //
DELIMITER ;
```

Eliminar Reportes_ticket:

```
DELIMITER //
create procedure eliminar_ReportesTicket (in id_ReporteTicket int, out mensaje varchar(75))
begin
declare existe_ReporteTicket int default null;
select ID_rt  into existe_ReporteTicket from reportes_ticket
where ID_rt = id_ReporteTicket;

if existe_ReporteTicket is null then
set mensaje = "Error. La union Reportes_ticket ingresada no existe.";
rollback;
else
        delete from reportes_ticket
   where ID_rt = id_ReporteTicket;
   set mensaje = "Union Reportes_ticket eliminada correctamente.";
end if;
end //
DELIMITER ;
```

Editar Reportes_ticket:

```
DELIMITER //
create procedure editar_ReporteTicket (in id_reporteTicket int, in idReporte int, in idTicket
int, out mensaje varchar (75))
begin
declare existe_reporteTicket int default null;

select ID_rt into existe_reporteTicket from reportes_ticket
where ID_rt = id_reporteTicket;

if existe_reporteTicket is null then
set mensaje = "Error. La union Reporte_ticket ingresada no existe.";
rollback;
else
        if idReporte is not null then
                update reportes_ticket
      set ID_Reporte = idReporte
      where ID_rt = id_reporteTicket;
        end if;
        if idTicket is not null then
                update reportes_ticket
```

```
        set ID_Ticket = idTicket
        where ID_rt = id_reporteTicket;
           end if;
set mensaje = "Editado correctamente.";
commit;
end if;
end //
DELIMITER ;
```

Para realizar estos procedures use los conocimientos que tengo y si me olvidaba de algo consultaba en trabajos realizados por mi durante el año en la materia de base de datos.

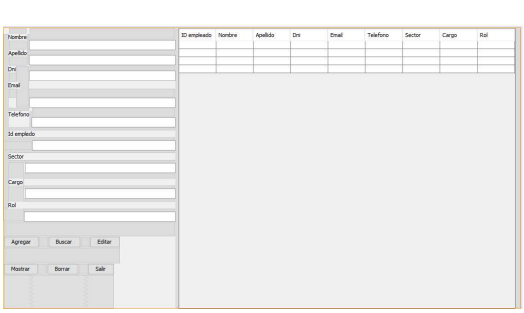
13/9

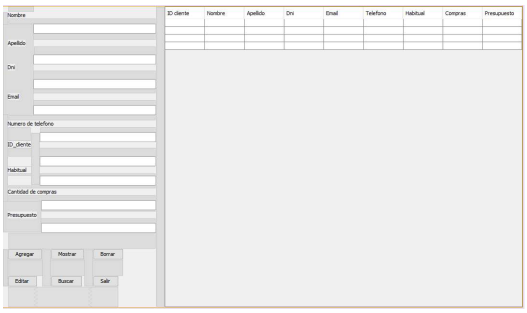
ZOWMIR:En esta fecha, me dispuse a realizar el trabajo que se me había asignado el día 9/9, que consistía en desarrollar los procedimientos almacenados de la tabla **Almacenes**.

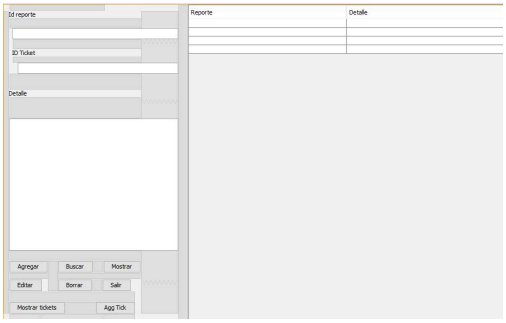
Al principio tuve algunas dificultades porque no recordaba bien cómo funcionaban estos procedimientos. Por ese motivo, decidí volver a repasar la teoría para comprender mejor cómo se creaban y de qué manera podían utilizarse en nuestra base de datos. Una vez que tuve más claros los conceptos, comencé a desarrollar los procedimientos correspondientes a mi tabla.

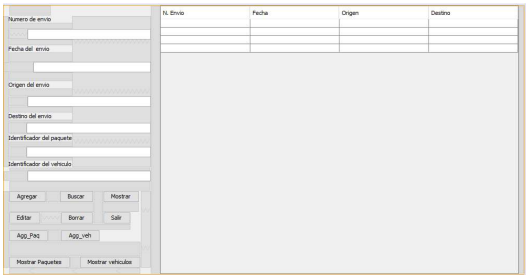
14/10

SARAGUSTI:El 14 de septiembre hice las 4 primeras ventanas de lo de poo. También modifique un poco el código de las clases para la futura gran modificación necesitada por el cambio de modalidad visto para usar pantallas en vez de puro joptionPane. Hice las ventanas de empleados, clientes, reportes y envíos pero solo la parte visual y no el código porque eso después se va a distribuir entre todos los integrantes del grupo. Y como algunas de las tablas que no son la main controller tienen listas hice un botón en envíos y en reportes para mostrar respectivamente los tickets y los paquete y vehículos pero solo usando joption porque no hay espacio para tablas específicas de mostrar esos campos









(Como se veían las interfaces gráficas sin codificar, aunque a día de hoy 25/9 se siguen viendo casi exactamente iguales. Pero con el código agregado claro está)

15/09

ROSATO:15 de Septiembre de 2026

Se avanzó con el diseño e implementación técnica de los procedimientos almacenados correspondientes a la capa de manipulación de datos para las tablas de Productos, Paquetes y la entidad relacional intermedia Productos_Paquetes. En esta fase, el desarrollo se centró en la optimización de las consultas estructuradas de inserción, actualización y filtrado paramétrico, asegurando que la comunicación entre el sistema de persistencia y la lógica del negocio fuera eficiente.

16 de septiembre:

Saragusti:El 16 de septiembre hice la pestaña de agregar cuentas, que convierte los datos puestos en un diccionario de datos para mandarlo a la lista del json y poder usarlo en el inicio de sesión, por ahora los valores que importan son solo el dni como contraseña y el gmail como el nombre de usuario, comprobé que además los datos por defecto se pueden acceder luego de la inserción, Además después de cargar los datos te lleva directo a la pestaña de inicio de sesión para poder cargar la cuenta recien creada, en un futuro se van a añadir

directamente a la base de datos y los datos que va a contener el diccionario será el nombre, id y el dni como contraseña para poder hacer los envíos sin estar requiriendo de poner una y otra vez el identificador del usuario

(Visión de la pestaña de creación de cuenta, la cual solo tomaba los datos de dni y de email hasta el momento)

17 de septiembre:

Pedraza: Una vez que todos los integrantes del grupo finalizaron con sus procedures y fueron corroborados por nosotros mismos, el integrante Benjamin Saragusti nos asignó a la semana siguiente (aproximadamente el **17/9**) que cada integrante debía realizar cincuenta inserts por cada tabla (de las que había cada uno realizado los procedures).

De esos cincuenta inserts, cinco debían ser realizados mediante los procedures que hicimos, y los restantes podían ser hardcodeados con el tradicional ‘’insert into values’’.

Por lo tanto, durante esa semana, realice mi total de ciento cincuenta inserts correspondientes a las tablas “Empleados” (50), “Vehiculos” (50) y “Envio_vehiculo” (50).

Los inserts en los cuales se aplican mis procedures quedaron de la siguiente manera:

CALL agregar_empleados('Carlos', 'Gómez', 32451892, 'carlos.gomez01@logistica.com', 11456701, null, null, null, @mensaje);

CALL agregar_empleados('María', 'Fernández', 28912304, 'maria.fernandez02@logistica.com', 11456702, null, null, null, @mensaje);

CALL agregar_empleados('Juan', 'Pérez', 41058291, 'juan.perez03@logistica.com', 11456703, null, null, null, @mensaje);

CALL agregar_empleados('Lucía', 'Martínez', 35781902, 'lucia.martinez04@logistica.com', 11456704, null, null, null, @mensaje);

CALL agregar_empleados('Santiago', 'López', 29384710, 'santiago.lopez05@logistica.com', 11456705, null, null, null, @mensaje);

Los atributos “Sector”, “Cargo” y “Rol” los definí como null debido a que aún no habíamos decidido bien cuales serían dichos atributos.

El 17 de septiembre hice modificaciones a la entidad empresa de java añadiendo el array de tickets, agregar tickets, la opción de buscar tickets en el buscador como también la opción de eliminar tickets y el editar tickets. No solo eso sino que rompí el editor universal en pequeños editar por cada clase ya que al recibir los atributos de un jframe seria mas practico hacerlo de esa manera.

Además codifique la ventana del cliente y ayude a Pedraza a hacer la ventana del Empleado.

Saragusti:El 17 de septiembre hice modificaciones a la entidad empresa de java añadiendo el array de tickets, agregar tickets, la opción de buscar tickets en el buscador como también la opción de eliminar tickets y el editar tickets. No solo eso sino que rompí el editor universal en pequeños editar por cada clase ya que al recibir los atributos de un jframe seria mas practico hacerlo de esa manera.

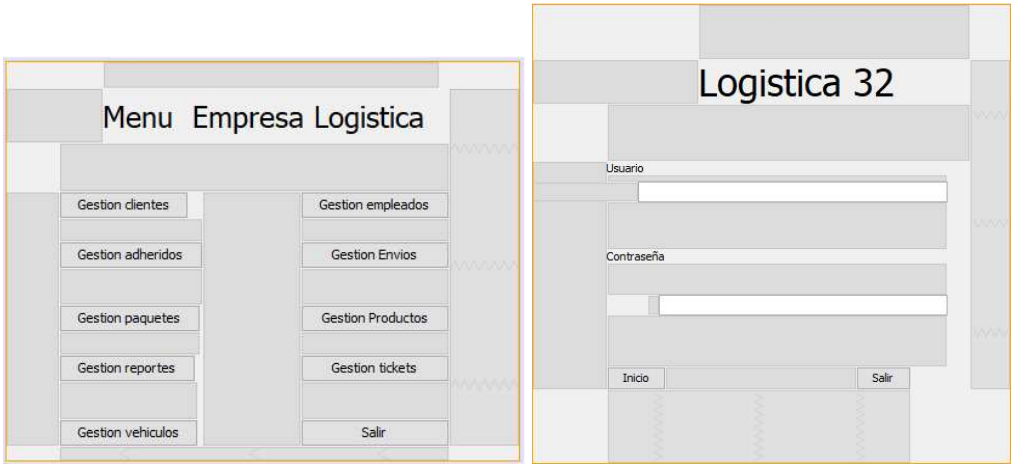
Además codifique la ventana del cliente y ayude a Pedraza a hacer la ventana del Empleado.

Xiomara:Una vez que todos los integrantes del grupo finalizaron con sus procedures y fueron corroborados por nosotros mismos, Benjamin nos asignó que cada integrante debía realizar cincuenta inserts por cada tabla, de las que cada uno habia realizado los procedures.

Realicé mis 150 insterts correspondientes a las tablas “Tickets”(50), “Reportes”(50) y Reportes_ticket(50) durante esa semana.

18 de septiembre:

Saragusti: El 18 de septiembre codifique la ventana de inicio de sesión con contraseña de usuarios predeterminados y además de eso programe el menú que te deja acceder a todas las gestiones de cada clase.



(Vista de las ventanas previamente mencionadas, con el menú y las opciones de salir. En el caso del menú todas las gestiones administrativas, por dentro de cada opcion del menu se abre una ventana que comparte la ventana del menú, no la de inicio de sesión, la misma entidad de empresa y el mismo contador de errores)

20 de septiembre:

Rosato:Se finalizó con éxito el proceso de generación e inserción de datos de prueba en el entorno de persistencia relacional. Se completó de forma masiva un lote de 50 registros de inserción exhaustivos dentro de la tabla de Paquetes. Cada uno de estos registros fue testeado de manera individual y validado bajo las reglas de negocio y restricciones lógicas de integridad establecidas en el modelo del sistema logístico.

21 de septiembre:

Rosato:Se dio continuidad y cierre formal a las tareas de poblamiento inicial de la base de datos relacional. Se concretó la inserción de 50 registros de prueba adicionales en las tablas de Productos y en la tabla intermedia

Productos_Paquetes. Mediante este procedimiento, se logró consolidar un conjunto de datos robusto, consistente y con integridad referencial completa, indispensable para dar inicio a la posterior fase de auditoría y pruebas integrales del sistema informático.

Zowmir:En esta fecha, me dispuse a agregar registros a la tabla Clientes mediante sentencias INSERT. Esta tarea no presentó demasiadas dificultades, ya que consistía principalmente en completar los campos correspondientes de la tabla con los datos necesarios para registrar nuevos clientes en la base de datos.

De esta manera, pude incorporar información de prueba y contar con registros que luego podrían utilizarse para realizar consultas y comprobar el funcionamiento del sistema.

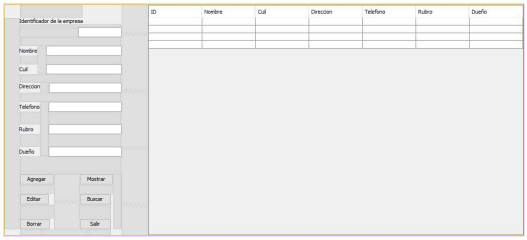
```
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Albert', 'Einstein', 12273827, 'albert.einstein@logistica.com', 18035309, 'Calle Falsa 553', 5197);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Isaac', 'Newton', 30699264, 'isaac.newton@logistica.com', 92202750, 'Av. Jujuy 1106', 1960);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Marie', 'Curie', 20455897, 'marie.curie@logistica.com', 93105204, 'Av. San Martín 2394', 1071);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Leonardo', 'da Vinci', 16219221, 'leonardo.davinci@logistica.com', 92753255, 'Av. Corrientes 574', 1872);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Aristoteles', 'Filosofo', 43717652, 'aristoteles.filosofo@logistica.com', 45899093, 'Av. San Martín 1233', 1843);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Platon', 'Filosofo', 26308719, 'platon.filosofo@logistica.com', 47005432, 'Av. de Mayo 2583', 3525);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Charles', 'Darwin', 19817371, 'charles.darwin@logistica.com', 19502587, 'Av. de Mayo 732', 5752);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Galileo', 'Galilei', 36064681, 'galileo.galilei@logistica.com', 53040039, 'Av. Rivadavia 1765', 5757);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Nikola', 'Tesla', 44615896, 'nikola.tesla@logistica.com', 03900854, 'Av. Belgrano 1312', 3843);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Thomas', 'Edison', 13931670, 'thomas.edison@logistica.com', 55895790, 'Av. Belgrano 2706', 8897);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Alexander', 'Fleming', 31257041, 'alexander.fleming@logistica.com', 02731180, 'Av. de Mayo 2985', 4737);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Louis', 'Pasteur', 15312706, 'louis.pasteur@logistica.com', 24784161, 'Calle Florida 593', 1788);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Sigmund', 'Freud', 25777763, 'sigmund.freud@logistica.com', 42310959, 'Calle Falsa 3356', 7208);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Karl', 'Marx', 43875036, 'karl.marx@logistica.com', 35964268, 'Calle Florida 2070', 7712);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Adam', 'Smith', 25313497, 'adam.smith@logistica.com', 56153267, 'Av. Corrientes 4001', 2276);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Mahatma', 'Gandhi', 21831225, 'mahatma.gandhi@logistica.com', 08216604, 'Av. de Mayo 2967', 8010);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Nelson', 'Mandela', 34258499, 'nelson.mandela@logistica.com', 14597377, 'Av. San Martín 2788', 8683);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Martin Luther', 'King', 30391330, 'martinluther.king@logistica.com', 64376631, 'Av. Rivadavia 3671', 6965);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('George', 'Washington', 23271293, 'george.washington@logistica.com', 66431004, 'Av. Rivadavia 2768', 2873);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Simon', 'Bolivar', 36704200, 'simon.bolivar@logistica.com', 97026233, 'Av. de Mayo 3237', 7379);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('San', 'Martin', 20738274, 'san.martin@logistica.com', 13351971, 'Av. Callao 3283', 2411);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Julio', 'Cesar', 29174908, 'julio.cesar@logistica.com', 48202936, 'Av. Santa Fe 578', 8054);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Alejandro', 'Magno', 21567139, 'alejandro.magno@logistica.com', 11755580, 'Calle Florida 1601', 9333);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Wolfgang', 'Mozart', 26010805, 'wolfgang.mozart@logistica.com', 40926861, 'Av. Jujuy 4496', 6601);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ludwig van', 'Beethoven', 37885811, 'ludwigvan.beethoven@logistica.com', 57926920, 'Calle Falsa 3530', 1601);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Diego', 'Maradona', 22465967, 'diego.maradona@logistica.com', 96837236, 'Av. Santa Fe 3872', 5262);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Lionel', 'Messi', 41202056, 'lionel.messi@logistica.com', 44270298, 'Av. Santa Fe 3005', 6140);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Juan Roman', 'Riquelme', 44384038, 'juanroman.riquelme@logistica.com', 45418689, 'Av. Rivadavia 3727', 6127);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Martin', 'Palermo', 28571568, 'martin.palermo@logistica.com', 99740657, 'Av. Rivadavia 3646', 2930);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Enzo', 'Francescoli', 41077261, 'enzo.francescoli@logistica.com', 38175105, 'Av. Belgrano 2981', 8016);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ariel', 'Ortega', 38473975, 'ariel.ortega@logistica.com', 96876577, 'Av. Belgrano 1315', 6420);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Carlos', 'Tevez', 12094861, 'carlos.tevez@logistica.com', 01812506, 'Av. Rivadavia 442', 6442);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Sergio', 'Aguero', 10086850, 'sergio.aguero@logistica.com', 55228240, 'Calle Florida 4061', 7245);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Angel', 'Di Maria', 18329079, 'angel.dimaria@logistica.com', 39356367, 'Av. Santa Fe 3403', 7738);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Gabriel', 'Batistuta', 15302771, 'gabriel.batistuta@logistica.com', 28588355, 'Av. Belgrano 1865', 5031);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Hernan', 'Crespo', 29376766, 'hernan.crespo@logistica.com', 17749154, 'Av. Santa Fe 2325', 6569);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Claudio', 'Caniggia', 26786311, 'claudio.caniggia@logistica.com', 22003670, 'Calle Falsa 507', 9048);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Mario', 'Kempes', 37144547, 'mario.kempes@logistica.com', 86809615, 'Av. San Martín 2344', 3762);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Daniel', 'Passarella', 10805063, 'daniel.passarella@logistica.com', 28775364, 'Av. Rivadavia 3666', 6539);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ubaldo', 'Fillol', 38922226, 'ubaldo.fillol@logistica.com', 21576356, 'Calle Florida 4375', 6217);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ricardo', 'Bochini', 43561837, 'ricardo.bochini@logistica.com', 33084111, 'Av. Santa Fe 2310', 9079);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Norberto', 'Alonso', 38242606, 'norberto.alonso@logistica.com', 99419529, 'Av. Callao 362', 2323);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Rene', 'Houseman', 18600586, 'rene.houseman@logistica.com', 74545965, 'Av. de Mayo 4391', 6759);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Javier', 'Zanetti', 42229798, 'javier.zanetti@logistica.com', 63119857, 'Av. Santa Fe 1995', 1878);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Diego', 'Simeone', 34412200, 'diego.simeone@logistica.com', 83718168, 'Av. de Mayo 2009', 8944);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Juan Sebastian', 'Veron', 20657150, 'juansebastian.veron@logistica.com', 13882605, 'Av. de Mayo 2364', 6520);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Fernando', 'Redondo', 13131666, 'fernando.redondo@logistica.com', 12206316, 'Av. San Martín 2612', 2446);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Javier', 'Mascherano', 29971903, 'javier.mascherano@logistica.com', 92421769, 'Calle Florida 3740', 2141);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Gonzalo', 'Higuain', 15363821, 'gonzalo.higuain@logistica.com', 06635937, 'Av. Santa Fe 981', 1101);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Julian', 'Alvarez', 33896037, 'julian.alvarez@logistica.com', 04766667, 'Av. Corrientes 3923', 2859);
```

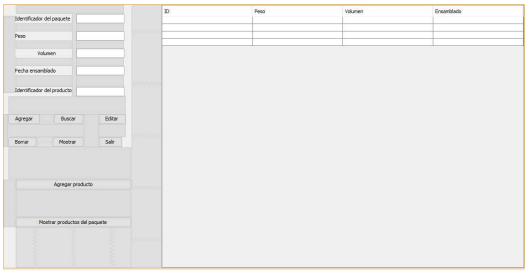
fotos sacadas del git hub

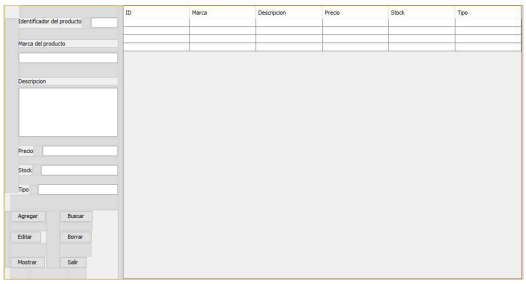
22 de septiembre

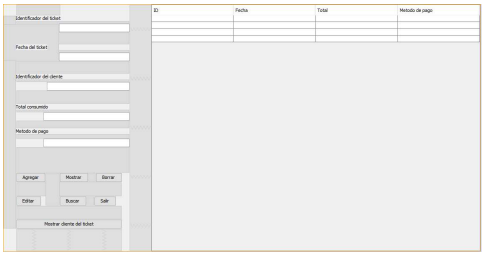
Saragusti:El 22 de septiembre termine de codificar la ventana de la clase envío y de la clase reportes, estas dos al tener arrays adentro les tuve que hacer botones específicos. En el caso de envíos tuve que hacer el botón de agregar paquetes, agregar vehículos y mostrar esos dos separados respectivamente, el tema es que los muestran con joptionpane, para verlos en tabla hay que ir a la ventana correspondiente pero bueno. Después hice 5 ventanas faltantes con

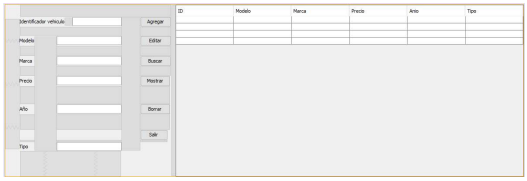
solo la parte visual ya que la parte codificada la voy a realizar mañana. Además lo de agregar a arrays se hace usando el identificador único de la clase a la que quieras agregar y se usa el buscador universal que permite no tener que tipear cada clase dos veces. El tema es que si no creas por ejemplo paquetes no vas a poder agregarlos a envíos por el tema de la dependencia, pero eso es normal en cualquier tipo de programa y se va a ir cuando tengamos la base de datos sincronizada con el código.










(Las ventanas ya hechas gráficamente, realizadas y que en lo que respecta al diseño se mantienen por ahora, despues se vera de modificar los botones y los colores para darles un toque más vivo)

23 de septiembre:

Pedraza:El día miércoles **23/09**, Benjamin Saragusti nos asigno a mis compañeros Nicolas Rosato y Xiomara Ureña a realizar modificaciones al código de JAVA en el que él había

estado trabajando durante la semana. La parte que nos asignó corregir era la correspondiente a las primeras versiones de las ventanas interactivas JFrame correspondientes a los CRUD (sin contar aún con la conexión a la base de datos).

Si bien no cuento con las pruebas originales, recuerdo que las modificaciones consistían principalmente en agregar validaciones básicas en los campos a completar de las tablas, por ejemplo, verificar que los campos no estén vacíos al intentar agregar algún elemento. Además, tuvimos que agregar un contador el cual, al detectar diez veces un error al querer realizar alguna tarea en una misma ventana, cerraba el programa.

Al principio, nuestras validaciones no funcionaron como correspondían, por lo que Goldfarb y Saragusti verificaron el código y encontraron una solución.

Rosato:Se realizó una jornada de codificación colaborativa junto a los compañeros Santiago Pedraza y Xiomara Ureña sobre el módulo principal del backend de la aplicación. Durante esta sesión, se diseñó e implementó un mecanismo de seguridad para la autenticación de usuarios que limita de forma estricta la cantidad máxima de intentos de acceso permitidos antes del bloqueo y cierre automático de la terminal del programa. Asimismo, se incorporó una rigurosa validación de tipos de datos y caracteres en las máscaras de entrada de los formularios de la interfaz de usuario, restringiendo el ingreso de datos únicamente al formato sintáctico correspondiente para mitigar fallas en tiempo de ejecución o inyecciones de código.

Saragusti:El 23 de septiembre realice el código final de las ventanas que quedaban, además de editar el contenedor de contraseña de la pestaña de login, haciendo que lo que ingreses no sea visible. Para eso use el password field of text que permite ocultarlo pero que funciona exactamente igual en lo que a código respecta. Probé la mayoría de cosas y funcionaban, faltan igualmente varios aspectos a corregir como la probabilidad de ingresar una letra en un número y el contador pero esas partes las realizarán mis compañeros porque si no lo estaría

haciendo todo yo. Hay ventanas específicas que tienen un botón para mostrar sus arrays con joptionpane porque no tenía el espacio para meter los datos de las clases auxiliares, pero para su mejor visibilización puedes acceder a la tabla que le corresponde a esa y ver los datos específicos. También al editar por ejemplo un cliente que tiene un ticket, en el apartado de ver cliente de ticket se podrá ver actualizado los datos y no los anteriores. Los arrays no se usa get porque son públicos y no privados, pero funcionan de manera igual solo que sin el get y el set. Falta menos para terminar todo lo de septiembre. El código de las ventanas presenta muchas similitudes porque me baso en la misma base para realizarlas, todas se conectan y acceden a la misma instancia de Empresa y al mismo contador y este se incrementa con cada error, Las verificaciones aplicadas hasta ahora por mi son pocas y básicas como por ejemplo: Nombres y textos sin numero verifique que no cuente con estos últimos. El correo y el teléfono están escritos de manera correcta junto también con su longitud, el dni con la longitud correcta. Luego se tendría que aplicar la condición de que si se trata de convertir un string a int y encuentra una letra que fallase antes que nada, así no salta un error en consola y se incrementa el medidor de errores. La función de cerrar la ventana cuando llega al tope se hará con el comando de system(0) que es el que se usa también para cerrar cuando tocas cerrar sesión, aunque también contará con una ventana indicando que llegaste al máximo de errores.

(Como se ve la contraseña al ser escrita por una persona para que no la puedas ver y sea privada)

Xiomara:El día miércoles 23, Benjamin Saragusti nos asigno a mis compañeros Nicolas Rosato y Santiago Pedraza a realizar modificaciones al código de JAVA en el que él había estado trabajando durante la semana. La parte nos asignó corregir era la correspondiente a las

primeras versiones de las ventanas interactivas JFrame correspondientes a los CRUD (sin contar aún con la conexión a la base de datos).

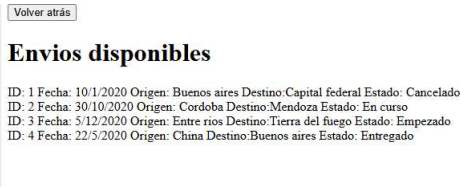
Las modificaciones consistían principalmente en agregar validaciones básicas en los campos a completar de las tablas, por ejemplo, verificar que los campos no estén vacíos al intentar agregar algún elemento. Además, tuvimos que agregar un contador el cual, al detectar diez veces un error al querer realizar alguna tarea en una misma ventana, cerraba el programa.

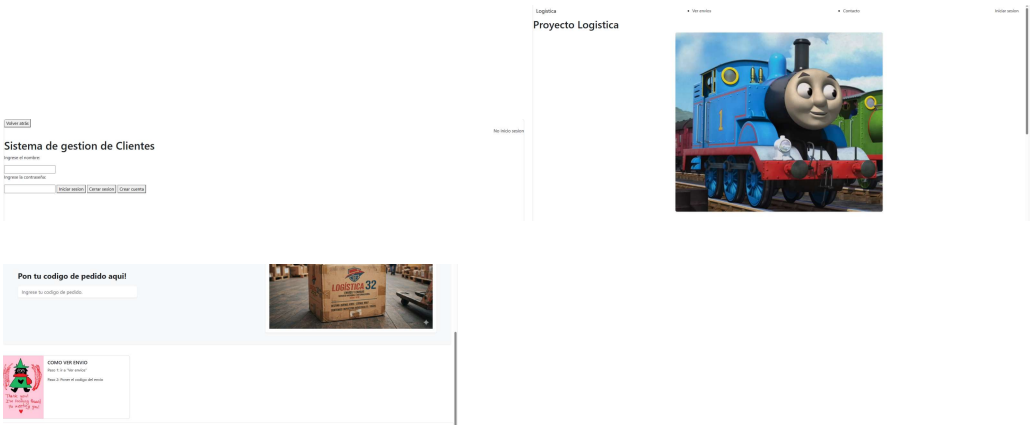
Al principio, nuestras validaciones no funcionaron como correspondía por lo que Golfarb y Saragusti verificaron el código y encontraron una solución.

24 de septiembre:

Saragusti:El 24 de septiembre estuve colaborando con html, conectando la ventana del índice con las demás ventanas, creando la ventana de ver envíos y la de ver un solo envio, expandiendo el uso de json en el localstorage para que apareciera el nombre también en el índice y no solo en iniciar sesion, añadir funcionalidades al botón de contacto que ahora tiene un alert con el contacto. Colabore en la muestra de cómo realizar una comprobación numérica en java, volviendo al número un null para que la llamada de vacío le haga return y tire un error.

Hice también en html un botón que sirve para volver a la pestaña anterior por si tocas un boton y te arrepientes,Hay dos listas principales de json actualmente, la que contiene los datos de los clientes y la que contiene algunos envíos, ambas tienen distintos nombres y se usan para distintos escenarios.





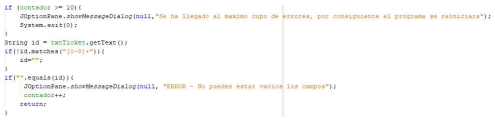
(Como se ven las ventanas creadas y finales hasta el dia de la fecha, que usan json para conectarse entre sí con el objetivo de permitir la compartición de datos sin necesitar copiarlos y pegarlos manualmente)

25 de septiembre:

Saragusti:El 25 de septiembre creó un borrador de página para la red de cisco porque todavía no podemos meter las páginas enteras debido a que no están terminadas.

Tambien le añadi algunas correcciones con respecto a la terminada de java, debido a que se cerraba solo sin decir nada y ahora aclara que se termina el sistema debido a que se habían cometido suficientes errores en el uso del sistema, este contador es universal y no es uno por ventana, se van compartiendo para que no se tengan que hacer cuentas raras con respecto a la manera de mantener sincronizado el ritmo.

(Vista de la ventana desde cisco packet tracer, la cual es solo la base para este programa, despues se iran agregando las ventanas que hacemos para proyecto informático en el buscador)



```
if (contador >= 10){
    JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
    System.exit(0);
}
String id = txtTicket.getText();
if(!id.matches("[0-9]+")){
    id="";
}
if("".equals(id)){
    JOptionPane.showMessageDialog(null, "ERROR - No pueden estar vacios los campos");
    contador++;
    return;
}
```

(Verificación de contador, verificación de que los datos de un string sean números antes de convertir, en el caso de que falle se vuelve el número uno “” para que la verificación de vacío lo tome como error y salte, para no tener que hacer dos verificaciones y solo hacer una)

26 de septiembre a 30 de septiembre:

Rosato:30 de Septiembre de 2026

Se dedicó la jornada técnica a la redacción, ordenamiento y estructuración formal de la Carpeta de Campo. Se recopilaron los diagramas, presupuestos, análisis de competencia y registros de actividades acumulados hasta el momento para conformar la entrega provisoria orientada a la supervisión, control de cambios y auditoría pedagógica del proyecto por parte del cuerpo docente.

Pedraza:Entre los días **26/9** y **30/9** realicé dos tareas prácticamente en simultáneo:

El día **25/7** se me había encargado realizar realizar una documentación con observaciones y un listado explicativo en el que se mostraría cómo funcionaban las ventanas del código de JAVA, pero durante la realización del mismo, descubrí que las ventanas presentaban muchas validaciones y detalles faltantes, así como también los aspectos visuales de las ventanas, los cuales se veían bastante primitivos. Por lo tanto, entre los tres días mencionados anteriormente me dediqué a mejorar los códigos de todas las ventanas de pies a cabeza, en el siguiente orden:

Entre los días **27/9 y 28/9**, comencé utilizando la ventana de gestión de clientes como mi conejillo de indias. Como todas las ventanas tenían más o menos las mismas fallas y detalles a corregir, mi idea fue revisar, corregir, y documentar de pies a cabeza todo el funcionamiento de esta ventana hasta que funcione al 100%, para así después hacer un calco con las demás ventanas.

Al principio, la documentación tenia mas formato de guia o manual para el usuario, pero a medida que iba encontrando los errores, pasó a ser una muestra de todas las cosas que corregí en la ventana clientes (no continué esta documentación con las demás ventanas, debido a que las funcionalidades de las ventanas debían estar terminadas para antes del **30/9**, dia en el que teníamos el punto de control de septiembre, y por lo tanto, realizar a la vez la documentación de todos los cambios me iba a llevar bastante más tiempo).

En la noche del **29/09** me dediqué a realizar todas las validaciones de cada una de las ventanas (Cliente, Adherido, Empleado, Envio, Paquete, Producto, Reporte, Ticket y Vehículo). Además, a varias ventanas les faltaban funcionalidades del CRUD (como Editar, Modificar, Borrar. Las de agregar estaban en todas las ventanas), por lo tanto, agregué todas las funcionalidades básicas de los CRUD que faltaban.
Como lo realice a media noche, lo finalice todo en la madrugada del **30/09**, horas antes del punto de control de septiembre.
Por último, mejore el aspecto de una de las ventanas, más precisamente la de ‘’Gestion de clientes’’, aunque continuaremos con eso luego de la entrega de Septiembre.

Asi se ve el código correspondiente a Agregar cliente, con todas sus validaciones incluidas:

```
    private void btnAgregarActionPerformed(java.awt.event.ActionEvent evt) {
        if (contador >= 10){
            JOptionPane.showMessageDialog(null,"Se ha llegado al maximo cupo de errores, por consiguiente el programa se reiniciara");
```

```
            System.exit(0);
        }
        String nombre = txtNombre.getText();
        String apellido = txtApellido.getText();
        String dni = txtDni.getText();
        String Email = txtEmail.getText();
        String telefono = txtTelefono.getText();
        String id_cliente=txtCliente.getText();
        String habi=txtHabitual.getText();
        String comp=txtCompras.getText();
        String pesu=txtPresupuesto.getText();

        // CAMPOS VACIOS
        if("".equals(nombre) || "".equals(dni) ||"".equals(apellido) ||"".equals(Email) ||
"".equals(telefono)||"".equals(id_cliente)||"".equals(comp)||"".equals(habi)||"".equals(pesu) ){
            JOptionPane.showMessageDialog(null, "ERROR - Los campos no pueden estar
vacios.");
            contador++;
           return;
        }
        // condicionales Nombre
        if(!nombre.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")){
            JOptionPane.showMessageDialog(null, "ERROR - El nombre solo puede contener
letras");
            contador++;
```

```
            return;
        }
        // condicionales Apellido
        if(!apellido.matches("[a-zA-ZáéíóúÁÉÍÓÚñÑ ]+")){
            JOptionPane.showMessageDialog(null, "ERROR - El apellido solo puede contener
letras");
            contador++;
            return;
        }
        // condicionales DNI
        if(!dni.matches("[0-9]+") && !dni.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El DNI solo puede contener
numeros.");
            contador++;
            return;
        }
        if(dni.length()!=8){
            JOptionPane.showMessageDialog(null, "ERROR - El DNI debe tener solo 8
digitos");
            contador++;
            return;
        }
        // condicionales email
        if(!Email.matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")){
```

```
            JOptionPane.showMessageDialog(null, "ERROR - Ingrese un correo electronico
valido");
            contador++;
            return;
        }
        // condicionales telefono
        if(!telefono.matches("[0-9]+") && !telefono.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El telefono solo puede contener
numeros.");
            contador++;
            return;
        }
        if(telefono.length()!=11 && telefono.length()!=10){
            JOptionPane.showMessageDialog(null, "ERROR - El telefono debe contar con 10 u
11 digitos");
            contador++;
            return;
        }
        // condicionales id_cliente
        if(!id_cliente.matches("[0-9]+") && !id_cliente.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - El ID solo puede contener
numeros enteros.");
            contador++;
            return;
        }
```

```
        int num2=Integer.parseInt(id_cliente);
        if(gestion.buscador_universal(num2, 1) != -1){
            JOptionPane.showMessageDialog(null, "ERROR - El ID ingresado ya existe.");
            contador++;
            return;
        }
        // condicionales habitual
        habi = habi.toLowerCase();
        if (!habi.matches("si|no") && !habi.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - En el campo 'Habitual', solo se
puede ingresar 'Si' o 'No'");
            contador++;
            return;
        }
        // condicionales cantidad de compras
        if(!comp.matches("[0-9]+") && !comp.equals("")){
            JOptionPane.showMessageDialog(null, "ERROR - La cantidad de compras solo
puede contener numeros enteros.");
            contador++;
            return;
        }
        // condicionales presupuesto
        if (!pesu.matches("[0-9]+([.][0-9]+)?") && !pesu.equals("")) {
            JOptionPane.showMessageDialog(null, "ERROR - El presupuesto solo puede
contener un numero decimal (usar . y no ,).");
```

```
            contador++;
            return;
        }
        int num=Integer.parseInt(comp);
        double num3;
        num3 = Double.parseDouble(pesu);

        Cliente nueva_persona = new Cliente(num2,habi, num,  num3,
nombre,apellido,dni,Email,telefono);
        gestion.agregarCliente(nueva_persona);
        JOptionPane.showMessageDialog(null, "Cliente registrado/a exitosamente.");
        txtNombre.setText("");
        txtApellido.setText("");
        txtDni.setText("");
        txtEmail.setText("");
        txtTelefono.setText("");
        txtCliente.setText("");
        txtHabitual.setText("");
        txtCompras.setText("");
        txtPresupuesto.setText("");
    }
```

El enlace a continuación, corresponde a la documentación de funcionamiento que había empezado el día anterior. Si bien, como mencione, no contiene todo el contenido que le corresponde, contiene captura de todos los posibles mensajes de error que pueden aparecer,

los cuales funcionan gracias a las validaciones que realice (se usa el ejemplo de la ventana “Clientes”). Además, muestra capturas comparativas del aspecto de esa misma ventana, antes y después de mis modificaciones.

Pruebas Codigo POO - Proyecto Logistica - Santiago Agustín Pedraza Clérici

Ese mismo día, tras el punto de control, nos dijeron que todo estaba relativamente bien, pero con algunos detalles a tener en cuenta. Primero que nada, teníamos que empezar de una vez con el sitio web HTML, por otro lado, el profesor Gabriel Tuinyla nos dijo que teníamos muy pocos procedures. No bastaba solo con Agregar, Borrar y Editar, teníamos que hacer mínimo 10-15 procedures por tabla. Por último, nos dijeron que empezáramos con la conexión de Java a MySQL.

Saragusti:28/09

Este dia estuve laburando en colaborar agregando las validaciones de la ventana de reportes tickets y vehículos, ya había puesto algunas previamente pero finalice con las pocas que faltaban en estas ventanas, luego cree el html de crear envios para poder agregar envíos a la lista previamente creado por el auxiliar.

En general eran validaciones de cómo el texto introducido de algunos campos debía ser solo numérico, verificar que los datos de números sean números antes de volverlos a int, y verificar algunas longitudes de datos para que no puedas introducir cualquier tipo de cosa dentro.

30/09

Nos hicieron la devolucion del mes, con los cambios predestinados a octubre siendo estos:

POO:

Implementar máxima cantidad de errores (3) para el inicio de sesión,Implementar conexion a base de datos,CRUD Funcional de todas las tablas,Validaciones para el registro de datos,Limpiar campos al momento de realizar un insert/update/delete,Actualizar tabla post insert/update/delete,Implementar inicio de sesion funcional,Implementar cerrar sesión,Sección para modificar credenciales de usuario,Implementar procedimientos SQL,Actualizar diseño de interfaces (Investigar UI/UX - *opcional utilizar misma paleta de colores de la versión web)

Base de datos

10 - 15 Procedimientos por tabla.

Redes

Implementar servidor de correo (SMTP - POP3),Maqueta digital Sketch UP con las 20 áreas y todos los componentes solicitados en el excel inicial del proyecto.

De Poo ya tenemos lo de máximo 3 errores en el inicio de sesión, el crud funcional en cada tabla, la validación de datos para el registro, teniendo que hacer la actualización post tabla, el inicio de sesion funcional, el cerrar cesión, modificar credenciales, los procedimientos sql, y el diseño de las interfaces.

De la base de datos solo me faltan 7 procedimientos en mis tablas, y a los demás aproximadamente lo mismo y de redes falta la implementación del sistema de correo(Que el servidor ya esta puesto en el lugar, solo falta configurar el servicio del programa en este y en cada computadora) y tambien falta el inicio de la maqueta en sketchup, con las 20 oficinas y los 3000 dispositivos aprox en total, que son los que tocan.

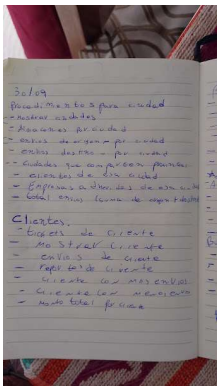
Además de eso recibí las correcciones de la carpeta de campo, que fue solo agregar fotos que demostrarán las actividades que estaba realizando en ese momento como constancia de que lo realizaba y también para tener una vista física de los cambios en el momento hecho, por lo

que estuve la mañana corrigiendo y agregando secciones con imágenes y explicaciones de lo que se visualizaba a las secciones que se consideraban necesarias.

Además hoy estuve actualizando y poniendo al día las actividades realizadas en el gantt y colocando las marcas de colores en cada una de estas, para que quede constancia de los horarios y días laborales con respecto al proyecto interdisciplinario.

En la tarde de ese día plantee los procedimientos restantes para lograr llenar los 10 procedimientos en mis dos tablas, siendo 8 procedimientos adicionales para la tabla ciudades y 7 procedimientos adicionales para la tabla clientes.

Estos siendo: Mostrar ciudades, almacenes por ciudad, envios origen, envios destino, ciudades por provincia, clientes de ciudad, empresas adheridas de ciudad, total de envíos contado origen y destino, por la parte de clientes son: Tickets de cliente, mostrar clientes, envíos de cliente, reportes de cliente, cliente con más y menos envíos, monto total por cliente.



(Foto de como anote todos esos procedimientos en la hoja, siendo planteados en la escuela, para completar y ser de utilidad para el uso de los administradores del sistema sql.)

Esa misma tarde codifique todos los procedimientos adicionales de ciudad y comprobé que funcionase cada uno de ellos, además de que los subí al documento de los procedures generales del grupo.

Todos los envios con origen de esa ciudad:

```
delimiter //
create procedure origen_ciudad(in p_id_ciudad int)
        BEGIN
        select * from envios where envios.Origen=p_id_ciudad;
      end//
      delimiter;
```

Todos los envios con destino de esa ciudad:

```
delimiter //
create procedure destino_ciudad(in p_id_ciudad int)
        BEGIN
        select * from envios where envios.Destino=p_id_ciudad;
      end//
      delimiter;
```

Ciudades por provincia:

```
DELIMITER //
create procedure ciudades_provincia(in P_provincia varchar(100))
        BEGIN
        select * from ciudades WHERE ciudades.Provincia=P_provincia;
```

(Imagen de referencia de cómo fueron quedando los procedimientos, no estando todos pero para tomar como se veían aproximadamente todos)

Xiomara:**30/09/2026**

Este día tuvimos una pre entrega, el balance general que nos dieron los profesores fue bastante positivo dentro de todo. Nos comentaron que el proyecto venía relativamente bien encaminado, pero nos llamaron la atención sobre un par de cosas.

Lo primero fue que teníamos que arrancar si o si con el armado y diseño del sitio web en html para empezar a darle forma a la interfaz en el navegador, por otro lado a la hora de revisar la base de datos, el profesor Gabriel Tuinyla nos marcó que la cantidad de stored procedures que habíamos programado hasta el momento era baja, no alcanzaba únicamente con tener los procedimientos básicos de “Agregar”, “Borrar” y “Editar” registros, teníamos que hacer entre 10 y 15 procedures específicos para cada una de las tablas. Finalmente, para cerrar con las tareas que teníamos que hacer, los profesores nos dijeron que empezáramos con la programación de la conectividad para enlazar el código de Java con la base de datos Mysql, esto era fundamental para que los datos que cargaramos en la interfaz empezaran a guardarse de verdad y el sistema funcionara.

30/09/2026 - 06/10/2026

Durante estos días me encargue de realizar los procedimientos para llegar a la cantidad mínima solicitada, de las mismas tablas a las cuales les hice los primeros procedures previos a la entrega de septiembre(“Tickets”, “Reportes” y “Reportes_ticket”).

Ticket:

Eliminar tickets (usando id cliente)

```
DELIMITER //

CREATE PROCEDURE eliminarTicket2 (in p_idTicket int)
BEGIN
	delete  from Tickets where ID_Cliente=p_idTicket;
END //
DELIMITER ;
```

Buscar tickets (por id cliente)

```
DELIMITER //
CREATE PROCEDURE ticketes_cliente(in t_id_cliente int)
BEGIN
	select * from tickets where tickets.ID_Cliente=t_id_cliente;
END //
DELIMITER ;

 call ticketes_cliente(2);
```

Mostrar tickets

```
delimiter //
create procedure mostrar_tickets()
	BEGIN
	select * from tickets;
END //
DELIMITER ;
```

Mostrar tickets donde el pago sea en efectivo(este mismo procedimiento lo hice con Mercado Pago, Debito, Credito y Transferencia, lo pongo aca ya que es el mismo código solo cambia WHERE metodospago = "Efectivo"; **)**

```
DELIMITER //
```

```
CREATE PROCEDURE metodoPago_efectivo()
BEGIN
 SELECT * FROM tickets WHERE metodospago = "Efectivo";
END //
DELIMITER ;
```

Mostrar tickets donde el total sea menor a 50000

```
DELIMITER //
CREATE PROCEDURE total_menor50000()
BEGIN
  SELECT * FROM tickets where total < 50000;
END //
DELIMITER ;
```

Al sumar estos 9 procedimientos nuevos que hice a los 3 que ya veníamos teniendo listos desde antes (que eran los clásicos de “agregar”, “editar” y “eliminar”), logramos alcanzar un total de 12 procedimientos almacenados asignados a la tabla tickets. Con esto ya dejamos esa parte super completa y cumpliendo con lo que habían pedido los profesores.

Después de terminar eso, me puse a realizar el mismo trabajo con la tabla de reportes. La diferencia acá fue que, al ser una tabla mucho más simple que sólo contiene el campo de id y el detalle, no se podía meter tanta lógica, así que en este caso diseñe solamente 2 procedimientos nuevos. Si sumamos estos 2 a los 3 básicos que ya estaban hechos (“agregar”, “editar” y “eliminar”), nos quedó un total de 5 procedimientos para la tabla de reportes. Para terminar con todos los procedimientos, pasó exactamente lo mismo cuando me tocó trabajar con la tabla reportesTicket. Como compartía una estructura similar y sencilla, le sume 2 procedimientos más a los 3 que ya estaban hechos antes, dejando un total de 5 para esta última tabla.

Reportes:

Buscar Reportes:

```
DELIMITER //
CREATE PROCEDURE buscar_reportess(in id int)
```

```
BEGIN
  select * from reportes WHERE ID_Reporte = id;
END //
DELIMITER ;
```

Mostrar Reportes:

```
delimiter //
create procedure mostrar_reportes()
        BEGIN
        select * from reportes;
END //
DELIMITER ;
```

Reportes_Ticket:

Buscar Reportes_Ticket por id:

```
DELIMITER //
CREATE PROCEDURE buscar_reportesTickets(in id int)
BEGIN
  select * from reportes_ticket WHERE ID_rt = id;
END //
DELIMITER ;
```

Mostrar Reportes_Ticket:

```
delimiter //
create procedure mostrar_reportesTickets()
        BEGIN
        select * from reportes_ticket;
END //
DELIMITER ;
```

1/10

SARAGUSTI:los procedimientos, no estando todos pero para tomar como se veían aproximadamente todos)

El 1 de octubre, después de haber visto los servidores de correo electrónico(Email), con el compañero Zowmir, planteamos cómo iba a ser la configuración de cada uno de los usuarios, al contar con 150 computadoras, ideamos 150 correos electrónicos, usuarios y contraseñas por cada uno de ellos, anotarlos en un documento google para no olvidarnos y para tener centralizada la información referida a aquello.

Además en ese archivo se decía la configuración de los otros servidores, como el ftp y el https para poder acceder a las direcciones planteadas previamente en el desarrollo del servidor.

(Visión de la configuración más sencilla de los otros servidores, sin contar el dns y el dhcp ya que estos no nos conectamos directamente y están para respaldar y facilitar funciones que se podrían hacer de manera manual y uno por uno)

(Como se ve una parte de la seccion de configuracion del servidor, contando con cada uno de los usuarios, contraseña y gmail, el mail y el usuario deben

compartir nombres o la verificación de smtp va a tirar error al momento de querer enviar/redactar un correo)

Siendo que lo único que faltaba era actualizar los datos y plasmarlos en el archivo de cisco packet tracer, ya que no tuvimos tiempo a realizarlo.

2/10

ROSATO:Se inició una etapa de investigación técnica y relevamiento de herramientas de diseño para el modelado de la infraestructura física del proyecto logístico. Se evaluó el entorno de modelado tridimensional SketchUp, utilizando la inteligencia artificial Gemini como soporte interactivo para acelerar el aprendizaje de los comandos de la interfaz, el manejo de escalas y la renderización de los almacenes centrales de distribución.

1. El espacio de trabajo y la navegación

Al abrir un lienzo nuevo, verás tres líneas de colores (los ejes **rojo, verde y azul**) que representan el espacio tridimensional. La clave para no perderte es dominar el mouse: YouTube · Sección Fugada

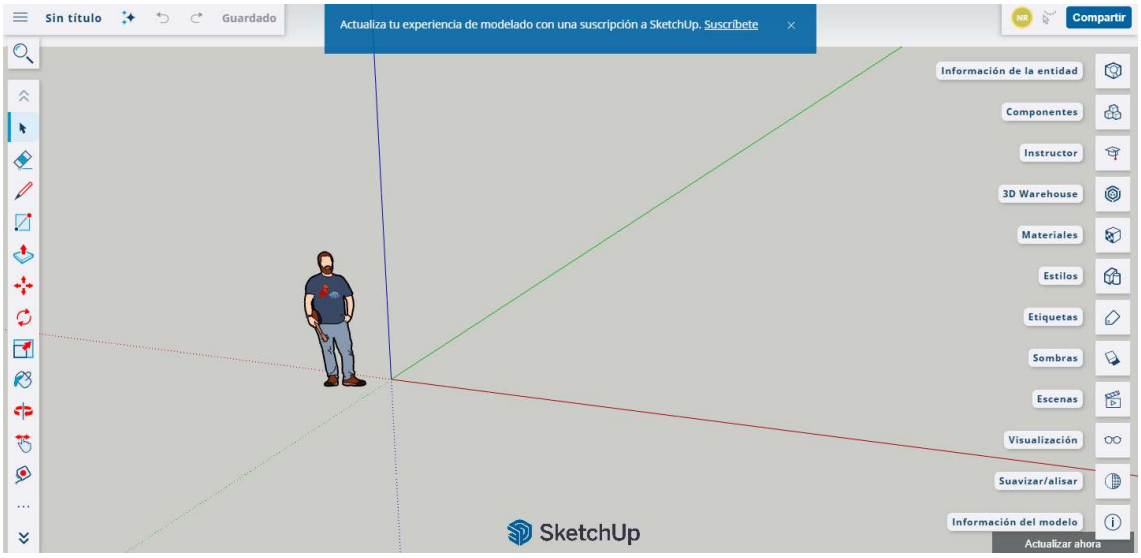
- **Zoom:** Gira la rueda del mouse hacia adelante para acercarte y hacia atrás para alejarte.
- **Orbitar (Girar la vista):** Mantén **presionada la rueda del mouse** y muévelo para rotar alrededor de tu objeto.
- **Desplazar (Moverse lateralmente):** Mantén presionada la tecla **Shift + la rueda del mouse** para mover la pantalla de lado a lado. YouTube · Un Arquitect... +2

---

2. Las herramientas esenciales de dibujo

En la barra de herramientas izquierda o superior encontrarás todo lo necesario para empezar a crear: Domestika +1

- **Seleccionar (Flecha negra / Barra espaciadora):** Sirve para tocar líneas, caras u objetos enteros.
- **Línea (Lápiz / Tecla L):** Dibuja tramos rectos. Al trazar, cuida que la línea cambie al color del eje (rojo, verde o azul) para asegurarte de que estás dibujando de forma derecha en el espacio tridimensional.



3/10

SARAGUSTI:El 3 de octubre codifique el resto de procedimientos restantes, siendo todos los referidos a la tabla clientes, usando inner joins y relaciones para obtener datos más precisos.

Envios de un cliente:

```
delimiter //
create procedure envios_cliente(in p_id_cliente int)
        BEGIN
        select * from envios inner join clientes on
clientes.ID_Cliente=envios.ID_Cliente where clientes.ID_Cliente=p_id_cliente;
      end//
      delimiter;
```

Reportes de cliente:

```
delimiter //
create procedure reportes_de_cliente(in p_id_cliente int)
        BEGIN
        select reportes.ID_Reporte,reportes.detalle from reportes INNER join
reportes_ticket on reportes_ticket.ID_Reporte=reportes.ID_Reporte inner join tickets
on tickets.ID_Ticket=reportes_ticket.ID_Ticket where tickets.ID_Cliente=p_id_cliente;
      end//
      delimiter;
```

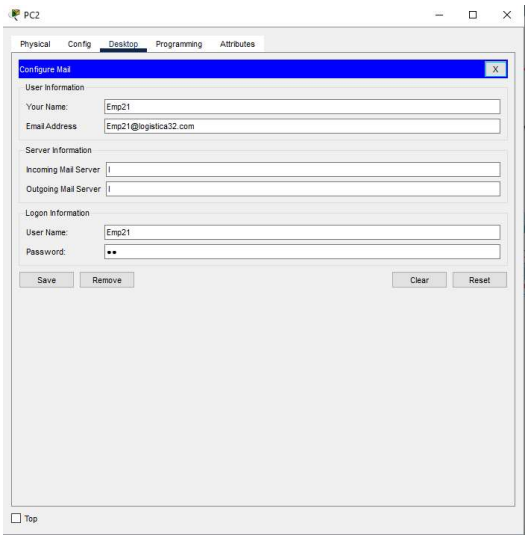
Cliente con mas envios:

```
delimiter //
create procedure cliente_mas_envios()
```

(Referencia visual de cómo se veían algunos de los procedimientos hechos en sql para la tabla clientes)

5/10

SARAGUSTI:El día 5 de septiembre trabajé en la sección de redes, finalizando por completo los 158 dispositivos, 8 más que antes, debido a que no eran 150 perfiles. Agregue de emp 151 a emp 158, configurando cada uno con su respectivo mail y contraseña. Además de eso la ip del servidor del correo, siendo 192.168.2.6 ya se puede golpear usando el dns, usando el nombre de l, siendo una abreviación de logística, debido a que estar escribiendo todas las veces la dirección ip era demasiado largo y además para una persona que usa el sistema sería más fácil de entender. Por lo que está más dirigido al usuario final de este esquema.



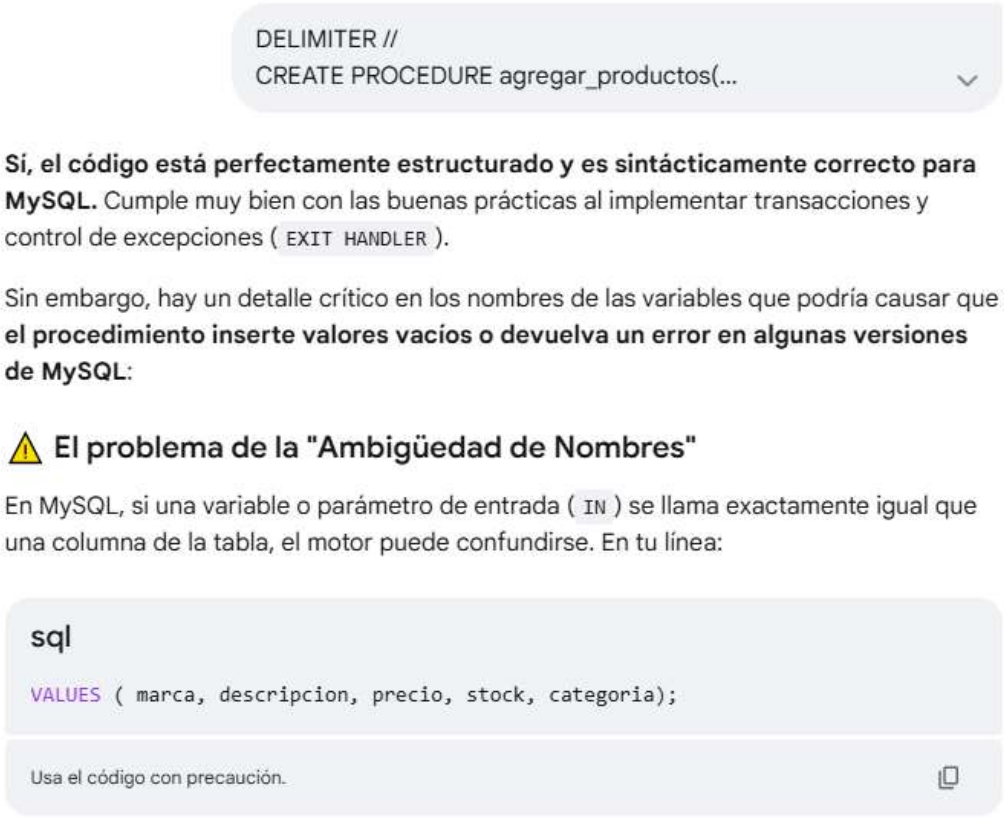
(Ejemplo visual de una de las configuraciones de computadoras, con el servidor de correo aplicado, teniendo en cuenta que se usa en la parte de incoming y de sending l en vez de la ip del servidor)

ROSATO:Se llevó a cabo el diseño final, depuración y codificación de los últimos procedimientos almacenados requeridos para el motor de la base de datos relacional. Durante esta jornada, las tareas se enfocaron en verificar la correcta implementación de las consultas estructuradas de manipulación de datos, asegurando la consistencia de las restricciones de integridad y la optimización de los tiempos de respuesta del servidor en el flujo de información del sistema logístico.

Agregar Productos:

```
DELIMITER //
CREATE PROCEDURE agregar_productos(
   IN marca VARCHAR(100),
   in descripcion varchar(200),
   in precio decimal (10,2),
   in stock int,
   in categoria varchar (50),
   OUT mensaje VARCHAR(200)
)
BEGIN
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      SET mensaje = 'Error: operación revertida';
      ROLLBACK;
   END;

   START TRANSACTION;

   INSERT INTO productos( marca, descripcion,precio , stock,categoria )
   VALUES ( marca, descripcion, precio, stock, categoria);

   COMMIT;
   SET mensaje = 'Hecho con éxito';
END//
DELIMITER ;
```

utilización de la herramienta de la IA gemini para su corrección y comprobación



codigo:
Eliminar productos:

```
delimiter //
create procedure borrar_producto(in p_producto int , out mensaje
varchar(100))
        BEGIN
        DECLARE v_producto int default 0;
       start transaction;
       select count(id_producto) into v_producto
       from productos_paquetes
       where productos_paquetes.ID_Producto = p_producto;

       if v_producto > 0 THEN
```

```
        set mensaje="El producto esta en un paquete";
          ROLLBACK;
       else
        delete from productos
          where id_producto = p_producto;
          COMMIT;
          set mensaje = 'Producto Eliminado con Éxito.';
       end if;
end//
delimiter;
```

Comprobación del mismo

Sí, la lógica de tu código está bien pensada y es segura, ya que usas una transacción y verificas de forma excelente que el producto no esté amarrado a un paquete antes de intentar borrarlo.

Sin embargo, contiene **un error de sintaxis al final que hará que falle al ejecutarlo**, y un par de detalles de consistencia con las transacciones que deberías ajustar.

Aquí tienes el análisis de los detalles y el código corregido:

6/10

ZOWMIR: En esta etapa del proyecto, terminé de desarrollar los procedimientos almacenados correspondientes a la tabla Almacenes. Para realizarlos, primero tuve que pensar qué operaciones podían resultar útiles para administrar la información de esta tabla y facilitar su manejo desde la base de datos.

A partir de eso, se me ocurrió implementar distintos procedimientos que permitieran agregar, modificar, eliminar, mostrar y buscar almacenes, además de ordenarlos alfabéticamente de forma ascendente o descendente. De esta manera, pude completar las operaciones principales necesarias para gestionar los registros de la tabla.

1 procedimiento que ordene los nombres de los almacenes de la A a la Z

Este procedimiento permite mostrar los almacenes ordenados alfabéticamente según su nombre. Para realizarlo utilicé el comando ASC, que permite ordenar los resultados de forma ascendente.

1 procedimiento que ordene los nombres de los almacenes de la Z a la A

Este procedimiento cumple una función similar al anterior, pero muestra los almacenes en orden alfabético inverso. Para esto utilicé el comando DESC, que permite ordenar los resultados de forma descendente.

procedimiento agregar almacen

Permite modificar la información de un almacén que ya se encuentra registrado.

procedimiento editar almacen

procedimiento eliminar almacen

Permite eliminar un almacén de la base de datos.

procedimiento mostrar almacenes

Permite consultar todos los registros que se encuentran guardados en la tabla almacenes.

procedimiento buscar almacen por ID

Permite encontrar un almacén específico utilizando su identificador (ID), lo que facilita la búsqueda de un registro determinado.

Para realizar los procedimientos utilicé diferentes instrucciones de SQL, como SELECT, ORDER BY, ASC y DESC, dependiendo de la operación que necesitaba realizar.

Este trabajo me permitió comprender mejor cómo funcionan los procedimientos almacenados y cómo pueden utilizarse para organizar y simplificar las operaciones dentro de una base de datos.

```
DELIMITER //
CREATE PROCEDURE almacenes_AZ ()
BEGIN
select * from almacenes
order by nombre ASC;
end;
delimiter ;
```

```
DELIMITER //
CREATE PROCEDURE almacenes_ZA ()
BEGIN
select * from almacenes
order by nombre DESC;
end;
delimiter ;
```

AGREGAR ALMACEN:

```
DELIMITER //

CREATE PROCEDURE AgregarAlmacen(
    IN p_direccion VARCHAR(50),
    IN p_ciudad INT,
    IN p_nombre VARCHAR(50)
)
BEGIN
    IF EXISTS (SELECT 1 FROM Ciudades WHERE ID_ciudad = p_ciudad) THEN

        INSERT INTO Almacenes (direccion, ciudad, nombre)
        VALUES (p_direccion, p_ciudad, p_nombre);

    ELSE
```

EDITAR UN ALMACEN:

*

```
DELIMITER //

CREATE PROCEDURE EditarAlmacen(
   IN p_id_almacen INT,
   IN p_direccion VARCHAR(50),
   IN p_ciudad INT,
   IN p_nombre VARCHAR(50)
)
BEGIN
   UPDATE Almacenes
   SET direccion = p_direccion,
      ciudad = p_ciudad,
      nombre = p_nombre
   WHERE id_almacen = p_id_almacen;
END //

DELIMITER ;
```

BORRAR ALMACÉN:

```
DELIMITER //

CREATE PROCEDURE BorrarAlmacen(
   IN p_id_almacen INT
)
BEGIN
   DELETE FROM Almacenes
   WHERE id_almacen = p_id_almacen;
END //
```

MOSTRAR TODOS LOS ALMACENES

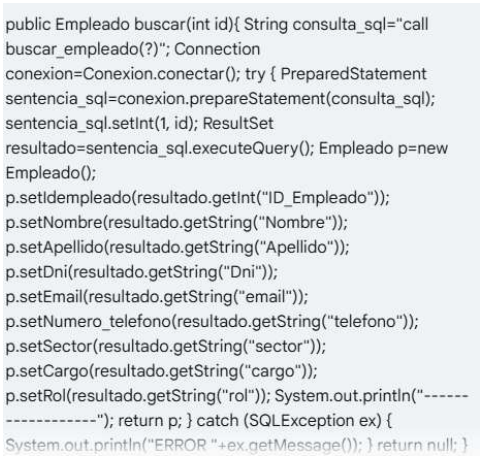
```
DELIMITER //
create procedure mostrar_almacen()
BEGIN
        select *
   from almacenes;
end;
DELIMITER ;
```

BUSCAR UN ALMACÉN

```
DELIMITER //
create PROCEDURE buscar_almacen(in id int)
BEGIN
        SELECT
   from almacenes
   where id = almacenes.ID_Almacen
   end;
        DELIMITER ;
```

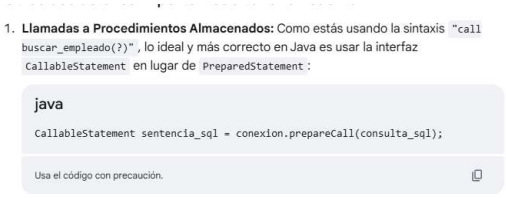
SARAGUSTI:El día 6/09 realice la clase de empleados, además de reconfigurar todas las clases con los mismos componentes de la base de datos.

Para hacer la clase DAO tuve dificultades debido a que como usaba el SQL con procedimientos en vez de las consultas tuve que cambiar algunos comandos que vienen por defecto enseñados por el docente, para eso tuve que subir el codigo que hice con respecto a lo que el profesor subió y luego lo reemplace por las correcciones que me había hecho la ia, una vez corregido en ese momento ya funcionó cada sección del código.



```
public Empleado buscar(int id){ String consulta_sql="call
buscar_empleado(?)"; Connection
conexion=Conexion.conectar(); try { PreparedStatement
sentencia_sql=conexion.prepareStatement(consulta_sql);
sentencia_sql.setInt(1, id); ResultSet
resultado=sentencia_sql.executeQuery(); Empleado p=new
Empleado();
p.setIdempleado(resultado.getInt("ID_Empleado"));
p.setNombre(resultado.getString("Nombre"));
p.setApellido(resultado.getString("Apellido"));
p.setDni(resultado.getString("Dni"));
p.setEmail(resultado.getString("email"));
p.setNumero_telefono(resultado.getString("telefono"));
p.setSector(resultado.getString("sector"));
p.setCargo(resultado.getString("cargo"));
p.setRol(resultado.getString("rol")); System.out.println("------
------------"); return p; } catch (SQLException ex) {
System.out.println("ERROR "+ex.getMessage()); } return null; }
```

```
public Empleado buscar(int id) {
    String consulta_sql = "call buscar_empleado(?)";
    Connection conexion = Conexion.conectar();

    try {
        PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sq
        sentencia_sql.setInt(1, id);
        ResultSet resultado = sentencia_sql.executeQuery();

        // ¡El cambio importante está aquí!
        if (resultado.next()) {
            Empleado p = new Empleado();
            p.setIdempleado(resultado.getInt("ID_Empleado"));
            p.setNombre(resultado.getString("Nombre"));
            p.setApellido(resultado.getString("Apellido"));
            p.setDni(resultado.getString("Dni"));
            p.setEmail(resultado.getString("email"));
            p.setNumero_telefono(resultado.getString("telefono"));
            p.setSector(resultado.getString("sector"));
            p.setCargo(resultado.getString("cargo"));
```



1. **Llamadas a Procedimientos Almacenados:** Como estás usando la sintaxis `"call buscar_empleado(?)"`, lo ideal y más correcto en Java es usar la interfaz `CallableStatement` en lugar de `PreparedStatement`:

java

```
CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
```

Usa el código con precaución.

(La primera foto indica cómo se veía el fragmento de código que no sabía porque daba error al ejecutar, la segunda foto muestra la adaptación que realizó la ia que no modifican muchos aspectos del original, y la tercera foto es otra corrección que hizo la ia referido a la manera de realizar llamados a la base de datos.) De las clases comunes lo único que hice fue modificar el constructor, siendo que en algunos los deje vacíos y en otros casos, reemplace borrando solo el id debido a que este lo da la base de datos.

PEDRAZA: realice todos los restantes correspondientes a las tablas Empleados y Vehículos, las mismas a las cuales realice los primeros procedures previos a la entrega de septiembre. Para la tabla empleados, realice los siguientes trece procedures:

```
202     -- mostrar empleados
203 •   drop procedure if exists mostrar_empleados;
204     DELIMITER //
205 •   create procedure mostrar_empleados()
206     begin
207     select * from empleados;
208     end //
209     DELIMITER ;
210 •   call mostrar_empleados();
211
212     -- buscar empleados
213     delimiter //
214 •   create procedure buscar_empleado(in p_id_empleado int)
215         begin
216             select * from empleados where ID_Empleado=p_id_empleado;
217         end//
218     delimiter;
219
220     -- mostrar empleado mas viejo
221     drop procedure if exists empleado_mas_viejo;
222     DELIMITER //
223 •   create procedure empleado_mas_viejo()
224     begin
225     select * from Empleados
226     where dni = (select min(dni) from empleados);
227     end //
228     DELIMITER ;
```

```sql
-- mostrar empleados del sector ‘’transporte’’
drop procedure if exists empleados_transporte;
DELIMITER //
create procedure empleados_transporte()
begin
select * from Empleados
where Sector = "Transporte";
end //
DELIMITER ;

-- mostrar empleados del sector “deposito”
drop procedure if exists empleados_deposito;
DELIMITER //
create procedure empleados_deposito()
begin
select * from Empleados
where Sector = "Deposito";
end //
DELIMITER ;

-- mostrar empleados del sector “administracion”
drop procedure if exists empleados_administracion;
DELIMITER //
create procedure empleados_administracion()
begin
select * from Empleados
where Sector = "Administracion";
end //
DELIMITER ;
```

```sql
-- mostrar empleados del sector “atencion al cliente”
drop procedure if exists empleados_atencion;
DELIMITER //
create procedure empleados_atencion()
begin
select * from Empleados
where Sector = "Atencion al cliente";
end //
DELIMITER ;

-- mostrar empleados del sector “finanzas”
drop procedure if exists empleados_finanzas;
DELIMITER //
create procedure empleados_finanzas()
begin
select * from Empleados
where Sector = "Finanzas";
end //
DELIMITER ;

-- mostrar empleados del sector “mantenimiento”
drop procedure if exists empleados_mantenimiento;
DELIMITER //
create procedure empleados_mantenimiento()
begin
select * from Empleados
where Sector = "Mantenimiento";
end //
DELIMITER ;
```

```sql
-- mostrar empleados con rol “lider”
drop procedure if exists empleados_lideres;
DELIMITER //
create procedure empleados_lideres()
begin
select * from Empleados
where Rol = "Lider";
end //
DELIMITER ;

-- mostrar empleados con rol “empleado comun”
drop procedure if exists empleados_comunes;
DELIMITER //
create procedure empleados_comunes()
begin
select * from Empleados
where Rol = "Empleado comun";
end //
DELIMITER ;

-- mostrar empleados con rol “empleado con beneficios”
drop procedure if exists empleados_beneficiados;
DELIMITER //
create procedure empleados_beneficiados()
begin
select * from Empleados
where Rol= "Empleado con beneficios";
end //
DELIMITER ;
```

```sql
-- mostrar empleados con rol “empleado temporal”
drop procedure if exists empleados_temporales;
DELIMITER //
create procedure empleados_temporales()
begin
select * from Empleados
where Rol= "Empleado temporal";
end //
DELIMITER ;
```

Sumados estos 13 procedures a los 3 realizados previamente correspondientes a “Agregar”, “Editar” y “Eliminar”, queda un total de 16 procedures para la tabla Empleados.

Lo mismo hice con la tabla vehículos, solo que en lugar de 13 procedures nuevos, realice solo 8. Sumándole los 3 previos, nos da un total de 11 procedures para la tabla.

Nuevos procedures de la tabla Vehículos (además de buscar y mostrar que son iguales a los de Empleados):

```
-- buscar vehículos mediante el id del empleado
drop procedure if exists buscar_vehiculo_por_empleado;
DELIMITER //
create procedure buscar_vehiculo_por_empleado(in id_emp_ing int)
begin
select * from vehículos
where ID_Empleado = id_emp_ing;
end //
DELIMITER ;

-- buscar vehículos mediante la marca
drop procedure if exists buscar_vehiculo_por_marca;
DELIMITER //
create procedure buscar_vehiculo_por_marca(in marca_ing varchar(100))
begin
select * from vehículos
where Marca = marca_ing;
end //
DELIMITER ;

-- buscar vehículos mediante el modelo
drop procedure if exists buscar_vehiculo_por_modelo;
DELIMITER //
create procedure buscar_vehiculo_por_modelo(in modelo_ing varchar(100))
begin
select * from vehículos
where Modelo = modelo_ing;
end //
DELIMITER ;
```

```
362 •  -- mostrar vehiculos modernos (fabricados en 2020 o posterior)
363    drop procedure if exists vehiculos_modernos;
364    DELIMITER //
365 •  create procedure vehiculos_modernos()
366    begin
367    select * from Vehiculos
368    where anio >= 2020;
369    end //
370    DELIMITER ;
371
372 •  -- mostrar vehiculos economicos (precio menor o igual a 15.000.000)
373    drop procedure if exists vehiculos_economicos;
374    DELIMITER //
375 •  create procedure vehiculos_economicos()
376    begin
377    select * from Vehiculos
378    where precio <= 15000000;
379    end //
380    DELIMITER ;
```

UREÑA: Durante estos días me encargue de realizar los procedimientos para llegar a la cantidad mínima solicitada, de las mismas tablas a las cuales les hice los primeros procedures previos a la entrega de septiembre(“Tickets”, “Reportes” y “Reportes_ticket”).

Ticket:

Eliminar tickets (usando id cliente)

```
DELIMITER //

CREATE PROCEDURE eliminarTicket2 (in p_idTicket int)
BEGIN
        delete  from Tickets where ID_Cliente=p_idTicket;
END //
DELIMITER ;
```

Buscar tickets (por id cliente)

```
DELIMITER //
```

```
CREATE PROCEDURE ticketes_cliente(in t_id_cliente int)
BEGIN
	select * from tickets where tickets.ID_Cliente=t_id_cliente;
END //
DELIMITER ;

 call ticketes_cliente(2);
```

Mostrar tickets

```
delimiter //
create procedure mostrar_tickets()
	BEGIN
	select * from tickets;
END //
DELIMITER ;
```

Mostrar tickets donde el pago sea en efectivo(este mismo procedimiento lo hice con Mercado Pago, Debito, Credito y Transferencia, lo pongo aca ya que es el mismo código solo cambia WHERE metodospago = "Efectivo"; **)**

```
DELIMITER //
CREATE PROCEDURE metodoPago_efectivo()
BEGIN
 SELECT * FROM tickets WHERE metodospago = "Efectivo";
END //
DELIMITER ;
```

Mostrar tickets donde el total sea menor a 50000

```
DELIMITER //
CREATE PROCEDURE total_menor50000()
BEGIN
  SELECT * FROM tickets where total < 50000;
END //
DELIMITER ;
```

Al sumar estos 9 procedimientos nuevos que hice a los 3 que ya veníamos teniendo listos desde antes (que eran los clásicos de “agregar”, “editar” y

“eliminar”), logramos alcanzar un total de 12 procedimientos almacenados asignados a la tabla tickets. Con esto ya dejamos esa parte super completa y cumpliendo con lo que habían pedido los profesores.

Después de terminar eso, me puse a realizar el mismo trabajo con la tabla de reportes. La diferencia acá fue que, al ser una tabla mucho más simple que sólo contiene el campo de id y el detalle, no se podía meter tanta lógica, así que en este caso diseñe solamente 2 procedimientos nuevos. Si sumamos estos 2 a los 3 básicos que ya estaban hechos (“agregar”, “editar” y “eliminar”), nos quedó un total de 5 procedimientos para la tabla de reportes. Para terminar con todos los procedimientos, pasó exactamente lo mismo cuando me tocó trabajar con la tabla reportesTicket. Como compartía una estructura similar y sencilla, le sume 2 procedimientos más a los 3 que ya estaban hechos antes, dejando un total de 5 para esta última tabla.

Reportes:

Buscar Reportes:

```
DELIMITER //
CREATE PROCEDURE buscar_reportess(in id int)
BEGIN
  select * from reportes WHERE ID_Reporte = id;
END //
DELIMITER ;
```

Mostrar Reportes:

```
delimiter //
create procedure mostrar_reportes()
        BEGIN
        select * from reportes;
END //
DELIMITER ;
```

Reportes_Ticket:

Buscar Reportes_Ticket por id:

```
DELIMITER //
CREATE PROCEDURE buscar_reportesTickets(in id int)
BEGIN
  select * from reportes_ticket WHERE ID_rt = id;
END //
DELIMITER ;
```

Mostrar Reportes_Ticket:

```
delimiter //
create procedure mostrar_reportesTickets()
        BEGIN
        select * from reportes_ticket;
END //
DELIMITER ;
```

ROSATO:seguimiento de los procedimientos de base de datos

codigo:
Editar Productos :

```
DELIMITER //

CREATE PROCEDURE editar_productos(
   IN p_id_producto INT,
   IN p_Marca VARCHAR(200),
   IN p_Descripcion VARCHAR(200),
   IN p_Precio DECIMAL(10,2),
   IN p_Stock INT,
   IN p_Categoria VARCHAR(200),
```

```
    OUT p_mensaje VARCHAR(200)
)
BEGIN
    DECLARE auxiliar VARCHAR(100);

    START TRANSACTION;
        SELECT productos.Marca INTO auxiliar FROM productos WHERE
productos.ID_Producto = p_id_producto;

        IF auxiliar IS NULL THEN
            SET p_mensaje = 'No existe el producto';
            ROLLBACK;
        ELSE
            UPDATE productos
            SET
                Marca = IFNULL(p_Marca, Marca),
                Descripcion = IFNULL(p_Descripcion, Descripcion),
                Precio = IFNULL(p_Precio, Precio),
                Stock = IFNULL(p_Stock, Stock),
                Categoria = IFNULL(p_Categoria, Categoria)
            WHERE ID_Producto = p_id_producto;

            COMMIT;
            SET p_mensaje = 'Producto actualizado correctamente';
        END IF;
END //

DELIMITER ;
```

comprobación

Este código está **muy bien estructurado y la lógica para permitir actualizaciones parciales usando `sql IFNULL()` es excelente.** Te permite enviar solo los campos que quieres cambiar y mantener los valores actuales en los demás.

Sin embargo, tiene **un problema lógico importante que hará que falle cuando intentes actualizar un producto si dejas la marca igual o si esta viene vacía**, además de un detalle con las transacciones.

Aquí tienes los puntos a corregir y la versión optimizada:

codigo:

Agregar Productos_Paquetes:

```
DELIMITER //
CREATE PROCEDURE agregar_productos_paquetes(
   IN ID_Producto int,
   in ID_Paquete int,
   in cantidad int,
   OUT mensaje VARCHAR(200)
)
BEGIN
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      SET mensaje = 'Error: operación revertida';
      ROLLBACK;
   END;

   START TRANSACTION;

   INSERT INTO productos_paquetes ( ID_Producto, ID_Paquete
,cantidad)
   VALUES (ID_Productos, ID_Paquetes ,cantidad);

   COMMIT;
   SET mensaje = 'Hecho con éxito';
END//
DELIMITER ;
```

Este procedimiento tiene la estructura correcta y el manejo de excepciones está muy bien planteado, pero **va a fallar inmediatamente al ejecutarlo debido a errores en los nombres de las variables**.

Tiene exactamente los mismos dos detalles que corregimos en tus primeros procedimientos: **errores de dedo (typos)** en los nombres de las variables del `INSERT` y la **ambigüedad de nombres** con las columnas de tu tabla.

⚠ **Errores específicos a corregir:**

1. **Variables mal escritas (Plural vs. Singular):** En la declaración pusiste `IN ID_Producto` e `IN ID_Paquete`. Sin embargo, abajo en el `VALUES` escribiste `ID_Productos` e `ID_Paquetes` (con **S** al final). MySQL te dará un error diciendo que esas variables no existen.
2. **Ambigüedad de nombres:** Al llamarse tus parámetros exactamente igual que las columnas de la tabla (`ID_Producto`, `ID_Paquete`, `cantidad`), MySQL se va a confundir y podría intentar insertar el valor de la columna en sí misma.

Corrección

```
DELIMITER //

CREATE PROCEDURE agregar_productos_paquetes(
   IN p_id_producto INT,
   IN p_id_paquete INT,
   IN p_cantidad INT,
   OUT mensaje VARCHAR(200)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      SET mensaje = 'Error: operación revertida. Verifique que el producto
y el paquete existan.';
      ROLLBACK;
   END;

   START TRANSACTION;
```

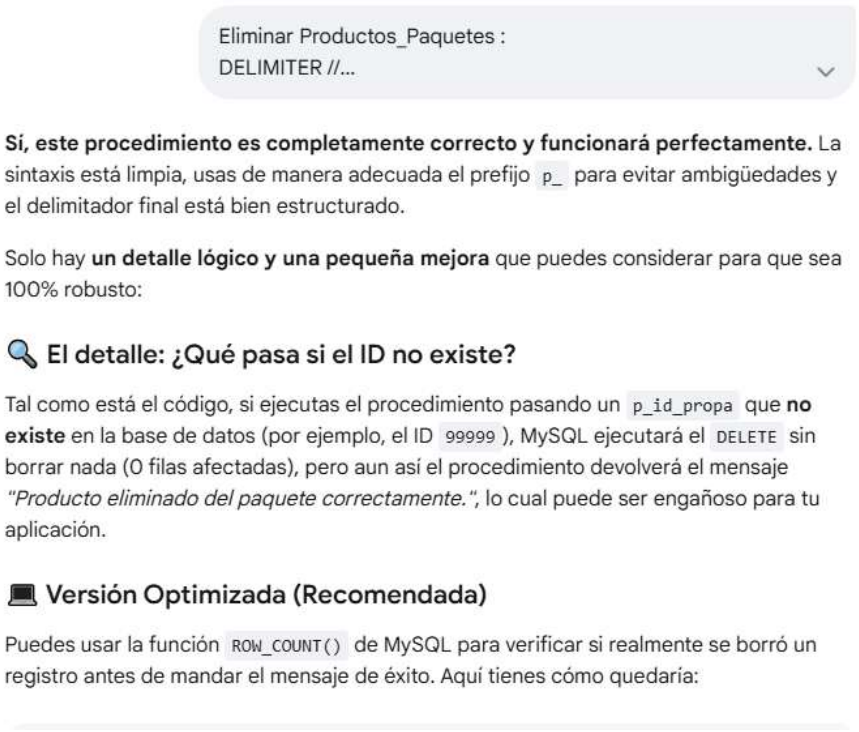
```
    INSERT INTO productos_paquetes (ID_Producto, ID_Paquete,
cantidad)
    VALUES (p_id_producto, p_id_paquete, p_cantidad);

    COMMIT;
    SET mensaje = 'Hecho con éxito';
END//

DELIMITER ;
```

codigo:

Eliminar Productos_Paquetes :

```
DELIMITER //
CREATE PROCEDURE borrar_producto_paquete(IN p_id_propa INT,
OUT mensaje VARCHAR(100))
BEGIN
        DELETE FROM Productos_Paquetes
        WHERE ID_Propa = p_id_propa;
        SET mensaje = 'Producto eliminado del paquete correctamente.';
END //

DELIMITER ;
```

Comprobacion:



7/10

ROSATO:El líder del equipo técnico, Santiago Goldfard, coordinó la asignación de tareas correspondientes al desarrollo de las interfaces gráficas de usuario para el módulo de Programación Orientada a Objetos. Mi contribución específica en esta etapa consistió en asumir la responsabilidad del diseño, maquetación y codificación de las ventanas operativas para los módulos de Empresas Adheridas y Control de Vehículos de la flota logística.

UREÑA: Durante esta jornada, nuestro líder de equipo Golfard, llevó a cabo la distribución de las tareas de desarrollo en Java, asignó a cada integrante del grupo la responsabilidad de programar el diseño de una ventana. Un detalle fundamental de esta etapa es que aca ya estábamos implementando la conexión real y directa con la base de datos por lo que el objetivo principal era lograr que cada ventana se acoplara y adaptara a la perfección con las tablas correspondientes y con los parámetros de entrada que ya teníamos definidos en la base de datos.

En mi caso, me tocó el desarrollo completo de la ventana Vehículo. Las funcionalidades que tenía que tener esta ventana son las de agregar, buscar, mostrar, editar y borrar.

PEDRAZA:El día 7/10, Goldfarb y Saragusti nos asignaron a cada integrante realizar de 1 a 2 clases DAO y las ventanas correspondientes con nuevas condiciones, para el programa en JAVA. A mi me asignaron las clases DAO y las ventanas de Reporte y Ticket.

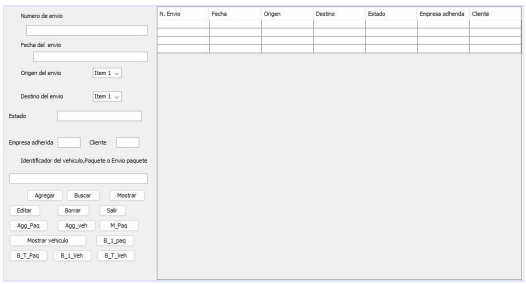
Las condiciones que me dio Saragusti para las ventanas eran las siguientes:

Ventana reporte:
Agregar, buscar(por id),editar, borrar, mostrar
Ademas:
Agregar tickets
Mostrar tickets
Borrar 1 ticket
Borrar todos los tickets
Preguntenme para mas profundidad en estos
Ventana tickets
Buscar,mostrar,agregar, borrar,editar
Ademas de borrar todos los tickets usando id cliente

SARAGUSTI:El 7 de septiembre realice la clase DAO del cliente, siendo que tiene los métodos de buscar, eliminar, modificar, mostrar, requeridos por cada tabla.

Además este dia realice la clase DAO de envío, conteniendo las opciones de mostrar envios, borrar envios, editar envios, buscar envios, y como opciones adicionales tenemos agregar paquete/vehiculo, mostrar paquetes, mostrar vehiculos, borrar 1 paquete/vehiculo, borrar todos los paquetes/vehículos. Además reemplace la parte en la que ponías texto en origen y

destino cambiándolo por una combo box que contiene las opciones de las ciudades haciendo uso de un procedure para llamar los nombres de las ciudades y luego usando un procedimiento que vuelve los nombres a ints, debido a que la base de datos solo usa ints para relacionar, por lo que en la checkbox seleccionar una ciudad y el programa automáticamente te la vuelve en un número.



(Como se ve la ventana completa con todas las opciones y las checkboxes)

(Como se ven las opciones con texto en la ventana de envíos)

(El código con el cual primero se obtiene la lista de ciudades, y luego con el nombre de la ciudad se obtiene el identificador numérico de la ciudad, este proceso se hace automáticamente cuando)

8/10

GOLDFARB:Actividades Realizadas:

Hoy hable con Benjamin sobre los horarios que íbamos a usar para dedicarnos a las cosas restantes. Estas son las cosas que nos faltan:

-Manual Instalación y Despliegue.

-Carpeta de campo grupal.

-Triggers y procedures.

-Ventanas HTML.

-Clases DAO.

-Conectar Java con HTML.

-Maqueta 3D.

Decidí hacer los siguientes Triggers:

Bloquear cambios en envíos entregados: Un trigger *BEFORE UPDATE* en Envíos que impida cualquier modificación a los datos del envío (como cambiar el destino o el cliente) una vez que su estado sea "Entregado" o "Cancelado".

1. **Estandarizar Dirección:** Un trigger *BEFORE INSERT* en Empresa_Adherida que convierte todo el texto de direccion a minúsculas o mayúsculas para unificar el formato urbano.

2. **Log de empresas eliminadas:** Un trigger *AFTER DELETE* en Empresa_Adherida que guarde los datos de la empresa que se dio de baja para conservar el registro histórico de proveedores.

3. **Historial de mudanzas de empresas:** Un trigger *AFTER UPDATE* en Empresa_Adherida que registre en una tabla de auditoría los cambios en la columna direccion.

4. **Historial de cambios de teléfono:**
   Un trigger *AFTER UPDATE* en Empresa_Adherida que guarde el teléfono viejo y el nuevo si la empresa actualiza su contacto.
5. **Historial de cambios de CUIT:** Un trigger *AFTER UPDATE* en Empresa_Adherida que deje constancia si una empresa modificó su CUIT legal.

Actividades Realizadas:

Hoy vino Benjamin para contarme que iba a querer explicarle algunas cosas que hizo en Java a mis compañeros. Le dije que lo haga a la tarde en una llamada de Discord para que los chicos sigan con sus cosas. Empecé a hacer mis triggers de base de datos. Agregue las tablas “Envio_eliminado” y “Empresa_adherida_eliminada”

```sql
drop database if exists Logistica32;
create database Logistica32;
use Logistica32;

create table Empresa_Adherida(
    ID_empresa int unsigned auto_increment primary key,
    nombre varchar(50),
    cuit varchar(13),
    direccion varchar(100),
    telefono int (10),
    rubro varchar(100),
    duenio varchar(50)
);

create table Empresa_Adherida_eliminada(
    ID_empresa_eliminada int unsigned auto_increment primary key,
    nombre_eliminado varchar(50),
    cuit_eliminado varchar(13),
    direccion_eliminada varchar(100),
    telefono_eliminado int (10),
    rubro_eliminado varchar(100),
    duenio_eliminado varchar(50)
);
```

```sql
create table Envios(
ID_Envio int unsigned auto_increment primary key,
Fecha date,
Origen int unsigned,
Destino int unsigned,
ID_Empresa int unsigned,
estado varchar(50),
ID_Cliente int unsigned,
foreign key (Origen) references Ciudades(ID_Ciudad),
foreign key (Destino) references Ciudades(ID_Ciudad),
foreign key (ID_Empresa) references Empresa_Adherida(ID_Empresa),
foreign key (ID_Cliente) references Clientes(ID_Cliente)
);

create table Envios_eliminados(
ID_Envio_eliminado int unsigned auto_increment primary key,
Fecha_eliminado date,
Origen_eliminado int unsigned,
Destino_eliminado int unsigned,
ID_Empresa_eliminado int unsigned,
estado_eliminado varchar(50),
ID_Cliente_eliminado int unsigned,
foreign key (Origen_eliminado) references Ciudades(ID_Ciudad),
foreign key (Destino_eliminado) references Ciudades(ID_Ciudad),
foreign key (ID_Empresa_eliminado) references Empresa_Adherida(ID_Empresa),
foreign key (ID_Cliente_eliminado) references Clientes(ID_Cliente)
);
```

Esto ya que cuando eliminemos una empresa o un envio, vamos a querer guardarlo antes de que desaparezca completamente del sistema.

Los triggers hechos son:

Bloquear cambios en envíos entregados: Un trigger *BEFORE UPDATE* en Envíos que impida cualquier modificación a los datos del envío (como cambiar el destino o el cliente) una vez que su estado sea "Entregado" o "Cancelado".

```
delimiter //
create trigger bloquear_cambios_pedidos_finalizados
before update
on envios
begin
if old.estado = "cancelado" and "entregado" then
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'El pedido ya fue finalizado. No se pueden alterar los
datos.';

end if ;
end //
delimiter ;
```

Estandarizar Dirección: Un trigger *BEFORE INSERT* en Empresa_Adherida que convierte todo el texto de direccion a minúsculas o mayúsculas para unificar el formato urbano.

```
delimiter //
create trigger pasar_direccion_a_minusculas
before insert on empresa_adherida
for each ROW
BEGIN
        set new.direccion = lower(new.direccion);
END //
delimiter ;
```

Log de empresas eliminadas: Un trigger *AFTER DELETE* en Empresa_Adherida que guarde los datos de la empresa que se dio de baja para conservar el registro histórico de proveedores.

```
DELIMITER //
create trigger guardar_empresas_borradas
AFTER delete on empresa_adherida
for each ROW
BEGIN
insert into empresa_adherida_eliminada (ID_empresa_eliminada,
nombre_eliminado, cuit_eliminado, direccion_eliminada, telefono_eliminado,
rubro_eliminado, duenio_eliminado)

values (old.ID_empresa, old.nombre, old.cuit, old.direccion, old.telefono, old.rubro,
old.duenio);
end //
DELIMITER ;
```

Junto con Federico Bouzon, discutí que Triggers estaban bien y cuales estaban de sobra. Aun así, los deje anotados por si acaso y mañana, Viernes, le pregunto a Gabriel.

Ahora, me pondré a trabajar en las clases DAO de Java.

ROSATO:Se dio continuidad al desarrollo y programación de la capa de persistencia en la base de datos relacional. Durante esta jornada, me enfoqué en la creación de rutinas avanzadas de consulta mediante procedimientos almacenados.

Buscar productos por fecha de ensamble

```
drop procedure if exists buscar_producto_fecha_ensamble;
DELIMITER //
create procedure buscar_producto_fecha_ensamble(in fecha_ensamble_ing
varchar(100))
begin
select * from fecha_ensamble
where Precio = fecha_ensamble_ing ;
```

```
end //
DELIMITER ;
```

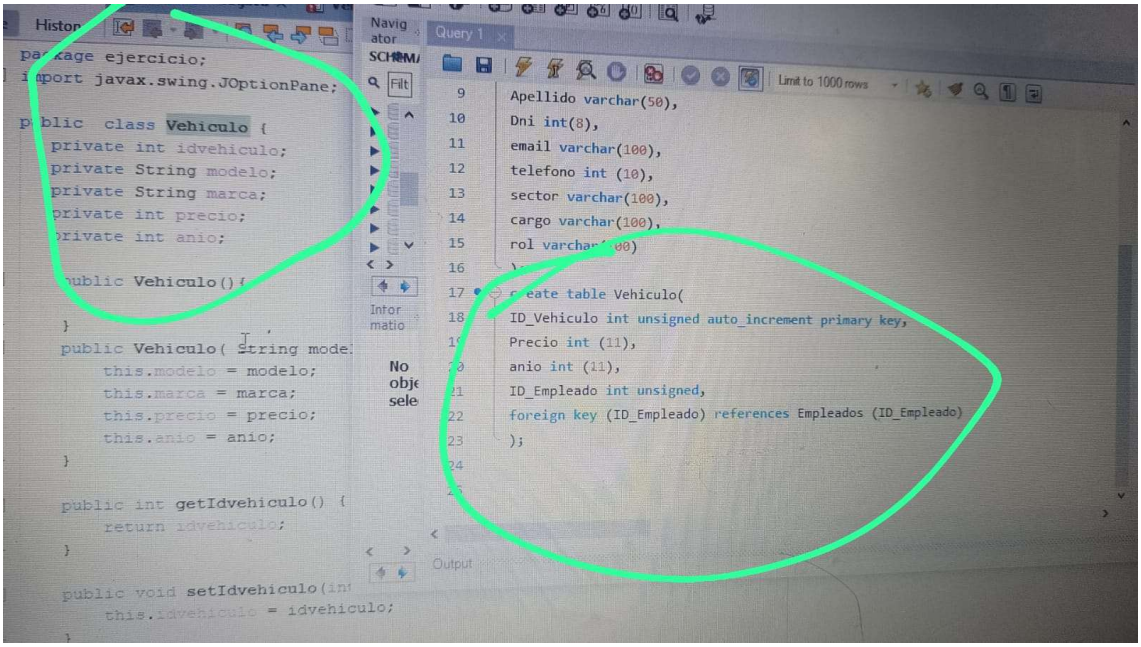
Buscar productos por precio

```
drop procedure if exists buscar_producto_por_precio;
DELIMITER //
create procedure buscar_producto_por_precio(in precio_ing varchar(100))
begin
select * from producto
where Precio = precio_ing;
end //
DELIMITER ;
```

UREÑA:Durante estos días, estuve metida de lleno con el desarrollo y la programación de la ventana que me había tocado, que era la de Vehículos. Sabiendo que venía bastante complicada con los tiempos y que el margen de entrega era ajustado, decidí arrancar lo antes posible para intentar adelantar. Sin embargo, a pesar del esfuerzo, se me presentaron varias complicaciones en el medio que me impidieron terminar todo a tiempo. La fecha límite original que teníamos pautada para la entrega era el domingo, pero debido a estos contratiempos, no me quedó otra opción que retrasarme un poco y terminar entregando todo el lunes 12.

El principal conflicto que tuve apareció apenas arranque con el código. Al ponerme a revisar en detalle para empezar a enlazar las pantallas, me di cuenta de que los atributos que definió alguno de mis compañeros para la clase “Vehículos” en Java no coinciden para nada con los campos que tenía la tabla de “vehículos” en la base

de datos de MySQL. Había diferencias en los nombres y en la cantidad de datos, lo que hacía imposible que la información viaje de un lado al otro.



Para resolver esto me comunique con mis compañeros Golfarb y Benjamin. Al principio yo pensaba que tenía que modificar la base de datos, ya que faltaban los atributos modelo y marca pero no, mis compañeros me dijeron que estos dos atributos estaban agregados al final de todas las tablas con el alter table. Lo único que tuve que agregar fue un atributo en Java, IdEmpleado ya que no estaba, como consecuencia también tuve que modificar el constructor. Otra cosa que también tuve que agregar fue el constructor vacío para después poder usarlo en la clase vehiculosDAO.

Una vez resuelto eso pude avanzar con la ventana VehiculoDAO quedando así:

```
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
```

```
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class VehiculoDAO {
    public void insertar(Vehiculo v)throws SQLException {
        String consulta_sql = "CALL crear_vehiculo(?, ?, ?, ?, ?, ?)";
        try{
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

            sentencia_sql.setInt(1, v.getPrecio());
            sentencia_sql.setInt(2, v.getAnio());
            sentencia_sql.setInt(3, v.getIdEmpleado());
            sentencia_sql.setString(4, v.getModelo());
            sentencia_sql.setString(5, v.getMarca());
            sentencia_sql.registerOutParameter(6, java.sql.Types.VARCHAR);
            sentencia_sql.executeUpdate();
            String mensajeRespuesta = sentencia_sql.getString(6);
            System.out.println("Respuesta del servidor: " + mensajeRespuesta);
            System.out.println("Vehiculo guardado correctamente");
        }catch(SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }
```

```
}

public ArrayList <Vehiculo> listar(){
    ArrayList <Vehiculo> lista_vehiculo = new ArrayList<>();
    String consulta_sql="call mostrar_vehiculos()";
    try{
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
        ResultSet resultado=sentencia_sql.executeQuery();
        while (resultado.next()){
            Vehiculo v = new Vehiculo();

            v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
            v.setPrecio(resultado.getInt("Precio"));
            v.setAnio(resultado.getInt("anio"));
            v.setIdEmpleado(resultado.getInt("idEmpleado"));
            v.setModelo(resultado.getString("Modelo"));
            v.setMarca(resultado.getString("Marca"));

            lista_vehiculo.add(v);
            System.out.println("~~~~~~~~~~~~~~~~~~");
        }
    }catch (SQLException ex){
        System.out.println("ERROR "+ ex.getMessage());
    }
```

```
    return lista_vehiculo;
}

public void actualizar (Vehiculo v){
     String consulta_sql = "call editar_vehiculos (?, ?, ?, ?, ?, ?, ?)";
    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setInt(1, v.getIdvehiculo());
        sentencia_sql.setInt(2, v.getPrecio());
        sentencia_sql.setInt(3, v.getAnio());
        sentencia_sql.setInt(4, v.getIdEmpleado());
        sentencia_sql.setString(5, v.getModelo());
        sentencia_sql.setString(6, v.getMarca());

        sentencia_sql.registerOutParameter(7, java.sql.Types.VARCHAR);
        sentencia_sql.executeUpdate();
        String respuesta = sentencia_sql.getString(7);
        System.out.println("Respuesta de la BD: " + respuesta);
        System.out.println("Datos actualizados exitoamnete");

    }catch (SQLException ex){
        System.out.println("ERROR "+ ex.getMessage());
    }
```

```
}

public Vehiculo buscar(int id) {
String consulta_sql = "call buscar_vehiculo(?)";
Connection conexion = Conexion.conectar();
try {
    CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
    sentencia_sql.setInt(1, id);
    ResultSet resultado = sentencia_sql.executeQuery();
    if (resultado.next()) {
        Vehiculo v =new Vehiculo();
            v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
            v.setPrecio(resultado.getInt("Precio"));
            v.setAnio(resultado.getInt("anio"));
            v.setIdEmpleado(resultado.getInt("idEmpleado"));
            v.setModelo(resultado.getString("Modelo"));
            v.setMarca(resultado.getString("Marca"));

        System.out.println("------------------");
        return v;
    } else {
        System.out.println("No se encontró ningún vehiculo con el ID: " + id);
    }
} catch (SQLException ex) {
    System.out.println("ERROR " + ex.getMessage());
```

```
    }
    return null;
}
    public void eliminar(int id) {
    String consulta_sql = "call eliminar_vehiculo (?, ?)";
    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setInt(1, id);
        sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);
        sentencia_sql.executeUpdate();

        String respuesta = sentencia_sql.getString(2);
        System.out.println("Respuesta de la BD: " + respuesta);
        System.out.println("Vehiculo eliminado");

    } catch (SQLException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
}
}
```

Después continúe haciendo la ventana y desarrollando el código de cada uno de los botones(no lo pongo ya que es muy largo). La ventana me quedo asi: